1.4.3 Chinese Remainder Theorem

The Chinese Remainder Theorem (which will be referred to as CRT in the rest of this article) was discovered by Chinese mathematician Sun Zi.

Formulation

Let $m = m_1 \cdot m_2 \cdots m_k$, where m_i are pairwise coprime. In addition to m_i, we are also given a system of congruences

$$\begin{cases} a &\equiv a_1 \pmod{m_1} \\ a &\equiv a_2 \pmod{m_2} \\ &\vdots \\ a &\equiv a_k \pmod{m_k} \end{cases}$$

where a_i are some given constants. The original form of CRT then states that the given system of congruences always has *one and exactly one* solution modulo m.

E.g. the system of congruences

$$\begin{cases} a &\equiv 2 \pmod{3} \\ a &\equiv 3 \pmod{5} \\ a &\equiv 2 \pmod{7} \end{cases}$$

has the solution 23 modulo 105, because $23 \bmod 3 = 2$, $23 \bmod 5 = 3$, and $23 \bmod 7 = 2$. We can write down every solution as $23 + 105 \cdot k$ for $k \in \mathbb{Z}$.

COROLLARY

A consequence of the CRT is that the equation

$$x \equiv a \pmod{m}$$

is equivalent to the system of equations

$$\begin{cases} x &\equiv a_1 \pmod{m_1} \\ &\vdots \\ x &\equiv a_k \pmod{m_k} \end{cases}$$

(As above, assume that $m = m_1 m_2 \cdots m_k$ and m_i are pairwise coprime).

Solution for Two Moduli

Consider a system of two equations for coprime m_1, m_2:

$$\begin{cases} a \equiv a_1 \pmod{m_1} \\ a \equiv a_2 \pmod{m_2} \end{cases}$$

We want to find a solution for $a \pmod{m_1 m_2}$. Using the Extended Euclidean Algorithm we can find Bézout coefficients n_1, n_2 such that

$$n_1 m_1 + n_2 m_2 = 1.$$

In fact n_1 and n_2 are just the modular inverses of m_1 and m_2 modulo m_2 and m_1. We have $n_1 m_1 \equiv 1 \pmod{m_2}$ so $n_1 \equiv m_1^{-1} \pmod{m_2}$, and vice versa $n_2 \equiv m_2^{-1} \pmod{m_1}$.

With those two coefficients we can define a solution:

$$a = a_1 n_2 m_2 + a_2 n_1 m_1 \bmod m_1 m_2$$

It's easy to verify that this is indeed a solution by computing $a \bmod m_1$ and $a \bmod m_2$.

$$\begin{aligned} a &\equiv a_1 n_2 m_2 + a_2 n_1 m_1 && \pmod{m_1} \\ &\equiv a_1(1 - n_1 m_1) + a_2 n_1 m_1 && \pmod{m_1} \\ &\equiv a_1 - a_1 n_1 m_1 + a_2 n_1 m_1 && \pmod{m_1} \\ &\equiv a_1 && \pmod{m_1} \end{aligned}$$

Notice, that the Chinese Remainder Theorem also guarantees, that only 1 solution exists modulo $m_1 m_2$. This is also easy to prove.

Lets assume that you have two different solutions x and y. Because $x \equiv a_i \pmod{m_i}$ and $y \equiv a_i \pmod{m_i}$, it follows that $x - y \equiv 0 \pmod{m_i}$ and therefore $x - y \equiv 0 \pmod{m_1 m_2}$ or equivalently $x \equiv y \pmod{m_1 m_2}$. So x and y are actually the same solution.

Solution for General Case

INDUCTIVE SOLUTION

As $m_1 m_2$ is coprime to m_3, we can inductively repeatedly apply the solution for two moduli for any number of moduli. First you compute $b_2 := a \pmod{m_1 m_2}$ using the first two congruences, then you can compute $b_3 := a \pmod{m_1 m_2 m_3}$ using the congruences $a \equiv b_2 \pmod{m_1 m_2}$ and $a \equiv a_3 \pmod{m_3}$, etc.

DIRECT CONSTRUCTION

A direct construction similar to Lagrange interpolation is possible.

Let $M_i := \prod_{i \neq j} m_j$, the product of all moduli but m_i, and N_i the modular inverses $N_i := M_i^{-1} \bmod m_i$. Then a solution to the system of congruences is:

$$a \equiv \sum_{i=1}^{k} a_i M_i N_i \pmod{m_1 m_2 \cdots m_k}$$

We can check this is indeed a solution, by computing $a \bmod m_i$ for all i. Because M_j is a multiple of m_i for $i \neq j$ we have

$$\begin{aligned} a &\equiv \textstyle\sum_{j=1}^{k} a_j M_j N_j && \pmod{m_i} \\ &\equiv a_i M_i N_i && \pmod{m_i} \\ &\equiv a_i M_i M_i^{-1} && \pmod{m_i} \\ &\equiv a_i && \pmod{m_i} \end{aligned}$$

IMPLEMENTATION

```
struct Congruence {
    long long a, m;
};

long long chinese_remainder_theorem(vector<Congruence> const& congruences) {
    long long M = 1;
    for (auto const& congruence : congruences) {
        M *= congruence.m;
    }

    long long solution = 0;
    for (auto const& congruence : congruences) {
        long long a_i = congruence.a;
        long long M_i = M / congruence.m;
        long long N_i = mod_inv(M_i, congruence.m);
        solution = (solution + a_i * M_i % M * N_i) % M;
    }
```

```
    return solution;
}
```

Solution for not coprime moduli

As mentioned, the algorithm above only works for coprime moduli $m_1, m_2, \dots m_k$.

In the not coprime case, a system of congruences has exactly one solution modulo $\operatorname{lcm}(m_1, m_2, \dots, m_k)$, or has no solution at all.

E.g. in the following system, the first congruence implies that the solution is odd, and the second congruence implies that the solution is even. It's not possible that a number is both odd and even, therefore there is clearly no solution.

$$\begin{cases} a \equiv 1 \pmod 4 \\ a \equiv 2 \pmod 6 \end{cases}$$

It is pretty simple to determine if a system has a solution. And if it has one, we can use the original algorithm to solve a slightly modified system of congruences.

A single congruence $a \equiv a_i \pmod{m_i}$ is equivalent to the system of congruences $a \equiv a_i \pmod{p_j^{n_j}}$ where $p_1^{n_1} p_2^{n_2} \cdots p_k^{n_k}$ is the prime factorization of m_i.

With this fact, we can modify the system of congruences into a system, that only has prime powers as moduli. E.g. the above system of congruences is equivalent to:

$$\begin{cases} a \equiv 1 \pmod 4 \\ a \equiv 2 \equiv 0 \pmod 2 \\ a \equiv 2 \pmod 3 \end{cases}$$

Because originally some moduli had common factors, we will get some congruences moduli based on the same prime, however possibly with different prime powers.

You can observe, that the congruence with the highest prime power modulus will be the strongest congruence of all congruences based on the same prime number. Either it will give a contradiction with some other congruence, or it will imply already all other congruences.

In our case, the first congruence $a \equiv 1 \pmod 4$ implies $a \equiv 1 \pmod 2$, and therefore contradicts the second congruence $a \equiv 0 \pmod 2$. Therefore this system of congruences has no solution.

If there are no contradictions, then the system of equation has a solution. We can ignore all congruences except the ones with the highest prime power moduli. These moduli are now coprime, and therefore we can solve this one with the algorithm discussed in the sections above.

E.g. the following system has a solution modulo $\operatorname{lcm}(10, 12) = 60$.

$$\begin{cases} a \equiv 3 \pmod{10} \\ a \equiv 5 \pmod{12} \end{cases}$$

The system of congruence is equivalent to the system of congruences:

$$\begin{cases} a \equiv 3 \equiv 1 \pmod 2 \\ a \equiv 3 \equiv 3 \pmod 5 \\ a \equiv 5 \equiv 1 \pmod 4 \\ a \equiv 5 \equiv 2 \pmod 3 \end{cases}$$

The only congruence with same prime modulo are $a \equiv 1 \pmod 4$ and $a \equiv 1 \pmod 2$. The first one already implies the second one, so we can ignore the second one, and solve the following system with coprime moduli instead:

$$\begin{cases} a \equiv 3 \equiv 3 \pmod 5 \\ a \equiv 5 \equiv 1 \pmod 4 \\ a \equiv 5 \equiv 2 \pmod 3 \end{cases}$$

It has the solution $53 \pmod{60}$, and indeed $53 \bmod 10 = 3$ and $53 \bmod 12 = 5$.

Garner's Algorithm

Another consequence of the CRT is that we can represent big numbers using an array of small integers.

Instead of doing a lot of computations with very large numbers numbers, which might be expensive (think of doing divisions with 1000-digit numbers), you can pick a couple of coprime moduli and represent the large number as a system of congruences, and perform all operations on the system of equations. Any number a less than $m_1 m_2 \cdots m_k$ can be represented as an array $a_1, \ldots, a_k$, where $a \equiv a_i \pmod{m_i}$.

By using the above algorithm, you can again reconstruct the large number whenever you need it.

Alternatively you can represent the number in the **mixed radix** representation:

$$a = x_1 + x_2 m_1 + x_3 m_1 m_2 + \ldots + x_k m_1 \cdots m_{k-1} \text{ with } x_i \in [0, m_i)$$

Garner's algorithm, which is discussed in the dedicated article Garner's algorithm, computes the coefficients x_i. And with those coefficients you can restore the full number.

3.5 Tasks

3.5.1 Dynamic Programming on Broken Profile. Problem "Parquet"

Common problems solved using DP on broken profile include:

- finding number of ways to fully fill an area (e.g. chessboard/grid) with some figures (e.g. dominoes)
- finding a way to fill an area with minimum number of figures
- finding a partial fill with minimum number of unfilled space (or cells, in case of grid)
- finding a partial fill with the minimum number of figures, such that no more figures can be added

Problem "Parquet"

Problem description. Given a grid of size $N \times M$. Find number of ways to fill the grid with figures of size 2×1 (no cell should be left unfilled, and figures should not overlap each other).

Let the DP state be: $dp[i, mask]$, where $i = 1, \dots N$ and $mask = 0, \dots 2^M - 1$.

i represents number of rows in the current grid, and $mask$ is the state of last row of current grid. If j-th bit of $mask$ is 0 then the corresponding cell is filled, otherwise it is unfilled.

Clearly, the answer to the problem will be $dp[N, 0]$.

We will be building the DP state by iterating over each $i = 1, \cdots N$ and each $mask = 0, \dots 2^M - 1$, and for each $mask$ we will be only transitioning forward, that is, we will be *adding* figures to the current grid.

IMPLEMENTATION

```
int n, m;
vector < vector<long long> > dp;


void calc (int x = 0, int y = 0, int mask = 0, int next_mask = 0)
{
    if (x == n)
        return;
    if (y >= m)
        dp[x+1][next_mask] += dp[x][mask];
    else
    {
        int my_mask = 1 << y;
        if (mask & my_mask)
            calc (x, y+1, mask, next_mask);
        else
        {
            calc (x, y+1, mask, next_mask | my_mask);
            if (y+1 < m && ! (mask & my_mask) && ! (mask & (my_mask << 1)))
                calc (x, y+2, mask, next_mask);
        }
    }
}


int main()
{
    cin >> n >> m;

    dp.resize (n+1, vector<long long> (1<<m));
    dp[0][0] = 1;
    for (int x=0; x<n; ++x)
        for (int mask=0; mask<(1<<m); ++mask)
            calc (x, 0, mask, 0);

    cout << dp[n][0];

}
```

3.5.2 Finding the largest zero submatrix

You are given a matrix with `n` rows and `m` columns. Find the largest submatrix consisting of only zeros (a submatrix is a rectangular area of the matrix).

Algorithm

Elements of the matrix will be `a[i][j]`, where `i = 0...n - 1`, `j = 0... m - 1`. For simplicity, we will consider all non-zero elements equal to 1.

STEP 1: AUXILIARY DYNAMIC

First, we calculate the following auxiliary matrix: `d[i][j]`, nearest row that has a 1 above `a[i][j]`. Formally speaking, `d[i][j]` is the largest row number (from `0` to `i - 1`), in which there is a element equal to `1` in the `j`-th column. While iterating from top-left to bottom-right, when we stand in row `i`, we know the values from the previous row, so, it is enough to update just the elements with value `1`. We can save the values in a simple array `d[i]`, `i = 1...m - 1`, because in the further algorithm we will process the matrix one row at a time and only need the values of the current row.

```
vector<int> d(m, -1);
for (int i = 0; i < n; ++i) {
    for (int j = 0; j < m; ++j) {
        if (a[i][j] == 1) {
            d[j] = i;
        }
    }
}
```

STEP 2: PROBLEM SOLVING

We can solve the problem in $O(nm^2)$ iterating through rows, considering every possible left and right columns for a submatrix. The bottom of the rectangle will be the current row, and using `d[i][j]` we can find the top row. However, it is possible to go further and significantly improve the complexity of the solution.

It is clear that the desired zero submatrix is bounded on all four sides by some ones, which prevent it from increasing in size and improving the answer. Therefore, we will not miss the answer if we act as follows: for every cell `j` in row `i` (the bottom row of a potential zero submatrix) we will have `d[i][j]` as the top row of the current zero submatrix. It now remains to determine the optimal left and right boundaries of the zero submatrix, i.e. maximally push this submatrix to the left and right of the `j`-th column.

What does it mean to push the maximum to the left? It means to find an index `k1` for which `d[i][k1] > d[i][j]`, and at the same time `k1` - the closest one to the left of the index `j`. It is clear that then `k1 + 1` gives the number of the left column of the required zero submatrix. If there is no such index at all, then put `k1 = -1` (this means that we were able to extend the current zero submatrix to the left all the way to the border of matrix `a`).

Symmetrically, you can define an index `k2` for the right border: this is the closest index to the right of `j` such that `d[i][k2] > d[i][j]` (or `m`, if there is no such index).

So, the indices `k1` and `k2`, if we learn to search for them effectively, will give us all the necessary information about the current zero submatrix. In particular, its area will be equal to `(i - d[i][j]) * (k2 - k1 - 1)`.

How to look for these indexes `k1` and `k2` effectively with fixed `i` and `j`? We can do that in $O(1)$ on average.

To achieve such complexity, you can use the stack as follows. Let's first learn how to search for an index `k1`, and save its value for each index `j` within the current row `i` in matrix `d1[i][j]`. To do this, we will look through all the columns `j` from left to right, and we will store in the stack only those columns that have `d[][]` strictly greater than `d[i][j]`. It is clear that when moving from a column `j` to the next column, it is necessary to update the content of the stack. When there is an inappropriate element at the top of the stack (i.e. `d[][] <= d[i][j]`) pop it. It is easy to understand that it is enough to remove from the stack only from its top, and from none of its other places (because the stack will contain an increasing `d` sequence of columns).

The value `d1[i][j]` for each `j` will be equal to the value lying at that moment on top of the stack.

The dynamics `d2[i][j]` for finding the indices `k2` is considered similar, only you need to view the columns from right to left.

It is clear that since there are exactly `m` pieces added to the stack on each line, there could not be more deletions either, the sum of complexities will be linear, so the final complexity of the algorithm is $O(nm)$.

It should also be noted that this algorithm consumes $O(m)$ memory (not counting the input data - the matrix `a[][]`).

IMPLEMENTATION

```
int zero_matrix(vector<vector<int>> a) {
    int n = a.size();
    int m = a[0].size();

    int ans = 0;
    vector<int> d(m, -1), d1(m), d2(m);
    stack<int> st;
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < m; ++j) {
            if (a[i][j] == 1)
                d[j] = i;
        }

        for (int j = 0; j < m; ++j) {
            while (!st.empty() && d[st.top()] <= d[j])
                st.pop();
            d1[j] = st.empty() ? -1 : st.top();
            st.push(j);
        }
        while (!st.empty())
            st.pop();

        for (int j = m - 1; j >= 0; --j) {
            while (!st.empty() && d[st.top()] <= d[j])
                st.pop();
            d2[j] = st.empty() ? m : st.top();
            st.push(j);
        }
        while (!st.empty())
            st.pop();

        for (int j = 0; j < m; ++j)
            ans = max(ans, (i - d[j]) * (d2[j] - d1[j] - 1));
    }
    return ans;
}
```

4.1.5 Suffix Array

Definition

Let s be a string of length n. The i-th suffix of s is the substring $s[i \dots n-1]$.

A **suffix array** will contain integers that represent the **starting indexes** of the all the suffixes of a given string, after the aforementioned suffixes are sorted.

As an example look at the string $s = abaab$. All suffixes are as follows

0. $abaab$
1. $baab$
2. aab
3. ab
4. b

After sorting these strings:

2. aab
3. ab
0. $abaab$
4. b
1. $baab$

Therefore the suffix array for s will be $(2,\ 3,\ 0,\ 4,\ 1)$.

As a data structure it is widely used in areas such as data compression, bioinformatics and, in general, in any area that deals with strings and string matching problems.

Construction

$O(n^2 \log n)$ **APPROACH**

This is the most naive approach. Get all the suffixes and sort them using quicksort or mergesort and simultaneously retain their original indices. Sorting uses $O(n \log n)$ comparisons, and since comparing two strings will additionally take $O(n)$ time, we get the final complexity of $O(n^2 \log n)$.

$O(n \log n)$ **APPROACH**

Strictly speaking the following algorithm will not sort the suffixes, but rather the cyclic shifts of a string. However we can very easily derive an algorithm for sorting suffixes from it: it is enough to append an arbitrary character to the end of the string which is smaller than any character from the string. It is common to use the symbol \$. Then the order of the sorted cyclic shifts is equivalent to the order of the sorted suffixes, as demonstrated here with the string $dabbb$.

1. $abbb\$d$ $abbb$
4. $b\$dabb$ b
3. $bb\$dab$ bb
2. $bbb\$da$ bbb
0. $dabbb\$$ $dabbb$

Since we are going to sort cyclic shifts, we will consider **cyclic substrings**. We will use the notation $s[i \dots j]$ for the substring of s even if $i > j$. In this case we actually mean the string $s[i \dots n-1] + s[0 \dots j]$. In addition we will take all indices modulo the length of s, and will omit the modulo operation for simplicity.

The algorithm we discuss will perform $\lceil \log n \rceil + 1$ iterations. In the k-th iteration ($k = 0 \dots \lceil \log n \rceil$) we sort the n cyclic substrings of s of length 2^k. After the $\lceil \log n \rceil$-th iteration the substrings of length $2^{\lceil \log n \rceil} \geq n$ will be sorted, so this is equivalent to sorting the cyclic shifts altogether.

In each iteration of the algorithm, in addition to the permutation $p[0 \dots n-1]$, where $p[i]$ is the index of the i-th substring (starting at i and with length 2^k) in the sorted order, we will also maintain an array $c[0 \dots n-1]$, where $c[i]$ corresponds to the **equivalence class** to which the substring belongs. Because some of the substrings will be identical, and the algorithm needs to treat them equally. For convenience the classes will be labeled by numbers started from zero. In addition the numbers $c[i]$ will be assigned in such a way that they preserve information about the order: if one substring is smaller than the other, then it should also have a smaller class label. The number of equivalence classes will be stored in a variable classes.

Let's look at an example. Consider the string $s = aaba$. The cyclic substrings and the corresponding arrays $p[]$ and $c[]$ are given for each iteration:

$$\begin{array}{lll} 0: & (a,\ a,\ b,\ a) & p = (0,\ 1,\ 3,\ 2) \quad c = (0,\ 0,\ 1,\ 0) \\ 1: & (aa,\ ab,\ ba,\ aa) & p = (0,\ 3,\ 1,\ 2) \quad c = (0,\ 1,\ 2,\ 0) \\ 2: & (aaba,\ abaa,\ baaa,\ aaab) & p = (3,\ 0,\ 1,\ 2) \quad c = (1,\ 2,\ 3,\ 0) \end{array}$$

It is worth noting that the values of $p[]$ can be different. For example in the 0-th iteration the array could also be $p = (3,\ 1,\ 0,\ 2)$ or $p = (3,\ 0,\ 1,\ 2)$. All these options permutation the substrings into a sorted order. So they are all valid. At the same time the array $c[]$ is fixed, there can be no ambiguities.

Let us now focus on the implementation of the algorithm. We will write a function that takes a string s and returns the permutations of the sorted cyclic shifts.

```
vector<int> sort_cyclic_shifts(string const& s) {
    int n = s.size();
    const int alphabet = 256;
```

At the beginning (in the **0-th iteration**) we must sort the cyclic substrings of length 1, that is we have to sort all characters of the string and divide them into equivalence classes (same symbols get assigned to the same class). This can be done trivially, for example, by using **counting sort**. For each character we count how many times it appears in the string, and then use this information to create the array $p[]$. After that we go through the array $p[]$ and construct $c[]$ by comparing adjacent characters.

```
    vector<int> p(n), c(n), cnt(max(alphabet, n), 0);
    for (int i = 0; i < n; i++)
        cnt[s[i]]++;
    for (int i = 1; i < alphabet; i++)
        cnt[i] += cnt[i-1];
    for (int i = 0; i < n; i++)
        p[--cnt[s[i]]] = i;
    c[p[0]] = 0;
    int classes = 1;
    for (int i = 1; i < n; i++) {
        if (s[p[i]] != s[p[i-1]])
            classes++;
        c[p[i]] = classes - 1;
    }
```

Now we have to talk about the iteration step. Let's assume we have already performed the $k-1$-th step and computed the values of the arrays $p[]$ and $c[]$ for it. We want to compute the values for the k-th step in $O(n)$ time. Since we perform this step $O(\log n)$ times, the complete algorithm will have a time complexity of $O(n \log n)$.

To do this, note that the cyclic substrings of length 2^k consists of two substrings of length 2^{k-1} which we can compare with each other in $O(1)$ using the information from the previous phase - the values of the equivalence classes $c[]$. Thus, for two substrings of length 2^k starting at position i and j, all necessary information to compare them is contained in the pairs $(c[i],\ c[i+2^{k-1}])$ and $(c[j],\ c[j+2^{k-1}])$.

$$\overbrace{\dots \underbrace{s_i \dots s_{i+2^{k-1}-1}}_{\text{length}=2^{k-1},\ \text{class}=c[i]} \dots}^{\text{length}=2^k}$$

This gives us a very simple solution: **sort** the substrings of length 2^k **by these pairs of numbers**. This will give us the required order $p[]$. However a normal sort runs in $O(n \log n)$ time, with which we are not satisfied. This will only give us an algorithm for constructing a suffix array in $O(n \log^2 n)$ times.

How do we quickly perform such a sorting of the pairs? Since the elements of the pairs do not exceed n, we can use counting sort again. However sorting pairs with counting sort is not the most efficient. To achieve a better hidden constant in the complexity, we will use another trick.

We use here the technique on which **radix sort** is based: to sort the pairs we first sort them by the second element, and then by the first element (with a stable sort, i.e. sorting without breaking the relative order of equal elements). However the second elements were already sorted in the previous iteration. Thus, in order to sort the pairs by the second elements, we just need to subtract 2^{k-1} from the indices in $p[]$ (e.g. if the smallest substring of length 2^{k-1} starts at position i, then the substring of length 2^k with the smallest second half starts at $i - 2^{k-1}$).

So only by simple subtractions we can sort the second elements of the pairs in $p[]$. Now we need to perform a stable sort by the first elements. As already mentioned, this can be accomplished with counting sort.

The only thing left is to compute the equivalence classes $c[]$, but as before this can be done by simply iterating over the sorted permutation $p[]$ and comparing neighboring pairs.

Here is the remaining implementation. We use temporary arrays $pn[]$ and $cn[]$ to store the permutation by the second elements and the new equivalent class indices.

```
    vector<int> pn(n), cn(n);
    for (int h = 0; (1 << h) < n; ++h) {
        for (int i = 0; i < n; i++) {
            pn[i] = p[i] - (1 << h);
            if (pn[i] < 0)
                pn[i] += n;
        }
        fill(cnt.begin(), cnt.begin() + classes, 0);
        for (int i = 0; i < n; i++)
            cnt[c[pn[i]]]++;
        for (int i = 1; i < classes; i++)
            cnt[i] += cnt[i-1];
        for (int i = n-1; i >= 0; i--)
            p[--cnt[c[pn[i]]]] = pn[i];
        cn[p[0]] = 0;
        classes = 1;
        for (int i = 1; i < n; i++) {
            pair<int, int> cur = {c[p[i]], c[(p[i] + (1 << h)) % n]};
            pair<int, int> prev = {c[p[i-1]], c[(p[i-1] + (1 << h)) % n]};
            if (cur != prev)
                ++classes;
            cn[p[i]] = classes - 1;
        }
        c.swap(cn);
    }
    return p;
}
```

The algorithm requires $O(n \log n)$ time and $O(n)$ memory. For simplicity we used the complete ASCII range as alphabet.

If it is known that the string only contains a subset of characters, e.g. only lowercase letters, then the implementation can be optimized, but the optimization factor would likely be insignificant, as the size of the alphabet only matters on the first iteration. Every other iteration depends on the number of equivalence classes, which may quickly reach $O(n)$ even if initially it was a string over the alphabet of size 2.

Also note, that this algorithm only sorts the cycle shifts. As mentioned at the beginning of this section we can generate the sorted order of the suffixes by appending a character that is smaller than all other characters of the string, and sorting this resulting string by cycle shifts, e.g. by sorting the cycle shifts of $s + \$$. This will obviously give the suffix array of s, however prepended with $|s|$.

```
vector<int> suffix_array_construction(string s) {
    s += "$";
    vector<int> sorted_shifts = sort_cyclic_shifts(s);
    sorted_shifts.erase(sorted_shifts.begin());
    return sorted_shifts;
}
```

Applications

FINDING THE SMALLEST CYCLIC SHIFT

The algorithm above sorts all cyclic shifts (without appending a character to the string), and therefore $p[0]$ gives the position of the smallest cyclic shift.

FINDING A SUBSTRING IN A STRING

The task is to find a string s inside some text t online - we know the text t beforehand, but not the string s. We can create the suffix array for the text t in $O(|t| \log |t|)$ time. Now we can look for the substring s in the following way. The occurrence of s must be a prefix of some suffix from t. Since we sorted all the suffixes we can perform a binary search for s in p. Comparing the current suffix and the substring s within the binary search can be done in $O(|s|)$ time, therefore the complexity for finding the substring is $O(|s| \log |t|)$. Also notice that if the substring occurs multiple times in t, then all occurrences will be next to each other in p. Therefore the number of occurrences can be found with a second binary search, and all occurrences can be printed easily.

COMPARING TWO SUBSTRINGS OF A STRING

We want to be able to compare two substrings of the same length of a given string s in $O(1)$ time, i.e. checking if the first substring is smaller than the second one.

For this we construct the suffix array in $O(|s| \log |s|)$ time and store all the intermediate results of the equivalence classes $c[]$.

Using this information we can compare any two substring whose length is equal to a power of two in O(1): for this it is sufficient to compare the equivalence classes of both substrings. Now we want to generalize this method to substrings of arbitrary length.

Let's compare two substrings of length l with the starting indices i and j. We find the largest length of a block that is placed inside a substring of this length: the greatest k such that $2^k \leq l$. Then comparing the two substrings can be replaced by comparing two overlapping blocks of length 2^k: first you need to compare the two blocks starting at i and j, and if these are equal then compare the two blocks ending in positions $i + l - 1$ and $j + l - 1$:

$$\overline{\quad \ldots \overbrace{\underbrace{s_i \ldots s_{i+l-2^k}}_{2^k} \ldots s_{i+2^k-1}}^{\text{first}}}$$

$$\overline{\quad \ldots \overbrace{s_i \ldots \underbrace{s_{i+l-2^k} \ldots s_{i+2^k-1}}_{2^k} \ldots s_{i+l-1}}^{\text{first}}}$$

Here is the implementation of the comparison. Note that it is assumed that the function gets called with the already calculated k. k can be computed with $\lfloor \log l \rfloor$, but it is more efficient to precompute all k values for every l. See for instance the article about the Sparse Table, which uses a similar idea and computes all $\log$ values.

```
int compare(int i, int j, int l, int k) {
    pair<int, int> a = {c[k][i], c[k][(i+l-(1 << k))%n]};
    pair<int, int> b = {c[k][j], c[k][(j+l-(1 << k))%n]};
    return a == b ? 0 : a < b ? -1 : 1;
}
```

LONGEST COMMON PREFIX OF TWO SUBSTRINGS WITH ADDITIONAL MEMORY

For a given string s we want to compute the longest common prefix (**LCP**) of two arbitrary suffixes with position i and j.

The method described here uses $O(|s| \log |s|)$ additional memory. A completely different approach that will only use a linear amount of memory is described in the next section.

We construct the suffix array in $O(|s| \log |s|)$ time, and remember the intermediate results of the arrays $c[]$ from each iteration.

Let's compute the LCP for two suffixes starting at i and j. We can compare any two substrings with a length equal to a power of two in $O(1)$. To do this, we compare the strings by power of twos (from highest to lowest power) and if the substrings of this length are the same, then we add the equal length to the answer and continue checking for the LCP to the right of the equal part, i.e. i and j get added by the current power of two.

```
int lcp(int i, int j) {
    int ans = 0;
    for (int k = log_n; k >= 0; k--) {
        if (c[k][i % n] == c[k][j % n]) {
            ans += 1 << k;
            i += 1 << k;
            j += 1 << k;
        }
    }
```

```
    return ans;
}
```

Here `log_n` denotes a constant that is equal to the logarithm of n in base 2 rounded down.

LONGEST COMMON PREFIX OF TWO SUBSTRINGS WITHOUT ADDITIONAL MEMORY

We have the same task as in the previous section. We have compute the longest common prefix (**LCP**) for two suffixes of a string s.

Unlike the previous method this one will only use $O(|s|)$ memory. The result of the preprocessing will be an array (which itself is an important source of information about the string, and therefore also used to solve other tasks). LCP queries can be answered by performing RMQ queries (range minimum queries) in this array, so for different implementations it is possible to achieve logarithmic and even constant query time.

The basis for this algorithm is the following idea: we will compute the longest common prefix for each **pair of adjacent suffixes in the sorted order**. In other words we construct an array $\text{lcp}[0 \dots n-2]$, where $\text{lcp}[i]$ is equal to the length of the longest common prefix of the suffixes starting at $p[i]$ and $p[i+1]$. This array will give us an answer for any two adjacent suffixes of the string. Then the answer for arbitrary two suffixes, not necessarily neighboring ones, can be obtained from this array. In fact, let the request be to compute the LCP of the suffixes $p[i]$ and $p[j]$. Then the answer to this query will be $\min(lcp[i],\ lcp[i+1],\ \dots,\ lcp[j-1])$.

Thus if we have such an array lcp, then the problem is reduced to the RMQ, which has many wide number of different solutions with different complexities.

So the main task is to **build** this array lcp. We will use **Kasai's algorithm**, which can compute this array in $O(n)$ time.

Let's look at two adjacent suffixes in the sorted order (order of the suffix array). Let their starting positions be i and j and their lcp equal to $k > 0$. If we remove the first letter of both suffixes - i.e. we take the suffixes $i+1$ and $j+1$ - then it should be obvious that the lcp of these two is $k-1$. However we cannot use this value and write it in the lcp array, because these two suffixes might not be next to each other in the sorted order. The suffix $i+1$ will of course be smaller than the suffix $j+1$, but there might be some suffixes between them. However, since we know that the LCP between two suffixes is the minimum value of all transitions, we also know that the LCP between any two pairs in that interval has to be at least $k-1$, especially also between $i+1$ and the next suffix. And possibly it can be bigger.

Now we already can implement the algorithm. We will iterate over the suffixes in order of their length. This way we can reuse the last value k, since going from suffix i to the suffix $i+1$ is exactly the same as removing the first letter. We will need an additional array rank, which will give us the position of a suffix in the sorted list of suffixes.

```
vector<int> lcp_construction(string const& s, vector<int> const& p) {
    int n = s.size();
    vector<int> rank(n, 0);
    for (int i = 0; i < n; i++)
        rank[p[i]] = i;

    int k = 0;
    vector<int> lcp(n-1, 0);
    for (int i = 0; i < n; i++) {
        if (rank[i] == n - 1) {
            k = 0;
            continue;
        }
        int j = p[rank[i] + 1];
        while (i + k < n && j + k < n && s[i+k] == s[j+k])
            k++;
        lcp[rank[i]] = k;
        if (k)
            k--;
    }
    return lcp;
}
```

It is easy to see, that we decrease k at most $O(n)$ times (each iteration at most once, except for $\text{rank}[i] == n-1$, where we directly reset it to 0), and the LCP between two strings is at most $n-1$, we will also increase k only $O(n)$ times. Therefore the algorithm runs in $O(n)$ time.

NUMBER OF DIFFERENT SUBSTRINGS

We preprocess the string s by computing the suffix array and the LCP array. Using this information we can compute the number of different substrings in the string.

To do this, we will think about which **new** substrings begin at position $p[0]$, then at $p[1]$, etc. In fact we take the suffixes in sorted order and see what prefixes give new substrings. Thus we will not overlook any by accident.

Because the suffixes are sorted, it is clear that the current suffix $p[i]$ will give new substrings for all its prefixes, except for the prefixes that coincide with the suffix $p[i-1]$. Thus, all its prefixes except the first $\text{lcp}[i-1]$ one. Since the length of the current suffix is $n-p[i]$, $n-p[i]-\text{lcp}[i-1]$ new prefixes start at $p[i]$. Summing over all the suffixes, we get the final answer:

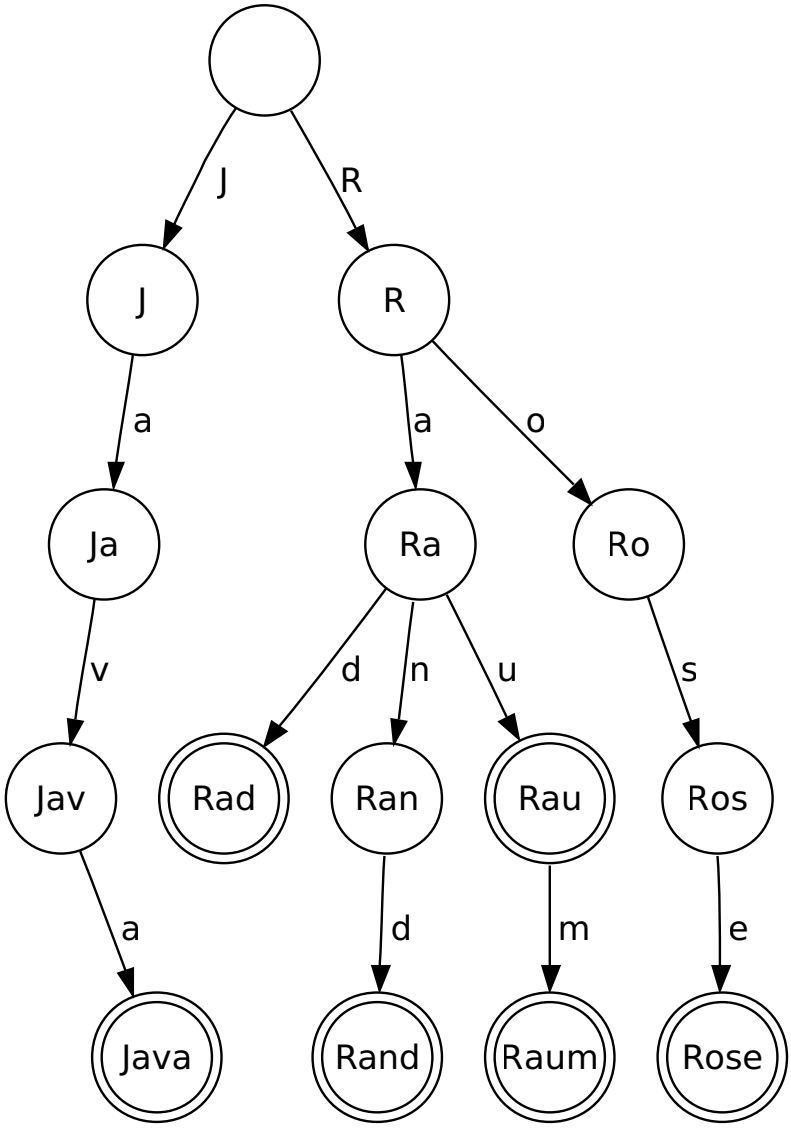
$$\sum_{i=0}^{n-1}(n-p[i])-\sum_{i=0}^{n-2}\text{lcp}[i]=\frac{n^2+n}{2}-\sum_{i=0}^{n-2}\text{lcp}[i]$$

4.1.6 Aho-Corasick algorithm

The Aho-Corasick algorithm allows us to quickly search for multiple patterns in a text. The set of pattern strings is also called a *dictionary*. We will denote the total length of its constituent strings by m and the size of the alphabet by k. The algorithm constructs a finite state automaton based on a trie in $O(mk)$ time and then uses it to process the text.

The algorithm was proposed by Alfred Aho and Margaret Corasick in 1975.

Construction of the trie



A trie based on words "Java", "Rad", "Rand", "Rau", "Raum" and "Rose".
The image by [nd](https://de.wikipedia.org/wiki/Benutzer:Nd) is distributed under CC BY-SA 3.0 license.

Formally, a trie is a rooted tree, where each edge of the tree is labeled with some letter and outgoing edges of a vertex have distinct labels.

We will identify each vertex in the trie with the string formed by the labels on the path from the root to that vertex.

Each vertex will also have a flag output which will be set if the vertex corresponds to a pattern in the dictionary.

Accordingly, a trie for a set of strings is a trie such that each output vertex corresponds to one string from the set, and conversely, each string of the set corresponds to one output vertex.

We now describe how to construct a trie for a given set of strings in linear time with respect to their total length.

We introduce a structure for the vertices of the tree:

```
const int K = 26;

struct Vertex {
    int next[K];
    bool output = false;

    Vertex() {
        fill(begin(next), end(next), -1);
    }
};

vector<Vertex> trie(1);
```

Here, we store the trie as an array of Vertex. Each Vertex contains the flag output and the edges in the form of an array $\text{next}[]$, where $\text{next}[i]$ is the index of the vertex that we reach by following the character i, or -1 if there is no such edge. Initially, the trie consists of only one vertex - the root - with the index 0.

Now we implement a function that will add a string s to the trie. The implementation is simple: we start at the root node, and as long as there are edges corresponding to the characters of s we follow them. If there is no edge for one character, we generate a new vertex and connect it with an edge. At the end of the process we mark the last vertex with the flag output.

```
void add_string(string const& s) {
    int v = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (trie[v].next[c] == -1) {
            trie[v].next[c] = trie.size();
            trie.emplace_back();
        }
        v = trie[v].next[c];
    }
    trie[v].output = true;
}
```

This implementation obviously runs in linear time, and since every vertex stores k links, it will use $O(mk)$ memory.

It is possible to decrease the memory consumption to $O(m)$ by using a map instead of an array in each vertex. However, this will increase the time complexity to $O(m \log k)$.

Construction of an automaton

Suppose we have built a trie for the given set of strings. Now let's look at it from a different side. If we look at any vertex, the string that corresponds to it is a prefix of one or more strings in the set, thus each vertex of the trie can be interpreted as a position in one or more strings from the set.

In fact, the trie vertices can be interpreted as states in a **finite deterministic automaton**. From any state we can transition - using some input letter - to other states, i.e., to another position in the set of strings. For example, if there is only one string abc in the dictionary, and we are standing at vertex ab, then using the letter c we can go to the vertex abc.

Thus we can understand the edges of the trie as transitions in an automaton according to the corresponding letter. However, in an automaton we need to have transitions for each combination of a state and a letter. If we try to perform a transition using a letter, and there is no corresponding edge in the trie, then we nevertheless must go into some state.

More precisely, suppose we are in a state corresponding to a string t, and we want to transition to a different state using the character c. If there is an edge labeled with this letter c, then we can simply go over this edge, and get the vertex corresponding to $t + c$. If there is no such edge, since we want to maintain the invariant that the current state is the longest partial match in the processed string, we must find the longest string in the trie that's a proper suffix of the string t, and try to perform a transition from there.

For example, let the trie be constructed by the strings ab and bc, and we are currently at the vertex corresponding to ab, which is also an output vertex. To transition with the letter c, we are forced to go to the state corresponding to the string b, and from there follow the edge with the letter c.

An Aho-Corasick automaton based on words "a", "ab", "bc", "bca", "c" and "caa".
Blue arrows are suffix links, green arrows are terminal links.

A **suffix link** for a vertex p is an edge that points to the longest proper suffix of the string corresponding to the vertex p. The only special case is the root of the trie, whose suffix link will point to itself. Now we can reformulate the statement about the transitions in the automaton like this: while there is no transition from the current vertex of the trie using the current letter (or until we reach the root), we follow the suffix link.

Thus we reduced the problem of constructing an automaton to the problem of finding suffix links for all vertices of the trie. However, we will build these suffix links, oddly enough, using the transitions constructed in the automaton.

The suffix links of the root vertex and all its immediate children point to the root vertex. For any vertex v deeper in the tree, we can calculate the suffix link as follows: if p is the ancestor of v with c being the letter labeling the edge from p to v, go to p, then follow its suffix link, and perform the transition with the letter c from there.

Thus, the problem of finding the transitions has been reduced to the problem of finding suffix links, and the problem of finding suffix links has been reduced to the problem of finding a suffix link and a transition, except for vertices closer to the root. So we have a recursive dependence that we can resolve in linear time.

Let's move to the implementation. Note that we now will store the ancestor p and the character pch of the edge from p to v for each vertex v. Also, at each vertex we will store the suffix link link (or -1 if it hasn't been calculated yet), and in the array $\text{go}[k]$ the transitions in the machine for each symbol (again -1 if it hasn't been calculated yet).

```
const int K = 26;

struct Vertex {
    int next[K];
    bool output = false;
    int p = -1;
    char pch;
    int link = -1;
    int go[K];

    Vertex(int p=-1, char ch='$') : p(p), pch(ch) {
        fill(begin(next), end(next), -1);
        fill(begin(go), end(go), -1);
    }
};

vector<Vertex> t(1);

void add_string(string const& s) {
    int v = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (t[v].next[c] == -1) {
            t[v].next[c] = t.size();
            t.emplace_back(v, ch);
        }
        v = t[v].next[c];
    }
    t[v].output = true;
}

int go(int v, char ch);

int get_link(int v) {
    if (t[v].link == -1) {
        if (v == 0 || t[v].p == 0)
            t[v].link = 0;
        else
            t[v].link = go(get_link(t[v].p), t[v].pch);
    }
    return t[v].link;
}

int go(int v, char ch) {
    int c = ch - 'a';
    if (t[v].go[c] == -1) {
        if (t[v].next[c] != -1)
            t[v].go[c] = t[v].next[c];
        else
            t[v].go[c] = v == 0 ? 0 : go(get_link(v), ch);
```

```
    }
    return t[v].go[c];
}
```

It is easy to see that thanks to memoization of the suffix links and transitions, the total time for finding all suffix links and transitions will be linear.

For an illustration of the concept refer to slide number 103 of the Stanford slides.

BFS-BASED CONSTRUCTION

Instead of computing transitions and suffix links with recursive calls to `go` and `get_link`, it is possible to compute them bottom-up starting from the root. (In fact, when the dictionary consists of only one string, we obtain the familiar Knuth-Morris-Pratt algorithm.)

This approach will have some advantages over the one described above as, instead of the total length m, its running time depends only on the number of vertices n in the trie. Moreover, it is possible to adapt it for large alphabets using a persistent array data structure, thus making the construction time $O(n \log k)$ instead of $O(mk)$, which is a significant improvement granted that m may go up to n^2.

We can reason inductively using the fact that BFS from the root traverses vertices in order of increasing length. We may assume that when we're in a vertex v, its suffix link $u = link[v]$ is already successfully computed, and for all vertices with shorter length transitions from them are also fully computed.

Assume that at the moment we stand in a vertex v and consider a character c. We essentially have two cases:

1. $go[v][c] = -1$. In this case, we may assign $go[v][c] = go[u][c]$, which is already known by the induction hypothesis;
2. $go[v][c] = w \neq -1$. In this case, we may assign $link[w] = go[u][c]$.

In this way, we spend $O(1)$ time per each pair of a vertex and a character, making the running time $O(nk)$. The major overhead here is that we copy a lot of transitions from u in the first case, while the transitions of the second case form the trie and sum up to n over all vertices. To avoid the copying of $go[u][c]$, we may use a persistent array data structure, using which we initially copy $go[u]$ into $go[v]$ and then only update values for characters in which the transition would differ. This leads to the $O(n \log k)$ algorithm.

Applications

FIND ALL STRINGS FROM A GIVEN SET IN A TEXT

We are given a set of strings and a text. We have to print all occurrences of all strings from the set in the given text in $O(\text{len} + \text{ans})$, where len is the length of the text and ans is the size of the answer.

We construct an automaton for this set of strings. We will now process the text letter by letter using the automaton, starting at the root of the trie. If we are at any time at state v, and the next letter is c, then we transition to the next state with $\text{go}(v, c)$, thereby either increasing the length of the current match substring by 1, or decreasing it by following a suffix link.

How can we find out for a state v, if there are any matches with strings for the set? First, it is clear that if we stand on a output vertex, then the string corresponding to the vertex ends at this position in the text. However this is by no means the only possible case of achieving a match: if we can reach one or more output vertices by moving along the suffix links, then there will be also a match corresponding to each found output vertex. A simple example demonstrating this situation can be created using the set of strings $\{dabce, abc, bc\}$ and the text $dabc$.

Thus if we store in each output vertex the index of the string corresponding to it (or the list of indices if duplicate strings appear in the set), then we can find in $O(n)$ time the indices of all strings which match the current state, by simply following the suffix links from the current vertex to the root. This is not the most efficient solution, since this results in $O(n \text{ len})$ complexity overall. However, this can be optimized by computing and storing the nearest output vertex that is reachable using suffix links (this is sometimes called the **exit link**). This value we can compute lazily in linear time. Thus for each vertex we can advance in $O(1)$ time to the next marked vertex in the suffix link path, i.e. to the next match. Thus for each match we spend $O(1)$ time, and therefore we reach the complexity $O(\text{len} + \text{ans})$.

If you only want to count the occurrences and not find the indices themselves, you can calculate the number of marked vertices in the suffix link path for each vertex v. This can be calculated in $O(n)$ time in total. Thus we can sum up all matches in $O(\text{len})$.

FINDING THE LEXICOGRAPHICALLY SMALLEST STRING OF A GIVEN LENGTH THAT DOESN'T MATCH ANY GIVEN STRINGS

A set of strings and a length L is given. We have to find a string of length L, which does not contain any of the strings, and derive the lexicographically smallest of such strings.

We can construct the automaton for the set of strings. Recall that output vertices are the states where we have a match with a string from the set. Since in this task we have to avoid matches, we are not allowed to enter such states. On the other hand we can enter all other vertices. Thus we delete all "bad" vertices from the machine, and in the remaining graph of the automaton we find the lexicographically smallest path of length L. This task can be solved in $O(L)$ for example by depth first search.

FINDING THE SHORTEST STRING CONTAINING ALL GIVEN STRINGS

Here we use the same ideas. For each vertex we store a mask that denotes the strings which match at this state. Then the problem can be reformulated as follows: initially being in the state $(v = \text{root},\ \text{mask} = 0)$, we want to reach the state $(v,\ \text{mask} = 2^n - 1)$, where n is the number of strings in the set. When we transition from one state to another using a letter, we update the mask accordingly. By running a breadth first search we can find a path to the state $(v,\ \text{mask} = 2^n - 1)$ with the smallest length.

FINDING THE LEXICOGRAPHICALLY SMALLEST STRING OF LENGTH L CONTAINING k STRINGS

As in the previous problem, we calculate for each vertex the number of matches that correspond to it (that is the number of marked vertices reachable using suffix links). We reformulate the problem: the current state is determined by a triple of numbers $(v,\ \text{len},\ \text{cnt})$, and we want to reach from the state $(\text{root},\ 0,\ 0)$ the state $(v,\ L,\ k)$, where v can be any vertex. Thus we can find such a path using depth first search (and if the search looks at the edges in their natural order, then the found path will automatically be the lexicographically smallest).

4.2 Advanced

4.2.1 Suffix Tree. Ukkonen's Algorithm

This article is a stub and doesn't contain any descriptions. For a description of the algorithm, refer to other sources, such as Algorithms on Strings, Trees, and Sequences by Dan Gusfield.

This algorithm builds a suffix tree for a given string s of length n in $O(n \log(k))$ time, where k is the size of the alphabet (if k is considered to be a constant, the asymptotic behavior is linear).

The input to the algorithm are the string s and its length n, which are passed as global variables.

The main function `build_tree` builds a suffix tree. It is stored as an array of structures `node`, where `node[0]` is the root of the tree.

In order to simplify the code, the edges are stored in the same structures: for each vertex its structure `node` stores the information about the edge between it and its parent. Overall each `node` stores the following information:

- `(l, r)` - left and right boundaries of the substring `s[l..r-1]` which correspond to the edge to this node,
- `par` - the parent node,
- `link` - the suffix link,
- `next` - the list of edges going out from this node.

```
string s;
int n;

struct node {
    int l, r, par, link;
    map<char,int> next;

    node (int l=0, int r=0, int par=-1)
        : l(l), r(r), par(par), link(-1) {}
    int len()  {  return r - l;  }
    int &get (char c) {
        if (!next.count(c))  next[c] = -1;
        return next[c];
    }
};
node t[MAXN];
int sz;

struct state {
    int v, pos;
    state (int v, int pos) : v(v), pos(pos)  {}
};
state ptr (0, 0);

state go (state st, int l, int r) {
    while (l < r)
        if (st.pos == t[st.v].len()) {
            st = state (t[st.v].get( s[l] ), 0);
            if (st.v == -1)  return st;
        }
        else {
            if (s[ t[st.v].l + st.pos ] != s[l])
                return state (-1, -1);
            if (r-l < t[st.v].len() - st.pos)
                return state (st.v, st.pos + r-l);
            l += t[st.v].len() - st.pos;
            st.pos = t[st.v].len();
        }
    return st;
}

int split (state st) {
    if (st.pos == t[st.v].len())
        return st.v;
    if (st.pos == 0)
        return t[st.v].par;
    node v = t[st.v];
```

```
    int id = sz++;
    t[id] = node (v.l, v.l+st.pos, v.par);
    t[v.par].get( s[v.l] ) = id;
    t[id].get( s[v.l+st.pos] ) = st.v;
    t[st.v].par = id;
    t[st.v].l += st.pos;
    return id;
}

int get_link (int v) {
    if (t[v].link != -1)  return t[v].link;
    if (t[v].par == -1)  return 0;
    int to = get_link (t[v].par);
    return t[v].link = split (go (state(to,t[to].len()), t[v].l + (t[v].par==0), t[v].r));
}

void tree_extend (int pos) {
    for(;;) {
        state nptr = go (ptr, pos, pos+1);
        if (nptr.v != -1) {
            ptr = nptr;
            return;
        }

        int mid = split (ptr);
        int leaf = sz++;
        t[leaf] = node (pos, n, mid);
        t[mid].get( s[pos] ) = leaf;

        ptr.v = get_link (mid);
        ptr.pos = t[ptr.v].len();
        if (!mid)  break;
    }
}

void build_tree() {
    sz = 1;
    for (int i=0; i<n; ++i)
        tree_extend (i);
}
```

Compressed Implementation

This compressed implementation was proposed by freopen.

```
const int N=1000000,INF=1000000000;
string a;
int t[N][26],l[N],r[N],p[N],s[N],tv,tp,ts,la;

void ukkadd (int c) {
    suff:;
    if (r[tv]<tp) {
        if (t[tv][c]==-1) { t[tv][c]=ts;  l[ts]=la;
            p[ts++]=tv;  tv=s[tv];  tp=r[tv]+1;  goto suff; }
        tv=t[tv][c]; tp=l[tv];
    }
    if (tp==-1 || c==a[tp]-'a') tp++; else {
        l[ts+1]=la;  p[ts+1]=ts;
        l[ts]=l[tv];  r[ts]=tp-1;  p[ts]=p[tv];  t[ts][c]=ts+1;  t[ts][a[tp]-'a']=tv;
        l[tv]=tp;  p[tv]=ts;  t[p[ts]][a[l[ts]]-'a']=ts;  ts+=2;
        tv=s[p[ts-2]];  tp=l[ts-2];
        while (tp<=r[ts-2]) {  tv=t[tv][a[tp]-'a'];  tp+=r[tv]-l[tv]+1;}
        if (tp==r[ts-2]+1)  s[ts-2]=tv;  else s[ts-2]=ts;
        tp=r[tv]-(tp-r[ts-2])+2;  goto suff;
    }
}

void build() {
    ts=2;
    tv=0;
    tp=0;
    fill(r,r+N,(int)a.size()-1);
    s[0]=1;
    l[0]=-1;
    r[0]=-1;
    l[1]=-1;
    r[1]=-1;
```

```
    memset (t, -1, sizeof t);
    fill(t[1],t[1]+26,0);
    for (la=0; la<(int)a.size(); ++la)
        ukkadd (a[la]-'a');
}
```

Same code with comments:

```
const int N=1000000,    // maximum possible number of nodes in suffix tree
    INF=1000000000; // infinity constant
string a;       // input string for which the suffix tree is being built
int t[N][26],   // array of transitions (state, letter)
    l[N],   // left...
    r[N],   // ...and right boundaries of the substring of a which correspond to incoming edge
    p[N],   // parent of the node
    s[N],   // suffix link
    tv,     // the node of the current suffix (if we're mid-edge, the lower node of the edge)
    tp,     // position in the string which corresponds to the position on the edge (between l[tv] and r[tv], inclusive)
    ts,     // the number of nodes
    la;     // the current character in the string

void ukkadd(int c) { // add character s to the tree
    suff:;      // we'll return here after each transition to the suffix (and will add character again)
    if (r[tv]<tp) { // check whether we're still within the boundaries of the current edge
        // if we're not, find the next edge. If it doesn't exist, create a leaf and add it to the tree
        if (t[tv][c]==-1) {t[tv][c]=ts;l[ts]=la;p[ts++]=tv;tv=s[tv];tp=r[tv]+1;goto suff;}
        tv=t[tv][c];tp=l[tv];
    } // otherwise just proceed to the next edge
    if (tp==-1 || c==a[tp]-'a')
        tp++; // if the letter on the edge equal c, go down that edge
    else {
        // otherwise split the edge in two with middle in node ts
        l[ts]=l[tv];r[ts]=tp-1;p[ts]=p[tv];t[ts][a[tp]-'a']=tv;
        // add leaf ts+1. It corresponds to transition through c.
        t[ts][c]=ts+1;l[ts+1]=la;p[ts+1]=ts;
        // update info for the current node - remember to mark ts as parent of tv
        l[tv]=tp;p[tv]=ts;t[p[ts]][a[l[ts]]-'a']=ts;ts+=2;
        // prepare for descent
        // tp will mark where are we in the current suffix
        tv=s[p[ts-2]];tp=l[ts-2];
        // while the current suffix is not over, descend
        while (tp<=r[ts-2]) {tv=t[tv][a[tp]-'a'];tp+=r[tv]-l[tv]+1;}
        // if we're in a node, add a suffix link to it, otherwise add the link to ts
        // (we'll create ts on next iteration).
        if (tp==r[ts-2]+1) s[ts-2]=tv; else s[ts-2]=ts;
        // add tp to the new edge and return to add letter to suffix
        tp=r[tv]-(tp-r[ts-2])+2;goto suff;
    }
}

void build() {
    ts=2;
    tv=0;
    tp=0;
    fill(r,r+N,(int)a.size()-1);
    // initialize data for the root of the tree
    s[0]=1;
    l[0]=-1;
    r[0]=-1;
    l[1]=-1;
    r[1]=-1;
    memset (t, -1, sizeof t);
    fill(t[1],t[1]+26,0);
    // add the text to the tree, letter by letter
    for (la=0; la<(int)a.size(); ++la)
        ukkadd (a[la]-'a');
}
```

4.2.2 Suffix Automaton

A **suffix automaton** is a powerful data structure that allows solving many string-related problems.

For example, you can search for all occurrences of one string in another, or count the amount of different substrings of a given string. Both tasks can be solved in linear time with the help of a suffix automaton.

Intuitively a suffix automaton can be understood as a compressed form of **all substrings** of a given string. An impressive fact is, that the suffix automaton contains all this information in a highly compressed form. For a string of length n it only requires $O(n)$ memory. Moreover, it can also be built in $O(n)$ time (if we consider the size k of the alphabet as a constant), otherwise both the memory and the time complexity will be $O(n \log k)$.

The linearity of the size of the suffix automaton was first discovered in 1983 by Blumer et al., and in 1985 the first linear algorithms for the construction was presented by Crochemore and Blumer.

Definition of a suffix automaton

A suffix automaton for a given string s is a minimal **DFA** (deterministic finite automaton / deterministic finite state machine) that accepts all the suffixes of the string s.

In other words:

- A suffix automaton is an oriented acyclic graph. The vertices are called **states**, and the edges are called **transitions** between states.
- One of the states t_0 is the **initial state**, and it must be the source of the graph (all other states are reachable from t_0).
- Each **transition** is labeled with some character. All transitions originating from a state must have **different** labels.
- One or multiple states are marked as **terminal states**. If we start from the initial state t_0 and move along transitions to a terminal state, then the labels of the passed transitions must spell one of the suffixes of the string s. Each of the suffixes of s must be spellable using a path from t_0 to a terminal state.
- The suffix automaton contains the minimum number of vertices among all automata satisfying the conditions described above.

SUBSTRING PROPERTY

The simplest and most important property of a suffix automaton is, that it contains information about all substrings of the string s. Any path starting at the initial state t_0, if we write down the labels of the transitions, forms a **substring** of s. And conversely every substring of s corresponds to a certain path starting at t_0.

In order to simplify the explanations, we will say that the substring **corresponds** to that path (starting at t_0 and the labels spell the substring). And conversely we say that any path **corresponds** to the string spelled by its labels.

One or multiple paths can lead to a state. Thus, we will say that a state **corresponds** to the set of strings, which correspond to these paths.

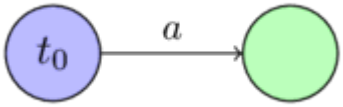
EXAMPLES OF CONSTRUCTED SUFFIX AUTOMATA

Here we will show some examples of suffix automata for several simple strings.

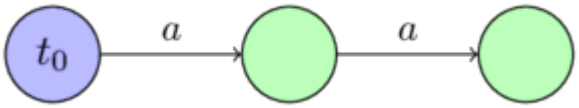
We will denote the initial state with blue and the terminal states with green.

For the string $s = $ "":

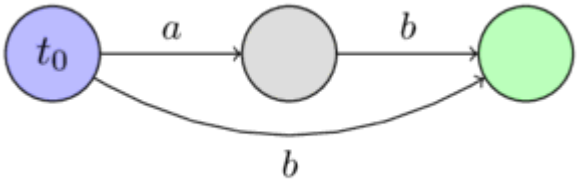
For the string $s = $ "a":



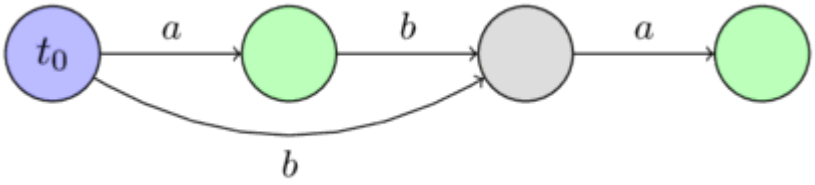
For the string $s = \text{"aa"}$:



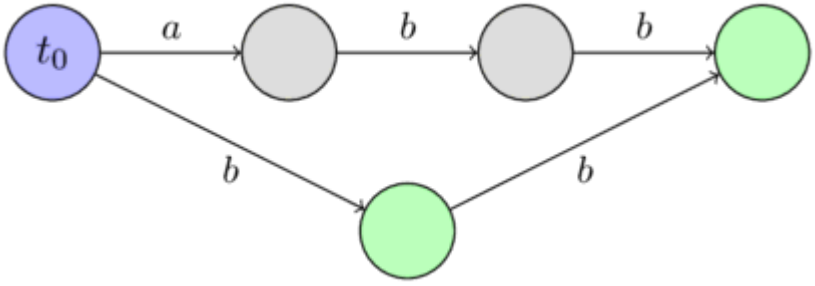
For the string $s = \text{"ab"}$:



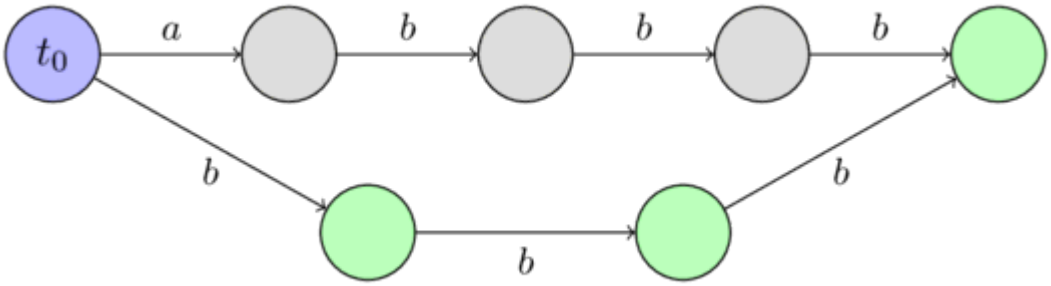
For the string $s = \text{"aba"}$:



For the string $s = \text{"abb"}$:



For the string $s = \text{"abbb"}$:



Construction in linear time

Before we describe the algorithm to construct a suffix automaton in linear time, we need to introduce several new concepts and simple proofs, which will be very important in understanding the construction.

END POSITIONS $endpos$

Consider any non-empty substring t of the string s. We will denote with $endpos(t)$ the set of all positions in the string s, in which the occurrences of t end. For instance, we have $endpos(\text{"bc"}) = \{2, 4\}$ for the string "abcbc".

We will call two substrings t_1 and t_2 $endpos$-equivalent, if their ending sets coincide: $endpos(t_1) = endpos(t_2)$. Thus all non-empty substrings of the string s can be decomposed into several **equivalence classes** according to their sets $endpos$.

It turns out, that in a suffix machine $endpos$-equivalent substrings **correspond to the same state**. In other words the number of states in a suffix automaton is equal to the number of equivalence classes among all substrings, plus the initial state. Each state of a suffix automaton corresponds to one or more substrings having the same value $endpos$.

We will later describe the construction algorithm using this assumption. We will then see, that all the required properties of a suffix automaton, except for the minimality, are fulfilled. And the minimality follows from Nerode's theorem (which will not be proven in this article).

We can make some important observations concerning the values $endpos$:

Lemma 1: Two non-empty substrings u and w (with $length(u) \le length(w)$) are $endpos$-equivalent, if and only if the string u occurs in s only in the form of a suffix of w.

The proof is obvious. If u and w have the same $endpos$ values, then u is a suffix of w and appears only in the form of a suffix of w in s. And if u is a suffix of w and appears only in the form as a suffix in s, then the values $endpos$ are equal by definition.

Lemma 2: Consider two non-empty substrings u and w (with $length(u) \le length(w)$). Then their sets $endpos$ either don't intersect at all, or $endpos(w)$ is a subset of $endpos(u)$. And it depends on if u is a suffix of w or not.

$$\begin{cases} endpos(w) \subseteq endpos(u) & \text{if } u \text{ is a suffix of } w \\ endpos(w) \cap endpos(u) = \emptyset & \text{otherwise} \end{cases}$$

Proof: If the sets $endpos(u)$ and $endpos(w)$ have at least one common element, then the strings u and w both end in that position, i.e. u is a suffix of w. But then at every occurrence of w also appears the substring u, which means that $endpos(w)$ is a subset of $endpos(u)$.

Lemma 3: Consider an $endpos$-equivalence class. Sort all the substrings in this class by decreasing length. Then in the resulting sequence each substring will be one shorter than the previous one, and at the same time will be a suffix of the previous one. In other words, in a same equivalence class, the shorter substrings are actually suffixes of the longer substrings, and they take all possible lengths in a certain interval $[x; y]$.

Proof: Fix some $endpos$-equivalence class. If it only contains one string, then the lemma is obviously true. Now let's say that the number of strings in the class is greater than one.

According to Lemma 1, two different $endpos$-equivalent strings are always in such a way, that the shorter one is a proper suffix of the longer one. Consequently, there cannot be two strings of the same length in the equivalence class.

Let's denote by w the longest, and through u the shortest string in the equivalence class. According to Lemma 1, the string u is a proper suffix of the string w. Consider now any suffix of w with a length in the interval $[length(u); length(w)]$. It is easy to see, that this suffix is also contained in the same equivalence class. Because this suffix can only appear in the form of a suffix of w in the string s (since also the shorter suffix u occurs in s only in the form of a suffix of w). Consequently, according to Lemma 1, this suffix is $endpos$-equivalent to the string w.

SUFFIX LINKS $link$

Consider some state $v \neq t_0$ in the automaton. As we know, the state v corresponds to the class of strings with the same $endpos$ values. And if we denote by w the longest of these strings, then all the other strings are suffixes of w.

We also know the first few suffixes of a string w (if we consider suffixes in descending order of their length) are all contained in this equivalence class, and all other suffixes (at least one other - the empty suffix) are in some other classes. We denote by t the biggest such suffix, and make a suffix link to it.

In other words, a **suffix link** $link(v)$ leads to the state that corresponds to the **longest suffix** of w that is in another $endpos$-equivalence class.

Here we assume that the initial state t_0 corresponds to its own equivalence class (containing only the empty string), and for convenience we set $endpos(t_0) = \{-1, 0, \dots, length(s) - 1\}$.

Lemma 4: Suffix links form a **tree** with the root t_0.

Proof: Consider an arbitrary state $v \neq t_0$. A suffix link $link(v)$ leads to a state corresponding to strings with strictly smaller length (this follows from the definition of the suffix links and from Lemma 3). Therefore, by moving along the suffix links, we will sooner or later come to the initial state t_0, which corresponds to the empty string.

Lemma 5: If we construct a tree using the sets $endpos$ (by the rule that the set of a parent node contains the sets of all children as subsets), then the structure will coincide with the tree of suffix links.

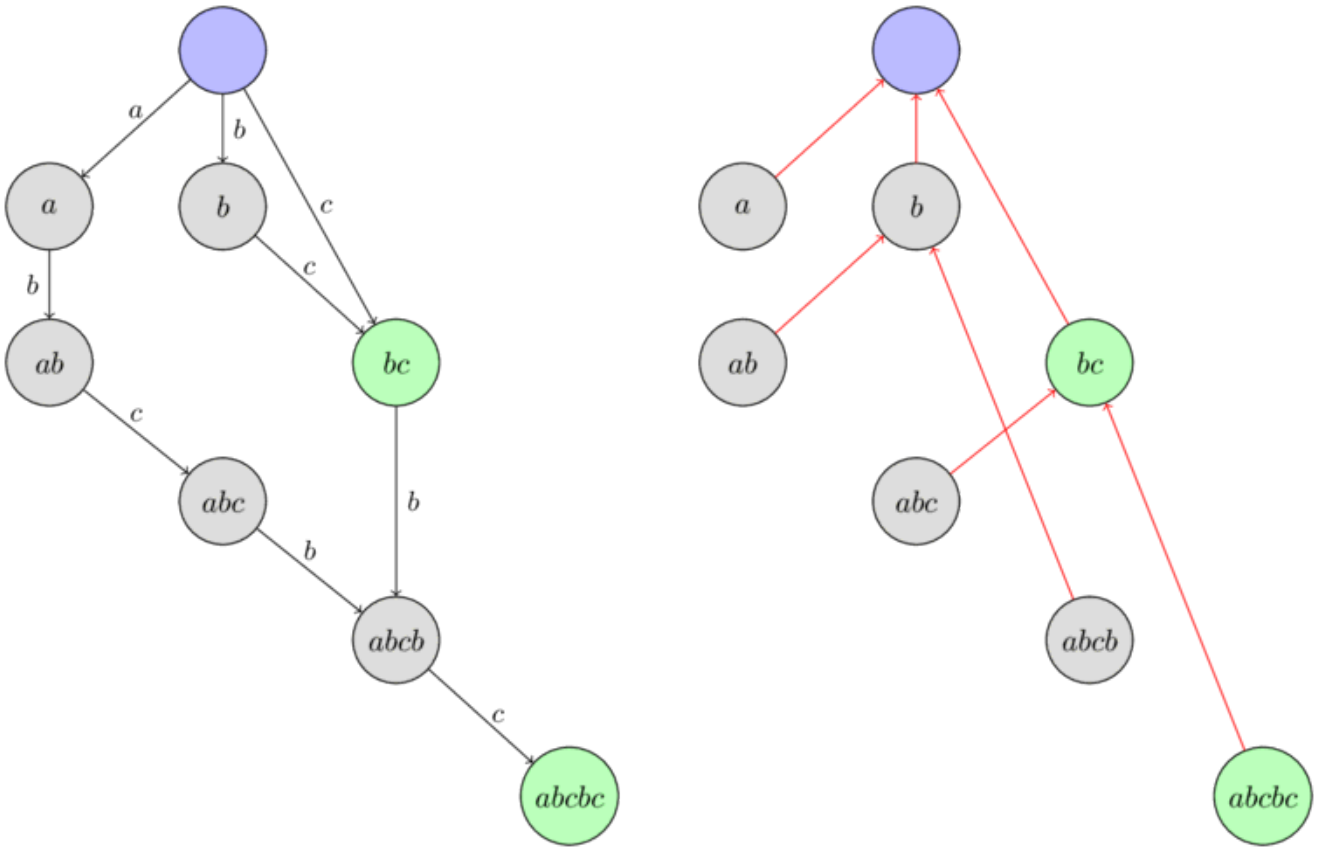
Proof: The fact that we can construct a tree using the sets $endpos$ follows directly from Lemma 2 (that any two sets either do not intersect or one is contained in the other).

Let us now consider an arbitrary state $v \neq t_0$, and its suffix link $link(v)$. From the definition of the suffix link and from Lemma 2 it follows that

$$endpos(v) \subseteq endpos(link(v)),$$

which together with the previous lemma proves the assertion: the tree of suffix links is essentially a tree of sets $endpos$.

Here is an **example** of a tree of suffix links in the suffix automaton build for the string "abcbc". The nodes are labeled with the longest substring from the corresponding equivalence class.



RECAP

Before proceeding to the algorithm itself, we recap the accumulated knowledge, and introduce a few auxiliary notations.

- The substrings of the string s can be decomposed into equivalence classes according to their end positions $endpos$.
- The suffix automaton consists of the initial state t_0, as well as of one state for each $endpos$-equivalence class.
- For each state v one or multiple substrings match. We denote by $longest(v)$ the longest such string, and through $len(v)$ its length. We denote by $shortest(v)$ the shortest such substring, and its length with $minlen(v)$. Then all the strings corresponding to this state are different suffixes of the string $longest(v)$ and have all possible lengths in the interval $[minlen(v); len(v)]$.
- For each state $v \neq t_0$ a suffix link is defined as a link, that leads to a state that corresponds to the suffix of the string $longest(v)$ of length $minlen(v) - 1$. The suffix links form a tree with the root in t_0, and at the same time this tree forms an inclusion relationship between the sets $endpos$.
- We can express $minlen(v)$ for $v \neq t_0$ using the suffix link $link(v)$ as:

$$minlen(v) = len(link(v)) + 1$$

- If we start from an arbitrary state v_0 and follow the suffix links, then sooner or later we will reach the initial state t_0. In this case we obtain a sequence of disjoint intervals $[minlen(v_i); len(v_i)]$, which in union forms the continuous interval $[0; len(v_0)]$.

ALGORITHM

Now we can proceed to the algorithm itself. The algorithm will be **online**, i.e. we will add the characters of the string one by one, and modify the automaton accordingly in each step.

To achieve linear memory consumption, we will only store the values len, $link$ and a list of transitions in each state. We will not label terminal states (but we will later show how to arrange these labels after constructing the suffix automaton).

Initially the automaton consists of a single state t_0, which will be the index 0 (the remaining states will receive the indices $1, 2, \ldots$). We assign it $len = 0$ and $link = -1$ for convenience (-1 will be a fictional, non-existing state).

Now the whole task boils down to implementing the process of **adding one character** c to the end of the current string. Let us describe this process:

- Let $last$ be the state corresponding to the entire string before adding the character c. (Initially we set $last = 0$, and we will change $last$ in the last step of the algorithm accordingly.)
- Create a new state cur, and assign it with $len(cur) = len(last) + 1$. The value $link(cur)$ is not known at the time.
- Now we do the following procedure: We start at the state $last$. While there isn't a transition through the letter c, we will add a transition to the state cur, and follow the suffix link. If at some point there already exists a transition through the letter c, then we will stop and denote this state with p.
- If we haven't found such a state p, then we reached the fictitious state -1, then we can just assign $link(cur) = 0$ and leave.
- Suppose now that we have found a state p, from which there exists a transition through the letter c. We will denote the state, to which the transition leads, with q.
- Now we have two cases. Either $len(p) + 1 = len(q)$, or not.
- If $len(p) + 1 = len(q)$, then we can simply assign $link(cur) = q$ and leave.
- Otherwise it is a bit more complicated. It is necessary to **clone** the state q: we create a new state $clone$, copy all the data from q (suffix link and transition) except the value len. We will assign $len(clone) = len(p) + 1$.

  After cloning we direct the suffix link from cur to $clone$, and also from q to clone.

  Finally we need to walk from the state p back using suffix links as long as there is a transition through c to the state q, and redirect all those to the state $clone$.
- In any of the three cases, after completing the procedure, we update the value $last$ with the state cur.

If we also want to know which states are **terminal** and which are not, we can find all terminal states after constructing the complete suffix automaton for the entire string s. To do this, we take the state corresponding to the entire string (stored in the variable $last$), and follow its suffix links until we reach the initial state. We will mark all visited states as terminal. It is easy to understand that by doing so we will mark exactly the states corresponding to all the suffixes of the string s, which are exactly the terminal states.

In the next section we will look in detail at each step and show its **correctness**.

Here we only note that, since we only create one or two new states for each character of s, the suffix automaton contains a **linear number of states**.

The linearity of the number of transitions, and in general the linearity of the runtime of the algorithm is less clear, and they will be proven after we proved the correctness.

CORRECTNESS

- We will call a transition (p, q) **continuous** if $len(p) + 1 = len(q)$. Otherwise, i.e. when $len(p) + 1 < len(q)$, the transition will be called **non-continuous**.

  As we can see from the description of the algorithm, continuous and non-continuous transitions will lead to different cases of the algorithm. Continuous transitions are fixed, and will never change again. In contrast non-continuous transition may change, when new letters are added to the string (the end of the transition edge may change).
- To avoid ambiguity we will denote the string, for which the suffix automaton was built before adding the current character c, with s.
- The algorithm begins with creating a new state cur, which will correspond to the entire string $s + c$. It is clear why we have to create a new state. Together with the new character a new equivalence class is created.
- After creating a new state we traverse by suffix links starting from the state corresponding to the entire string s. For each state we try to add a transition with the character c to the new state cur. Thus we append to each suffix of s the character c. However we can only add these new transitions, if they don't conflict with an already existing one. Therefore as soon as we find an already existing transition with c we have to stop.
- In the simplest case we reached the fictitious state -1. This means we added the transition with c to all suffixes of s. This also means, that the character c hasn't been part of the string s before. Therefore the suffix link of cur has to lead to the state 0.
- In the second case we came across an existing transition (p, q). This means that we tried to add a string $x + c$ (where x is a suffix of s) to the machine that **already exists** in the machine (the string $x + c$ already appears as a substring of s). Since we assume that the automaton for the string s is built correctly, we should not add a new transition here.

  However there is a difficulty. To which state should the suffix link from the state cur lead? We have to make a suffix link to a state, in which the longest string is exactly $x + c$, i.e. the len of this state should be $len(p) + 1$. However it is possible, that such a state doesn't yet exists, i.e. $len(q) > len(p) + 1$. In this case we have to create such a state, by **splitting** the state q.

- If the transition (p, q) turns out to be continuous, then $len(q) = len(p) + 1$. In this case everything is simple. We direct the suffix link from cur to the state q.
- Otherwise the transition is non-continuous, i.e. $len(q) > len(p) + 1$. This means that the state q corresponds to not only the suffix of $s + c$ with length $len(p) + 1$, but also to longer substrings of s. We can do nothing other than **splitting** the state q into two sub-states, so that the first one has length $len(p) + 1$.

  How can we split a state? We **clone** the state q, which gives us the state $clone$, and we set $len(clone) = len(p) + 1$. We copy all the transitions from q to $clone$, because we don't want to change the paths that traverse through q. Also we set the suffix link from $clone$ to the target of the suffix link of q, and set the suffix link of q to $clone$.

  And after splitting the state, we set the suffix link from cur to $clone$.

  In the last step we change some of the transitions to q, we redirect them to $clone$. Which transitions do we have to change? It is enough to redirect only the transitions corresponding to all the suffixes of the string $w + c$ (where w is the longest string of p), i.e. we need to continue to move along the suffix links, starting from the vertex p until we reach the fictitious state -1 or a transition that leads to a different state than q.

LINEAR NUMBER OF OPERATIONS

First we immediately make the assumption that the size of the alphabet is **constant**. If this is not the case, then it will not be possible to talk about the linear time complexity. The list of transitions from one vertex will be stored in a balanced tree, which allows you to quickly perform key search operations and adding keys. Therefore if we denote with k the size of the alphabet, then the asymptotic behavior of the algorithm will be $O(n \log k)$ with $O(n)$ memory. However if the alphabet is small enough, then you can sacrifice memory by avoiding balanced trees, and store the transitions at each vertex as an array of length k (for quick searching by key) and a dynamic list (to quickly traverse all available keys). Thus we reach the $O(n)$ time complexity for the algorithm, but at a cost of $O(nk)$ memory complexity.

So we will consider the size of the alphabet to be constant, i.e. each operation of searching for a transition on a character, adding a transition, searching for the next transition - all these operations can be done in $O(1)$.

If we consider all parts of the algorithm, then it contains three places in the algorithm in which the linear complexity is not obvious:

- The first place is the traversal through the suffix links from the state $last$, adding transitions with the character c.
- The second place is the copying of transitions when the state q is cloned into a new state $clone$.
- Third place is changing the transition leading to q, redirecting them to $clone$.

We use the fact that the size of the suffix automaton (both in the number of states and in the number of transitions) is **linear**. (The proof of the linearity of the number of states is the algorithm itself, and the proof of linearity of the number of states is given below, after the implementation of the algorithm).

Thus the total complexity of the **first and second places** is obvious, after all each operation adds only one amortized new transition to the automaton.

It remains to estimate the total complexity of the **third place**, in which we redirect transitions, that pointed originally to q, to $clone$. We denote $v = longest(p)$. This is a suffix of the string s, and with each iteration its length decreases - and therefore the position v as the suffix of the string s increases monotonically with each iteration. In this case, if before the first iteration of the loop, the corresponding string v was at the depth k ($k \geq 2$) from $last$ (by counting the depth as the number of suffix links), then after the last iteration the string $v + c$ will be a 2-th suffix link on the path from cur (which will become the new value $last$).

Thus, each iteration of this loop leads to the fact that the position of the string $longest(link(link(last))$ as a suffix of the current string will monotonically increase. Therefore this cycle cannot be executed more than n iterations, which was required to prove.

IMPLEMENTATION

First we describe a data structure that will store all information about a specific transition (len, $link$ and the list of transitions). If necessary you can add a terminal flag here, as well as other information. We will store the list of transitions in the form of a map, which allows us to achieve total $O(n)$ memory and $O(n \log k)$ time for processing the entire string.

```
struct state {
    int len, link;
    map<char, int> next;
};
```

The suffix automaton itself will be stored in an array of these structures $state$. We store the current size sz and also the variable $last$, the state corresponding to the entire string at the moment.

```
const int MAXLEN = 100000;
state st[MAXLEN * 2];
int sz, last;
```

We give a function that initializes a suffix automaton (creating a suffix automaton with a single state).

```
void sa_init() {
    st[0].len = 0;
    st[0].link = -1;
    sz++;
    last = 0;
}
```

And finally we give the implementation of the main function - which adds the next character to the end of the current line, rebuilding the machine accordingly.

```
void sa_extend(char c) {
    int cur = sz++;
    st[cur].len = st[last].len + 1;
    int p = last;
    while (p != -1 && !st[p].next.count(c)) {
        st[p].next[c] = cur;
        p = st[p].link;
    }
    if (p == -1) {
        st[cur].link = 0;
    } else {
        int q = st[p].next[c];
        if (st[p].len + 1 == st[q].len) {
            st[cur].link = q;
        } else {
            int clone = sz++;
            st[clone].len = st[p].len + 1;
            st[clone].next = st[q].next;
            st[clone].link = st[q].link;
            while (p != -1 && st[p].next[c] == q) {
                st[p].next[c] = clone;
                p = st[p].link;
            }
            st[q].link = st[cur].link = clone;
        }
    }
    last = cur;
}
```

As mentioned above, if you sacrifice memory ($O(nk)$, where k is the size of the alphabet), then you can achieve the build time of the machine in $O(n)$, even for any alphabet size k. But for this you will have to store an array of size k in each state (for quickly jumping to the transition of the letter), and additional a list of all transitions (to quickly iterate over the transitions them).

Additional properties

NUMBER OF STATES

The number of states in a suffix automaton of the string s of length n **doesn't exceed** $2n - 1$ (for $n \geq 2$).

The proof is the construction algorithm itself, since initially the automaton consists of one state, and in the first and second iteration only a single state will be created, and in the remaining $n - 2$ steps at most 2 states will be created each.

However we can also **show** this estimation **without knowing the algorithm**. Let us recall that the number of states is equal to the number of different sets $endpos$. In addition these sets $endpos$ form a tree (a parent vertex contains all children sets in his set). Consider this tree and transform it a little bit: as long as it has an internal vertex with only one child (which means that the set of the child misses at least one position from the parent set), we create a new child with the set of the missing positions. In the end we have a tree in which each inner vertex has a degree greater than one, and the number of leaves does not exceed n. Therefore there are no more than $2n - 1$ vertices in such a tree.

This bound of the number of states can actually be achieved for each n. A possible string is:

$$\text{"abbb} \ldots \text{bbb"}$$

In each iteration, starting at the third one, the algorithm will split a state, resulting in exactly $2n - 1$ states.

NUMBER OF TRANSITIONS

The number of transitions in a suffix automaton of a string s of length n **doesn't exceed** $3n - 4$ (for $n \geq 3$).

Let us prove this:

Let us first estimate the number of continuous transitions. Consider a spanning tree of the longest paths in the automaton starting in the state t_0. This skeleton will consist of only the continuous edges, and therefore their number is less than the number of states, i.e. it does not exceed $2n - 2$.

Now let us estimate the number of non-continuous transitions. Let the current non-continuous transition be (p, q) with the character c. We take the correspondent string $u + c + w$, where the string u corresponds to the longest path from the initial state to p, and w to the longest path from q to any terminal state. On one hand, each such string $u + c + w$ for each incomplete strings will be different (since the strings u and w are formed only by complete transitions). On the other hand each such string $u + c + w$, by the definition of the terminal states, will be a suffix of the entire string s. Since there are only n non-empty suffixes of s, and none of the strings $u + c + w$ can contain s (because the entire string only contains complete transitions), the total number of incomplete transitions does not exceed $n - 1$.

Combining these two estimates gives us the bound $3n - 3$. However, since the maximum number of states can only be achieved with the test case $\text{"abbb}\dots\text{bbb"}$ and this case has clearly less than $3n - 3$ transitions, we get the tighter bound of $3n - 4$ for the number of transitions in a suffix automaton.

This bound can also be achieved with the string:

$$\text{"abbb} \dots \text{bbbc"}$$

Applications

Here we look at some tasks that can be solved using the suffix automaton. For the simplicity we assume that the alphabet size k is constant, which allows us to consider the complexity of appending a character and the traversal as constant.

CHECK FOR OCCURRENCE

Given a text T, and multiple patterns P. We have to check whether or not the strings P appear as a substring of T.

We build a suffix automaton of the text T in $O(length(T))$ time. To check if a pattern P appears in T, we follow the transitions, starting from t_0, according to the characters of P. If at some point there doesn't exists a transition, then the pattern P doesn't appear as a substring of T. If we can process the entire string P this way, then the string appears in T.

It is clear that this will take $O(length(P))$ time for each string P. Moreover the algorithm actually finds the length of the longest prefix of P that appears in the text.

NUMBER OF DIFFERENT SUBSTRINGS

Given a string S. You want to compute the number of different substrings.

Let us build a suffix automaton for the string S.

Each substring of S corresponds to some path in the automaton. Therefore the number of different substrings is equal to the number of different paths in the automaton starting at t_0.

Given that the suffix automaton is a directed acyclic graph, the number of different ways can be computed using dynamic programming.

Namely, let $d[v]$ be the number of ways, starting at the state v (including the path of length zero). Then we have the recursion:

$$d[v] = 1 + \sum_{w:(v,w,c)\in DAWG} d[w]$$

I.e. $d[v]$ can be expressed as the sum of answers for all ends of the transitions of v.

The number of different substrings is the value $d[t_0] - 1$ (since we don't count the empty substring).

Total time complexity: $O(length(S))$

Alternatively, we can take advantage of the fact that each state v matches to substrings of length $[minlen(v), len(v)]$. Therefore, given $minlen(v) = 1 + len(link(v))$, we have total distinct substrings at state v being $len(v) - minlen(v) + 1 = len(v) - (1 + len(link(v))) + 1 = len(v) - len(link(v))$.

This is demonstrated succinctly below:

```
long long get_diff_strings(){
    long long tot = 0;
    for(int i = 1; i < sz; i++) {
        tot += st[i].len - st[st[i].link].len;
    }
    return tot;
}
```

While this is also $O(length(S))$, it requires no extra space and no recursive calls, consequently running faster in practice.

TOTAL LENGTH OF ALL DIFFERENT SUBSTRINGS

Given a string S. We want to compute the total length of all its various substrings.

The solution is similar to the previous one, only now it is necessary to consider two quantities for the dynamic programming part: the number of different substrings $d[v]$ and their total length $ans[v]$.

We already described how to compute $d[v]$ in the previous task. The value $ans[v]$ can be computed using the recursion:

$$ans[v] = \sum_{w:(v,w,c)\in DAWG} d[w] + ans[w]$$

We take the answer of each adjacent vertex w, and add to it $d[w]$ (since every substring is one character longer when starting from the state v).

Again this task can be computed in $O(length(S))$ time.

Alternatively, we can, again, take advantage of the fact that each state v matches to substrings of length $[minlen(v), len(v)]$. Since $minlen(v) = 1 + len(link(v))$ and the arithmetic series formula $S_n = n \cdot \frac{a_1+a_n}{2}$ (where S_n denotes the sum of n terms, a_1 representing the first term, and a_n representing the last), we can compute the length of substrings at a state in constant time. We then sum up these totals for each state $v \neq t_0$ in the automaton. This is shown by the code below:

```
long long get_tot_len_diff_substings() {
    long long tot = 0;
    for(int i = 1; i < sz; i++) {
        long long shortest = st[st[i].link].len + 1;
        long long longest = st[i].len;

        long long num_strings = longest - shortest + 1;
        long long cur = num_strings * (longest + shortest) / 2;
        tot += cur;
    }
    return tot;
}
```

This approach runs in $O(length(S))$ time, but experimentally runs 20x faster than the memoized dynamic programming version on randomized strings. It requires no extra space and no recursion.

LEXICOGRAPHICALLY k-TH SUBSTRING

Given a string S. We have to answer multiple queries. For each given number K_i we have to find the K_i-th string in the lexicographically ordered list of all substrings.

The solution to this problem is based on the idea of the previous two problems. The lexicographically k-th substring corresponds to the lexicographically k-th path in the suffix automaton. Therefore after counting the number of paths from each state, we can easily search for the k-th path starting from the root of the automaton.

This takes $O(length(S))$ time for preprocessing and then $O(length(ans) \cdot k)$ for each query (where ans is the answer for the query and k is the size of the alphabet).

SMALLEST CYCLIC SHIFT

Given a string S. We want to find the lexicographically smallest cyclic shift.

We construct a suffix automaton for the string $S + S$. Then the automaton will contain in itself as paths all the cyclic shifts of the string S.

Consequently the problem is reduced to finding the lexicographically smallest path of length $length(S)$, which can be done in a trivial way: we start in the initial state and greedily pass through the transitions with the minimal character.

Total time complexity is $O(length(S))$.

NUMBER OF OCCURRENCES

For a given text T. We have to answer multiple queries. For each given pattern P we have to find out how many times the string P appears in the string T as a substring.

We construct the suffix automaton for the text T.

Next we do the following preprocessing: for each state v in the automaton we calculate the number $cnt[v]$ that is equal to the size of the set $endpos(v)$. In fact all strings corresponding to the same state v appear in the text T an equal amount of times, which is equal to the number of positions in the set $endpos$.

However we cannot construct the sets $endpos$ explicitly, therefore we only consider their sizes cnt.

To compute them we proceed as follows. For each state, if it was not created by cloning (and if it is not the initial state t_0), we initialize it with $cnt = 1$. Then we will go through all states in decreasing order of their length len, and add the current value $cnt[v]$ to the suffix links:

$$cnt[link(v)] \mathrel{+}= cnt[v]$$

This gives the correct value for each state.

Why is this correct? The total number of states obtained *not* via cloning is exactly $length(T)$, and the first i of them appeared when we added the first i characters. Consequently for each of these states we count the corresponding position at which it was processed. Therefore initially we have $cnt = 1$ for each such state, and $cnt = 0$ for all other.

Then we apply the following operation for each v: $cnt[link(v)] \mathrel{+}= cnt[v]$. The meaning behind this is, that if a string v appears $cnt[v]$ times, then also all its suffixes appear at the exact same end positions, therefore also $cnt[v]$ times.

Why don't we overcount in this procedure (i.e. don't count some positions twice)? Because we add the positions of a state to only one other state, so it can not happen that one state directs its positions to another state twice in two different ways.

Thus we can compute the quantities cnt for all states in the automaton in $O(length(T))$ time.

After that answering a query by just looking up the value $cnt[t]$, where t is the state corresponding to the pattern, if such a state exists. Otherwise answer with 0. Answering a query takes $O(length(P))$ time.

FIRST OCCURRENCE POSITION

Given a text T and multiple queries. For each query string P we want to find the position of the first occurrence of P in the string T (the position of the beginning of P).

We again construct a suffix automaton. Additionally we precompute the position $firstpos$ for all states in the automaton, i.e. for each state v we want to find the position $firstpos[v]$ of the end of the first occurrence. In other words, we want to find in advance the minimal element of each set $endpos$ (since obviously cannot maintain all sets $endpos$ explicitly).

To maintain these positions $firstpos$ we extend the function `sa_extend()`. When we create a new state cur, we set:

$$firstpos(cur) = len(cur) - 1$$

And when we clone a vertex q as $clone$, we set:

$$firstpos(clone) = firstpos(q)$$

(since the only other option for a value would be $firstpos(cur)$ which is definitely too big)

Thus the answer for a query is simply $firstpos(t) - length(P) + 1$, where t is the state corresponding to the string P. Answering a query again takes only $O(length(P))$ time.

ALL OCCURRENCE POSITIONS

This time we have to display all positions of the occurrences in the string T.

Again we construct a suffix automaton for the text T. Similar as in the previous task we compute the position $firstpos$ for all states.

Clearly $firstpos(t)$ is part of the answer, if t is the state corresponding to a query string P. So we took into account the state of the automaton containing P. What other states do we need to take into account? All states that correspond to strings for which P is a suffix. In other words we need to find all the states that can reach the state t via suffix links.

Therefore to solve the problem we need to save for each state a list of suffix references leading to it. The answer to the query then will then contain all $firstpos$ for each state that we can find on a DFS / BFS starting from the state t using only the suffix references.

Overall, this requires $O(length(T))$ for preprocessing and $O(length(P) + answer(P))$ for each request, where $answer(P)$ — this is the size of the answer.

First, we walk down the automaton for each character in the pattern to find our starting node requiring $O(length(P))$. Then, we use our workaround which will work in time $O(answer(P))$, because we will not visit a state twice (because only one suffix link leaves each state, so there cannot be two different paths leading to the same state).

We only must take into account that two different states can have the same $firstpos$ value. This happens if one state was obtained by cloning another. However, this doesn't ruin the complexity, since each state can only have at most one clone.

Moreover, we can also get rid of the duplicate positions, if we don't output the positions from the cloned states. In fact a state, that a cloned state can reach, is also reachable from the original state. Thus if we remember the flag `is_cloned` for each state, we can simply ignore the cloned states and only output $firstpos$ for all other states.

Here are some implementation sketches:

```
struct state {
    ...
    bool is_clone;
    int first_pos;
    vector<int> inv_link;
};

// after constructing the automaton
for (int v = 1; v < sz; v++) {
    st[st[v].link].inv_link.push_back(v);
}

// output all positions of occurrences
void output_all_occurrences(int v, int P_length) {
    if (!st[v].is_clone)
        cout << st[v].first_pos - P_length + 1 << endl;
    for (int u : st[v].inv_link)
        output_all_occurrences(u, P_length);
}
```

SHORTEST NON-APPEARING STRING

Given a string S and a certain alphabet. We have to find a string of the smallest length, that doesn't appear in S.

We will apply dynamic programming on the suffix automaton built for the string S.

Let $d[v]$ be the answer for the node v, i.e. we already processed part of the substring, are currently in the state v, and want to find the smallest number of characters that have to be added to find a non-existent transition. Computing $d[v]$ is very simple. If there is not transition using at least one character of the alphabet, then $d[v] = 1$. Otherwise one character is not enough, and so we need to take the minimum of all answers of all transitions:

$$d[v] = 1 + \min_{w:(v,w,c)\in SA} d[w].$$

The answer to the problem will be $d[t_0]$, and the actual string can be restored using the computed array $d[]$.

LONGEST COMMON SUBSTRING OF TWO STRINGS

Given two strings S and T. We have to find the longest common substring, i.e. such a string X that appears as a substring in S and also in T.

We construct a suffix automaton for the string S.

We will now take the string T, and for each prefix look for the longest suffix of this prefix in S. In other words, for each position in the string T, we want to find the longest common substring of S and T ending in that position.

For this we will use two variables, the **current state** v, and the **current length** l. These two variables will describe the current matching part: its length and the state that corresponds to it.

Initially $v = t_0$ and $l = 0$, i.e. the match is empty.

Now let us describe how we can add a character $T[i]$ and recalculate the answer for it.

- If there is a transition from v with the character $T[i]$, then we simply follow the transition and increase l by one.
- If there is no such transition, we have to shorten the current matching part, which means that we need to follow the suffix link: $v = link(v)$. At the same time, the current length has to be shortened. Obviously we need to assign $l = len(v)$, since after passing through the suffix link we end up in state whose corresponding longest string is a substring.
- If there is still no transition using the required character, we repeat and again go through the suffix link and decrease l, until we find a transition or we reach the fictional state -1 (which means that the symbol $T[i]$ doesn't appear at all in S, so we assign $v = l = 0$).

The answer to the task will be the maximum of all the values l.

The complexity of this part is $O(length(T))$, since in one move we can either increase l by one, or make several passes through the suffix links, each one ends up reducing the value l.

Implementation:

```
string lcs (string S, string T) {
    sa_init();
    for (int i = 0; i < S.size(); i++)
        sa_extend(S[i]);

    int v = 0, l = 0, best = 0, bestpos = 0;
    for (int i = 0; i < T.size(); i++) {
        while (v && !st[v].next.count(T[i])) {
            v = st[v].link ;
            l = st[v].len;
        }
        if (st[v].next.count(T[i])) {
            v = st [v].next[T[i]];
            l++;
        }
        if (l > best) {
            best = l;
            bestpos = i;
        }
    }
    return T.substr(bestpos - best + 1, best);
}
```

LARGEST COMMON SUBSTRING OF MULTIPLE STRINGS

There are k strings S_i given. We have to find the longest common substring, i.e. such a string X that appears as substring in each string S_i.

We join all strings into one large string T, separating the strings by a special characters D_i (one for each string):

$$T = S_1 + D_1 + S_2 + D_2 + \cdots + S_k + D_k.$$

Then we construct the suffix automaton for the string T.

Now we need to find a string in the machine, which is contained in all the strings S_i, and this can be done by using the special added characters. Note that if a substring is included in some string S_j, then in the suffix automaton exists a path starting from this substring containing the character D_j and not containing the other characters $D_1, \ldots, D_{j-1}, D_{j+1}, \ldots, D_k$.

Thus we need to calculate the attainability, which tells us for each state of the machine and each symbol D_i if there exists such a path. This can easily be computed by DFS or BFS and dynamic programming. After that, the answer to the problem will be the string $longest(v)$ for the state v, from which the paths were exists for all special characters.

4.2.3 Lyndon factorization

Lyndon factorization

First let us define the notion of the Lyndon factorization.

A string is called **simple** (or a Lyndon word), if it is strictly **smaller than** any of its own nontrivial **suffixes**. Examples of simple strings are: a, b, ab, aab, abb, $ababb$, $abcd$. It can be shown that a string is simple, if and only if it is strictly **smaller than** all its nontrivial **cyclic shifts**.

Next, let there be a given string s. The **Lyndon factorization** of the string s is a factorization $s = w_1 w_2 \dots w_k$, where all strings w_i are simple, and they are in non-increasing order $w_1 \geq w_2 \geq \dots \geq w_k$.

It can be shown, that for any string such a factorization exists and that it is unique.

Duval algorithm

The Duval algorithm constructs the Lyndon factorization in $O(n)$ time using $O(1)$ additional memory.

First let us introduce another notion: a string t is called **pre-simple**, if it has the form $t = ww \dots w\overline{w}$, where w is a simple string and $\overline{w}$ is a prefix of w (possibly empty). A simple string is also pre-simple.

The Duval algorithm is greedy. At any point during its execution, the string s will actually be divided into three strings $s = s_1 s_2 s_3$, where the Lyndon factorization for s_1 is already found and finalized, the string s_2 is pre-simple (and we know the length of the simple string in it), and s_3 is completely untouched. In each iteration the Duval algorithm takes the first character of the string s_3 and tries to append it to the string s_2. It s_2 is no longer pre-simple, then the Lyndon factorization for some part of s_2 becomes known, and this part goes to s_1.

Let's describe the algorithm in more detail. The pointer i will always point to the beginning of the string s_2. The outer loop will be executed as long as $i < n$. Inside the loop we use two additional pointers, j which points to the beginning of s_3, and k which points to the current character that we are currently comparing to. We want to add the character $s[j]$ to the string s_2, which requires a comparison with the character $s[k]$. There can be three different cases:

- $s[j] = s[k]$: if this is the case, then adding the symbol $s[j]$ to s_2 doesn't violate its pre-simplicity. So we simply increment the pointers j and k.
- $s[j] > s[k]$: here, the string $s_2 + s[j]$ becomes simple. We can increment j and reset k back to the beginning of s_2, so that the next character can be compared with the beginning of the simple word.
- $s[j] < s[k]$: the string $s_2 + s[j]$ is no longer pre-simple. Therefore we will split the pre-simple string s_2 into its simple strings and the remainder, possibly empty. The simple string will have the length $j - k$. In the next iteration we start again with the remaining s_2.

IMPLEMENTATION

Here we present the implementation of the Duval algorithm, which will return the desired Lyndon factorization of a given string s.

```
vector<string> duval(string const& s) {
    int n = s.size();
    int i = 0;
    vector<string> factorization;
    while (i < n) {
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j])
                k = i;
            else
                k++;
            j++;
        }
        while (i <= k) {
            factorization.push_back(s.substr(i, j - k));
            i += j - k;
        }
    }
    return factorization;
}
```

COMPLEXITY

Let us estimate the running time of this algorithm.

The **outer while loop** does not exceed n iterations, since at the end of each iteration i increases. Also the second inner while loop runs in $O(n)$, since is only outputs the final factorization.

So we are only interested in the **first inner while loop**. How many iterations does it perform in the worst case? It's easy to see that the simple words that we identify in each iteration of the outer loop are longer than the remainder that we additionally compared. Therefore also the sum of the remainders will be smaller than n, which means that we only perform at most $O(n)$ iterations of the first inner while loop. In fact the total number of character comparisons will not exceed $4n - 3$.

Finding the smallest cyclic shift

Let there be a string s. We construct the Lyndon factorization for the string $s + s$ (in $O(n)$ time). We will look for a simple string in the factorization, which starts at a position less than n (i.e. it starts in the first instance of s), and ends in a position greater than or equal to n (i.e. in the second instance) of s). It is stated, that the position of the start of this simple string will be the beginning of the desired smallest cyclic shift. This can be easily verified using the definition of the Lyndon decomposition.

The beginning of the simple block can be found easily - just remember the pointer i at the beginning of each iteration of the outer loop, which indicated the beginning of the current pre-simple string.

So we get the following implementation:

```
string min_cyclic_string(string s) {
    s += s;
    int n = s.size();
    int i = 0, ans = 0;
    while (i < n / 2) {
        ans = i;
        int j = i + 1, k = i;
        while (j < n && s[k] <= s[j]) {
            if (s[k] < s[j])
                k = i;
            else
                k++;
            j++;
        }
        while (i <= k)
            i += j - k;
    }
    return s.substr(ans, n / 2);
}
```

4.3 Tasks

4.3.1 Expression parsing

A string containing a mathematical expression containing numbers and various operators is given. We have to compute the value of it in $O(n)$, where n is the length of the string.

The algorithm discussed here translates an expression into the so-called **reverse Polish notation** (explicitly or implicitly), and evaluates this expression.

Reverse Polish notation

The reverse Polish notation is a form of writing mathematical expressions, in which the operators are located after their operands. For example the following expression

$$a + b * c * d + (e - f) * (g * h + i)$$

can be written in reverse Polish notation in the following way:

$$abc * d * + ef - gh * i + *+$$

The reverse Polish notation was developed by the Australian philosopher and computer science specialist Charles Hamblin in the mid 1950s on the basis of the Polish notation, which was proposed in 1920 by the Polish mathematician Jan Łukasiewicz.

The convenience of the reverse Polish notation is, that expressions in this form are very **easy to evaluate** in linear time. We use a stack, which is initially empty. We will iterate over the operands and operators of the expression in reverse Polish notation. If the current element is a number, then we put the value on top of the stack, if the current element is an operator, then we get the top two elements from the stack, perform the operation, and put the result back on top of the stack. In the end there will be exactly one element left in the stack, which will be the value of the expression.

Obviously this simple evaluation runs in $O(n)$ time.

Parsing of simple expressions

For the time being we only consider a simplified problem: we assume that all operators are **binary** (i.e. they take two arguments), and all are **left-associative** (if the priorities are equal, they get executed from left to right). Parentheses are allowed.

We will set up two stacks: one for numbers, and one for operators and parentheses. Initially both stacks are empty. For the second stack we will maintain the condition that all operations are ordered by strict descending priority. If there are parenthesis on the stack, than each block of operators (corresponding to one pair of parenthesis) is ordered, and the entire stack is not necessarily ordered.

We will iterate over the characters of the expression from left to right. If the current character is a digit, then we put the value of this number on the stack. If the current character is an opening parenthesis, then we put it on the stack. If the current character is a closing parenthesis, the we execute all operators on the stack until we reach the opening bracket (in other words we perform all operations inside the parenthesis). Finally if the current character is an operator, then while the top of the stack has an operator with the same or higher priority, we will execute this operation, and put the new operation on the stack.

After we processed the entire string, some operators might still be in the stack, so we execute them.

Here is the implementation of this method for the four operators $+ - * /$:

```
bool delim(char c) {
    return c == ' ';
}

bool is_op(char c) {
    return c == '+' || c == '-' || c == '*' || c == '/';
}
```

```
int priority (char op) {
    if (op == '+' || op == '-')
        return 1;
    if (op == '*' || op == '/')
        return 2;
    return -1;
}

void process_op(stack<int>& st, char op) {
    int r = st.top(); st.pop();
    int l = st.top(); st.pop();
    switch (op) {
        case '+': st.push(l + r); break;
        case '-': st.push(l - r); break;
        case '*': st.push(l * r); break;
        case '/': st.push(l / r); break;
    }
}

int evaluate(string& s) {
    stack<int> st;
    stack<char> op;
    for (int i = 0; i < (int)s.size(); i++) {
        if (delim(s[i]))
            continue;

        if (s[i] == '(') {
            op.push('(');
        } else if (s[i] == ')') {
            while (op.top() != '(') {
                process_op(st, op.top());
                op.pop();
            }
            op.pop();
        } else if (is_op(s[i])) {
            char cur_op = s[i];
            while (!op.empty() && priority(op.top()) >= priority(cur_op)) {
                process_op(st, op.top());
                op.pop();
            }
            op.push(cur_op);
        } else {
            int number = 0;
            while (i < (int)s.size() && isalnum(s[i]))
                number = number * 10 + s[i++] - '0';
            --i;
            st.push(number);
        }
    }

    while (!op.empty()) {
        process_op(st, op.top());
        op.pop();
    }
    return st.top();
}
```

Thus we learned how to calculate the value of an expression in $O(n)$, at the same time we implicitly used the reverse Polish notation. By slightly modifying the above implementation it is also possible to obtain the expression in reverse Polish notation in an explicit form.

Unary operators

Now suppose that the expression also contains **unary** operators (operators that take one argument). The unary plus and unary minus are common examples of such operators.

One of the differences in this case, is that we need to determine whether the current operator is a unary or a binary one.

You can notice, that before an unary operator, there always is another operator or an opening parenthesis, or nothing at all (if it is at the very beginning of the expression). On the contrary before a binary operator there will always be an operand (number) or a closing parenthesis. Thus it is easy to flag whether the next operator can be unary or not.

Additionally we need to execute a unary and a binary operator differently. And we need to chose the priority of a unary operator higher than all of the binary operators.

In addition it should be noted, that some unary operators (e.g. unary plus and unary minus) are actually **right-associative**.

Right-associativity

Right-associative means, that whenever the priorities are equal, the operators must be evaluated from right to left.

As noted above, unary operators are usually right-associative. Another example for an right-associative operator is the exponentiation operator ($a \wedge b \wedge c$ is usually perceived as a^{b^c} and not as $(a^b)^c$).

What difference do we need to make in order to correctly handle right-associative operators? It turns out that the changes are very minimal. The only difference will be, if the priorities are equal we will postpone the execution of the right-associative operation.

The only line that needs to be replaced is

```
while (!op.empty() && priority(op.top()) >= priority(cur_op))
```

with

```
while (!op.empty() && (
        (left_assoc(cur_op) && priority(op.top()) >= priority(cur_op)) ||
        (!left_assoc(cur_op) && priority(op.top()) > priority(cur_op))
    ))
```

where `left_assoc` is a function that decides if an operator is left_associative or not.

Here is an implementation for the binary operators $+ - * /$ and the unary operators $+$ and $-$.

```
bool delim(char c) {
    return c == ' ';
}

bool is_op(char c) {
    return c == '+' || c == '-' || c == '*' || c == '/';
}

bool is_unary(char c) {
    return c == '+' || c=='-';
}

int priority (char op) {
    if (op < 0) // unary operator
        return 3;
    if (op == '+' || op == '-')
        return 1;
    if (op == '*' || op == '/')
        return 2;
    return -1;
}

void process_op(stack<int>& st, char op) {
    if (op < 0) {
        int l = st.top(); st.pop();
        switch (-op) {
            case '+': st.push(l); break;
            case '-': st.push(-l); break;
        }
    } else {
        int r = st.top(); st.pop();
        int l = st.top(); st.pop();
        switch (op) {
            case '+': st.push(l + r); break;
            case '-': st.push(l - r); break;
            case '*': st.push(l * r); break;
            case '/': st.push(l / r); break;
        }
    }
}

int evaluate(string& s) {
    stack<int> st;
    stack<char> op;
    bool may_be_unary = true;
    for (int i = 0; i < (int)s.size(); i++) {
```

```
        if (delim(s[i]))
            continue;

        if (s[i] == '(') {
            op.push('(');
            may_be_unary = true;
        } else if (s[i] == ')') {
            while (op.top() != '(') {
                process_op(st, op.top());
                op.pop();
            }
            op.pop();
            may_be_unary = false;
        } else if (is_op(s[i])) {
            char cur_op = s[i];
            if (may_be_unary && is_unary(cur_op))
                cur_op = -cur_op;
            while (!op.empty() && (
                    (cur_op >= 0 && priority(op.top()) >= priority(cur_op)) ||
                    (cur_op < 0 && priority(op.top()) > priority(cur_op))
                )) {
                process_op(st, op.top());
                op.pop();
            }
            op.push(cur_op);
            may_be_unary = true;
        } else {
            int number = 0;
            while (i < (int)s.size() && isalnum(s[i]))
                number = number * 10 + s[i++] - '0';
            --i;
            st.push(number);
            may_be_unary = false;
        }
    }

    while (!op.empty()) {
        process_op(st, op.top());
        op.pop();
    }
    return st.top();
}
```

4.3.3 Finding repetitions

Given a string s of length n.

A **repetition** is two occurrences of a string in a row. In other words a repetition can be described by a pair of indices $i < j$ such that the substring $s[i \dots j]$ consists of two identical strings written after each other.

The challenge is to **find all repetitions** in a given string s. Or a simplified task: find **any** repetition or find the **longest** repetition.

The algorithm described here was published in 1982 by Main and Lorentz.

Example

Consider the repetitions in the following example string:

$$acababaee$$

The string contains the following three repetitions:

- $s[2 \dots 5] = abab$
- $s[3 \dots 6] = baba$
- $s[7 \dots 8] = ee$

Another example:

$$abaaba$$

Here there are only two repetitions

- $s[0 \dots 5] = abaaba$
- $s[2 \dots 3] = aa$

Number of repetitions

In general there can be up to $O(n^2)$ repetitions in a string of length n. An obvious example is a string consisting of n times the same letter, in this case any substring of even length is a repetition. In general any periodic string with a short period will contain a lot of repetitions.

On the other hand this fact does not prevent computing the number of repetitions in $O(n \log n)$ time, because the algorithm can give the repetitions in compressed form, in groups of several pieces at once.

There is even the concept, that describes groups of periodic substrings with tuples of size four. It has been proven that we the number of such groups is at most linear with respect to the string length.

Also, here are some more interesting results related to the number of repetitions:

- The number of primitive repetitions (those whose halves are not repetitions) is at most $O(n \log n)$.
- If we encode repetitions with tuples of numbers (called Crochemore triples) $(i,\ p,\ r)$ (where i is the position of the beginning, p the length of the repeating substring, and r the number of repetitions), then all repetitions can be described with $O(n \log n)$ such triples.
- Fibonacci strings, defined as

$$t_0 = a,$$

$$t_1 = b,$$

$$t_i = t_{i-1} + t_{i-2},$$

are "strongly" periodic. The number of repetitions in the Fibonacci string f_i, even in the compressed with Crochemore triples, is $O(f_n \log f_n)$. The number of primitive repetitions is also $O(f_n \log f_n)$.

Main-Lorentz algorithm

The idea behind the Main-Lorentz algorithm is **divide-and-conquer**.

It splits the initial string into halves, and computes the number of repetitions that lie completely in each halve by two recursive calls. Then comes the difficult part. The algorithm finds all repetitions starting in the first half and ending in the second half (which we will call **crossing repetitions**). This is the essential part of the Main-Lorentz algorithm, and we will discuss it in detail here.

The complexity of divide-and-conquer algorithms is well researched. The master theorem says, that we will end up with an $O(n \log n)$ algorithm, if we can compute the crossing repetitions in $O(n)$ time.

SEARCH FOR CROSSING REPETITIONS

So we want to find all such repetitions that start in the first half of the string, let's call it u, and end in the second half, let's call it v:

$$s = u + v$$

Their lengths are approximately equal to the length of s divided by two.

Consider an arbitrary repetition and look at the middle character (more precisely the first character of the second half of the repetition). I.e. if the repetition is a substring $s[i \dots j]$, then the middle character is $(i + j + 1)/2$.

We call a repetition **left** or **right** depending on which string this character is located - in the string u or in the string v. In other words a string is called left, if the majority of it lies in u, otherwise we call it right.

We will now discuss how to find **all left repetitions**. Finding all right repetitions can be done in the same way.

Let us denote the length of the left repetition by $2l$ (i.e. each half of the repetition has length l). Consider the first character of the repetition falling into the string v (it is at position $|u|$ in the string s). It coincides with the character l positions before it, let's denote this position $cntr$.

We will fixate this position $cntr$, and **look for all repetitions at this position** $cntr$.

For example:

$$\underset{cntr}{c\ a}\ c \mid a\ d\ a$$

The vertical lines divides the two halves. Here we fixated the position $cntr = 1$, and at this position we find the repetition $caca$.

It is clear, that if we fixate the position $cntr$, we simultaneously fixate the length of the possible repetitions: $l = |u| - cntr$. Once we know how to find these repetitions, we will iterate over all possible values for $cntr$ from 0 to $|u| - 1$, and find all left crossover repetitions of length $l = |u|,\ |u| - 1,\ \dots, 1$.

CRITERION FOR LEFT CROSSING REPETITIONS

Now, how can we find all such repetitions for a fixated $cntr$? Keep in mind that there still can be multiple such repetitions.

Let's again look at a visualization, this time for the repetition $abcabc$:

$$\overbrace{a}^{l_1}$$

Here we denoted the lengths of the two pieces of the repetition with l_1 and l_2: l_1 is the length of the repetition up to the position $cntr-1$, and l_2 is the length of the repetition from $cntr$ to the end of the half of the repetition. We have $2l = l_1 + l_2 + l_1 + l_2$ as the total length of the repetition.

Let us generate **necessary and sufficient** conditions for such a repetition at position $cntr$ of length $2l = 2(l_1 + l_2) = 2(|u| - cntr)$:

- Let k_1 be the largest number such that the first k_1 characters before the position $cntr$ coincide with the last k_1 characters in the string u:

$$u[cntr - k_1 \dots cntr - 1] = u[|u| - k_1 \dots |u| - 1]$$

- Let k_2 be the largest number such that the k_2 characters starting at position $cntr$ coincide with the first k_2 characters in the string v:

$$u[cntr \dots cntr + k_2 - 1] = v[0 \dots k_2 - 1]$$

- Then we have a repetition exactly for any pair $(l_1,\ l_2)$ with

$$l_1 \le k_1,$$

$$l_2 \le k_2.$$

To summarize:

- We fixate a specific position $cntr$.
- All repetition which we will find now have length $2l = 2(|u| - cntr)$. There might be multiple such repetitions, they depend on the lengths l_1 and $l_2 = l - l_1$.
- We find k_1 and k_2 as described above.
- Then all suitable repetitions are the ones for which the lengths of the pieces l_1 and l_2 satisfy the conditions:

$$l_1 + l_2 = l = |u| - cntr$$

$$l_1 \le k_1,$$

$$l_2 \le k_2.$$

Therefore the only remaining part is how we can compute the values k_1 and k_2 quickly for every position $cntr$. Luckily we can compute them in $O(1)$ using the Z-function:

- To can find the value k_1 for each position by calculating the Z-function for the string $\overline{u}$ (i.e. the reversed string u). Then the value k_1 for a particular $cntr$ will be equal to the corresponding value of the array of the Z-function.
- To precompute all values k_2, we calculate the Z-function for the string $v + \# + u$ (i.e. the string u concatenated with the separator character $\#$ and the string v). Again we just need to look up the corresponding value in the Z-function to get the k_2 value.

So this is enough to find all left crossing repetitions.

RIGHT CROSSING REPETITIONS

For computing the right crossing repetitions we act similarly: we define the center $cntr$ as the character corresponding to the last character in the string u.

Then the length k_1 will be defined as the largest number of characters before the position $cntr$ (inclusive) that coincide with the last characters of the string u. And the length k_2 will be defined as the largest number of characters starting at $cntr + 1$ that coincide with the characters of the string v.

Thus we can find the values k_1 and k_2 by computing the Z-function for the strings $\overline{u} + \# + \overline{v}$ and v.

After that we can find the repetitions by looking at all positions $cntr$, and use the same criterion as we had for left crossing repetitions.

IMPLEMENTATION

The implementation of the Main-Lorentz algorithm finds all repetitions in form of peculiar tuples of size four: $(cntr,\ l,\ k_1,\ k_2)$ in $O(n \log n)$ time. If you only want to find the number of repetitions in a string, or only want to find the longest repetition in a string, this information is enough and the runtime will still be $O(n \log n)$.

Notice that if you want to expand these tuples to get the starting and end position of each repetition, then the runtime will be the runtime will be $O(n^2)$ (remember that there can be $O(n^2)$ repetitions). In this implementation we will do so, and store all found repetition in a vector of pairs of start and end indices.

```
vector<int> z_function(string const& s) {
    int n = s.size();
    vector<int> z(n);
    for (int i = 1, l = 0, r = 0; i < n; i++) {
        if (i <= r)
            z[i] = min(r-i+1, z[i-l]);
        while (i + z[i] < n && s[z[i]] == s[i+z[i]])
            z[i]++;
        if (i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
        }
    }
    return z;
}

int get_z(vector<int> const& z, int i) {
    if (0 <= i && i < (int)z.size())
        return z[i];
    else
        return 0;
}

vector<pair<int, int>> repetitions;

void convert_to_repetitions(int shift, bool left, int cntr, int l, int k1, int k2) {
    for (int l1 = max(1, l - k2); l1 <= min(l, k1); l1++) {
        if (left && l1 == l) break;
        int l2 = l - l1;
        int pos = shift + (left ? cntr - l1 : cntr - l - l1 + 1);
        repetitions.emplace_back(pos, pos + 2*l - 1);
    }
}

void find_repetitions(string s, int shift = 0) {
    int n = s.size();
    if (n == 1)
        return;

    int nu = n / 2;
    int nv = n - nu;
    string u = s.substr(0, nu);
    string v = s.substr(nu);
    string ru(u.rbegin(), u.rend());
    string rv(v.rbegin(), v.rend());

    find_repetitions(u, shift);
    find_repetitions(v, shift + nu);

    vector<int> z1 = z_function(ru);
    vector<int> z2 = z_function(v + '#' + u);
    vector<int> z3 = z_function(ru + '#' + rv);
```

```
    vector<int> z4 = z_function(v);

    for (int cntr = 0; cntr < n; cntr++) {
        int l, k1, k2;
        if (cntr < nu) {
            l = nu - cntr;
            k1 = get_z(z1, nu - cntr);
            k2 = get_z(z2, nv + 1 + cntr);
        } else {
            l = cntr - nu + 1;
            k1 = get_z(z3, nu + 1 + nv - 1 - (cntr - nu));
            k2 = get_z(z4, (cntr - nu) + 1);
        }
        if (k1 + k2 >= l)
            convert_to_repetitions(shift, cntr < nu, cntr, l, k1, k2);
    }
}
```

6.2 Techniques

6.2.1 The Inclusion-Exclusion Principle

The inclusion-exclusion principle is an important combinatorial way to compute the size of a set or the probability of complex events. It relates the sizes of individual sets with their union.

Statement

THE VERBAL FORMULA

The inclusion-exclusion principle can be expressed as follows:

To compute the size of a union of multiple sets, it is necessary to sum the sizes of these sets **separately**, and then subtract the sizes of all **pairwise** intersections of the sets, then add back the size of the intersections of **triples** of the sets, subtract the size of **quadruples** of the sets, and so on, up to the intersection of **all** sets.

THE FORMULATION IN TERMS OF SETS

The above definition can be expressed mathematically as follows:

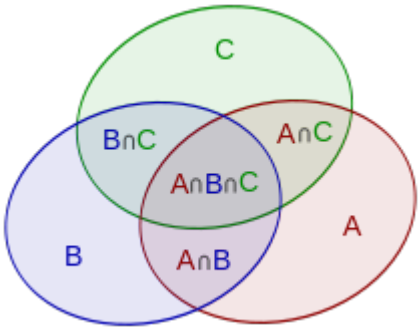
$$\bigcup_{i=1}^{n} A_i = \sum_{i=1}^{n} |A_i| - \sum_{1 \le i < j \le n} |A_i \cap A_j| + \sum_{1 \le i < j < k \le n} |A_i \cap A_j \cap A_k| - \cdots + (-1)^{n-1} |A_1 \cap \cdots \cap A_n|$$

And in a more compact way:

$$\bigcup_{i=1}^{n} A_i = \sum_{\emptyset \neq J \subseteq \{1,2,\ldots,n\}} (-1)^{|J|-1} \bigcap_{j \in J} A_j$$

THE FORMULATION USING VENN DIAGRAMS

Let the diagram show three sets A, B and C:



Then the area of their union $A \cup B \cup C$ is equal to the sum of the areas A, B and C less double-covered areas $A \cap B$, $A \cap C$, $B \cap C$, but with the addition of the area covered by three sets $A \cap B \cap C$:

$$S(A \cup B \cup C) = S(A) + S(B) + S(C) - S(A \cap B) - S(A \cap C) - S(B \cap C) + S(A \cap B \cap C)$$

It can also be generalized for an association of n sets.

THE FORMULATION IN TERMS OF PROBABILITY THEORY

If A_i $(i = 1, 2.. . n)$ are events and $\mathcal{P}(A_i)$ the probability of an event from A_i to occur, then the probability of their union (i.e. the probability that at least one of the events occur) is equal to:

$$\mathcal{P}\left(\bigcup_{i=1}^{n} A_i\right) = \sum_{i=1}^{n} \mathcal{P}(A_i) \; - \sum_{1 \leq i < j \leq n} \mathcal{P}(A_i \cap A_j) + \\ + \sum_{1 \leq i < j < k \leq n} \mathcal{P}(A_i \cap A_j \cap A_k) - \cdots + (-1)^{n-1} \mathcal{P}(A_1 \cap \cdots \cap A_n)$$

And in a more compact way:

$$\mathcal{P}\left(\bigcup_{i=1}^{n} A_i\right) = \sum_{\emptyset \neq J \subseteq \{1,2,\ldots,n\}} (-1)^{|J|-1} \, \mathcal{P}\left(\bigcap_{j \in J} A_j\right)$$

Proof

For the proof it is convenient to use the mathematical formulation in terms of set theory:

$$\bigcup_{i=1}^{n} A_i \; = \sum_{\emptyset \neq J \subseteq \{1,2,\ldots,n\}} (-1)^{|J|-1} \bigcap_{j \in J} A_j$$

We want to prove that any element contained in at least one of the sets A_i will occur in the formula only once (note that elements which are not present in any of the sets A_i will never be considered on the right part of the formula).

Consider an element x occurring in $k \geq 1$ sets A_i. We will show it is counted only once in the formula. Note that:

- in terms which $|J| = 1$, the item x will be counted $+ \; k$ times;
- in terms which $|J| = 2$, the item x will be counted $- \binom{k}{2}$ times - because it will be counted in those terms that include two of the k sets containing x;
- in terms which $|J| = 3$, the item x will be counted $+ \binom{k}{3}$ times;
- $\cdots$
- in terms which $|J| = k$, the item x will be counted $(-1)^{k-1} \cdot \binom{k}{k}$ times;
- in terms which $|J| > k$, the item x will be counted **zero** times;

This leads us to the following sum of binomial coefficients:

$$T = \binom{k}{1} - \binom{k}{2} + \binom{k}{3} - \cdots + (-1)^{i-1} \cdot \binom{k}{i} + \cdots + (-1)^{k-1} \cdot \binom{k}{k}$$

This expression is very similar to the binomial expansion of $(1 - x)^k$:

$$(1 - x)^k = \binom{k}{0} - \binom{k}{1} \cdot x + \binom{k}{2} \cdot x^2 - \binom{k}{3} \cdot x^3 + \cdots + (-1)^k \cdot \binom{k}{k} \cdot x^k$$

When $x = 1$, $(1 - x)^k$ looks a lot like T. However, the expression has an additional $\binom{k}{0} = 1$, and it is multiplied by -1. That leads us to $(1 - 1)^k = 1 - T$. Therefore $T = 1 - (1 - 1)^k = 1$, what was required to prove. The element is counted only once.

Generalization for calculating number of elements in exactly r sets

Inclusion-exclusion principle can be rewritten to calculate number of elements which are present in zero sets:

$$\bigcap_{i=1}^{n} \overline{A_i} = \sum_{m=0}^{n} (-1)^m \sum_{|X|=m} \bigcap_{i \in X} A_i$$

Consider its generalization to calculate number of elements which are present in exactly r sets:

$$\bigcup_{|B|=r} \left[\bigcap_{i \in B} A_i \cap \bigcap_{j \notin B} \overline{A_j} \right] = \sum_{m=r}^{n} (-1)^{m-r} \binom{m}{r} \sum_{|X|=m} \bigcap_{i \in X} A_i$$

To prove this formula, consider some particular B. Due to basic inclusion-exclusion principle we can say about it that:

$$\bigcap_{i \in B} A_i \cap \bigcap_{j \notin B} \overline{A_j} = \sum_{m=r}^{n} (-1)^{m-r} \sum_{\substack{|X|=m \\ B \subset X}} \bigcap_{i \in X} A_i$$

The sets on the left side do not intersect for different B, thus we can sum them up directly. Also one should note that any set X will always have coefficient $(-1)^{m-r}$ if it occurs and it will occur for exactly $\binom{m}{r}$ sets B.

Usage when solving problems

The inclusion-exclusion principle is hard to understand without studying its applications.

First, we will look at three simplest tasks "at paper", illustrating applications of the principle, and then consider more practical problems which are difficult to solve without inclusion-exclusion principle.

Tasks asking to "find the **number** of ways" are worth of note, as they sometimes lead to polynomial solutions, not necessarily exponential.

A SIMPLE TASK ON PERMUTATIONS

Task: count how many permutations of numbers from 0 to 9 exist such that the first element is greater than 1 and the last one is less than 8.

Let's count the number of "bad" permutations, that is, permutations in which the first element is ≤ 1 and/or the last is ≥ 8.

We will denote by X the set of permutations in which the first element is ≤ 1 and Y the set of permutations in which the last element is ≥ 8. Then the number of "bad" permutations, as on the inclusion-exclusion formula, will be:

$$|X \cup Y| = |X| + |Y| - |X \cap Y|$$

After a simple combinatorial calculation, we will get to:

$$2 \cdot 9! + 2 \cdot 9! - 2 \cdot 2 \cdot 8!$$

The only thing left is to subtract this number from the total of $10!$ to get the number of "good" permutations.

A SIMPLE TASK ON (0, 1, 2) SEQUENCES

Task: count how many sequences of length n exist consisting only of numbers $0, 1, 2$ such that each number occurs **at least once**.

Again let us turn to the inverse problem, i.e. we calculate the number of sequences which do **not** contain **at least one** of the numbers.

Let's denote by $A_i (i = 0, 1, 2)$ the set of sequences in which the digit i does **not** occur. The formula of inclusion-exclusion on the number of "bad" sequences will be:

$$|A_0 \cup A_1 \cup A_2| = |A_0| + |A_1| + |A_2| - |A_0 \cap A_1| - |A_0 \cap A_2| - |A_1 \cap A_2| + |A_0 \cap A_1 \cap A_2|$$

- The size of each A_i is 2^n, as each sequence can only contain two of the digits.
- The size of each pairwise intersection $A_i \cap A_j$ is equal to 1, as there will be only one digit to build the sequence.
- The size of the intersection of all three sets is equal to 0, as there will be no digits to build the sequence.

As we solved the inverse problem, we subtract it from the total of 3^n sequences:

$$3^n - (3 \cdot 2^n - 3 \cdot 1 + 0)$$

NUMBER OF UPPER-BOUND INTEGER SUMS

Consider the following equation:

$$x_1 + x_2 + x_3 + x_4 + x_5 + x_6 = 20$$

where $0 \leq x_i \leq 8$ $(i = 1, 2, \dots 6)$.

Task: count the number of solutions to the equation.

Forget the restriction on x_i for a moment and just count the number of nonnegative solutions to this equation. This is easily done using Stars and Bars: we want to break a sequence of 20 units into 6 groups, which is the same as arranging 5 *bars* and 20 *stars*:

$$N_0 = \binom{25}{5}$$

We will now calculate the number of "bad" solutions with the inclusion-exclusion principle. The "bad" solutions will be those in which one or more x_i are greater than or equal to 9.

Denote by A_k $(k = 1, 2 \dots 6)$ the set of solutions where $x_k \geq 9$, and all other $x_i \geq 0$ $(i \neq k)$ (they may be ≥ 9 or not). To calculate the size of A_k, note that we have essentially the same combinatorial problem that was solved in the two paragraphs above, but now 9 of the units are excluded from the slots and definitely belong to the first group. Thus:

$$|A_k| = \binom{16}{5}$$

Similarly, the size of the intersection between two sets A_k and A_p (for $k \neq p$) is equal to:

$$|A_k \cap A_p| = \binom{7}{5}$$

The size of each intersection of three sets is zero, since 20 units will not be enough for three or more variables greater than or equal to 9.

Combining all this into the formula of inclusions-exceptions and given that we solved the inverse problem, we finally get the answer:

$$\binom{25}{5} - \left(\binom{6}{1} \cdot \binom{16}{5} - \binom{6}{2} \cdot \binom{7}{5}\right)$$

This easily generalizes to d numbers that sum up to s with the restriction $0 \leq x_i \leq b$:

$$\sum_{i=0}^{d} (-1)^i \binom{d}{i} \binom{s + d - 1 - (b+1)i}{d-1}$$

As above, we treat binomial coefficients with negative upper index as zero.

Note this problem could also be solved with dynamic programming or generating functions. The inclusion-exclusion answer is computed in $O(d)$ time (assuming math operations like binomial coefficient are constant time), while a simple DP approach would take $O(ds)$ time.

THE NUMBER OF RELATIVE PRIMES IN A GIVEN INTERVAL

Task: given two numbers n and r, count the number of integers in the interval $[1; r]$ that are relatively prime to n (their greatest common divisor is 1).

Let's solve the inverse problem - compute the number of not mutually primes with n.

We will denote the prime factors of n as $p_i (i = 1 \cdots k)$.

How many numbers in the interval $[1; r]$ are divisible by p_i? The answer to this question is:

$$\left\lfloor \frac{r}{p_i} \right\rfloor$$

However, if we simply sum these numbers, some numbers will be summarized several times (those that share multiple p_i as their factors). Therefore, it is necessary to use the inclusion-exclusion principle.

We will iterate over all 2^k subsets of p_is, calculate their product and add or subtract the number of multiples of their product.

Here is a C++ implementation:

```
int solve (int n, int r) {
    vector<int> p;
    for (int i=2; i*i<=n; ++i)
        if (n % i == 0) {
            p.push_back (i);
            while (n % i == 0)
                n /= i;
        }
    if (n > 1)
        p.push_back (n);

    int sum = 0;
    for (int msk=1; msk<(1<<p.size()); ++msk) {
        int mult = 1,
            bits = 0;
        for (int i=0; i<(int)p.size(); ++i)
            if (msk & (1<<i)) {
                ++bits;
                mult *= p[i];
            }

        int cur = r / mult;
        if (bits % 2 == 1)
            sum += cur;
        else
            sum -= cur;
    }

    return r - sum;
}
```

Asymptotics of the solution is $O(\sqrt{n})$.

THE NUMBER OF INTEGERS IN A GIVEN INTERVAL WHICH ARE MULTIPLE OF AT LEAST ONE OF THE GIVEN NUMBERS

Given n numbers a_i and number r. You want to count the number of integers in the interval $[1; r]$ that are multiple of at least one of the a_i.

The solution algorithm is almost identical to the one for previous task — construct the formula of inclusion-exclusion on the numbers a_i, i.e. each term in this formula is the number of numbers divisible by a given subset of numbers a_i (in other words, divisible by their least common multiple).

So we will now iterate over all 2^n subsets of integers a_i with $O(n \log r)$ operations to find their least common multiple, adding or subtracting the number of multiples of it in the interval. Asymptotics is $O(2^n \cdot n \cdot \log r)$.

THE NUMBER OF STRINGS THAT SATISFY A GIVEN PATTERN

Consider n patterns of strings of the same length, consisting only of letters ($a \ldots z$) or question marks. You're also given a number k. A string matches a pattern if it has the same length as the pattern, and at each position, either the corresponding characters are equal or the character in the pattern is a question mark. The task is to count the number of strings that match exactly k of the patterns (first problem) and at least k of the patterns (second problem).

Notice first that we can easily count the number of strings that satisfy at once all of the specified patterns. To do this, simply "cross" patterns: iterate though the positions ("slots") and look at a position over all patterns. If all patterns have a question mark in this position, the character can be any letter from a to z. Otherwise, the character of this position is uniquely defined by the patterns that do not contain a question mark.

Learn now to solve the first version of the problem: when the string must satisfy exactly k of the patterns.

To solve it, iterate and fix a specific subset X from the set of patterns consisting of k patterns. Then we have to count the number of strings that satisfy this set of patterns, and only matches it, that is, they don't match any other pattern. We will use the inclusion-exclusion principle in a slightly different manner: we sum on all supersets Y (subsets from the original set of strings that contain X), and either add to the current answer or subtract it from the number of strings:

$$ans(X) = \sum_{Y \supseteq X} (-1)^{|Y|-k} \cdot f(Y)$$

Where $f(Y)$ is the number of strings that match Y (at least Y).

(If you have a hard time figuring out this, you can try drawing Venn Diagrams.)

If we sum up on all $ans(X)$, we will get the final answer:

$$ans = \sum_{X\,:\,|X|=k} ans(X)$$

However, asymptotics of this solution is $O(3^k \cdot k)$. To improve it, notice that different $ans(X)$ computations very often share Y sets.

We will reverse the formula of inclusion-exclusion and sum in terms of Y sets. Now it becomes clear that the same set Y would be taken into account in the computation of $ans(X)$ of $\binom{|Y|}{k}$ sets with the same sign $(-1)^{|Y|-k}$.

$$ans = \sum_{Y\,:\,|Y|\geq k} (-1)^{|Y|-k} \cdot \binom{|Y|}{k} \cdot f(Y)$$

Now our solution has asymptotics $O(2^k \cdot k)$.

We will now solve the second version of the problem: find the number of strings that match **at least** k of the patterns.

Of course, we can just use the solution to the first version of the problem and add the answers for sets with size greater than k. However, you may notice that in this problem, a set |Y| is considered in the formula for all sets with size $\geq k$ which are contained in Y. That said, we can write the part of the expression that is being multiplied by $f(Y)$ as:

$$(-1)^{|Y|-k} \cdot \binom{|Y|}{k} + (-1)^{|Y|-k-1} \cdot \binom{|Y|}{k+1} + (-1)^{|Y|-k-2} \cdot \binom{|Y|}{k+2} + \cdots + (-1)^{|Y|-|Y|} \cdot \binom{|Y|}{|Y|}$$

Looking at Graham's (Graham, Knuth, Patashnik. "Concrete mathematics" [1998]), we see a well-known formula for binomial coefficients:

$$\sum_{k=0}^{m} (-1)^k \cdot \binom{n}{k} = (-1)^m \cdot \binom{n-1}{m}$$

Applying it here, we find that the entire sum of binomial coefficients is minimized:

$$(-1)^{|Y|-k} \cdot \binom{|Y|-1}{|Y|-k}$$

Thus, for this task, we also obtained a solution with the asymptotics $O(2^k \cdot k)$:

$$ans = \sum_{Y\,:\,|Y| \geq k} (-1)^{|Y|-k} \cdot \binom{|Y|-1}{|Y|-k} \cdot f(Y)$$

THE NUMBER OF WAYS OF GOING FROM A CELL TO ANOTHER

There is a field $n \times m$, and k of its cells are impassable walls. A robot is initially at the cell $(1, 1)$ (bottom left). The robot can only move right or up, and eventually it needs to get into the cell (n, m), avoiding all obstacles. You need to count the number of ways he can do it.

Assume that the sizes n and m are very large (say, 10^9), and the number k is small (around 100).

For now, sort the obstacles by their coordinate x, and in case of equality — coordinate y.

Also just learn how to solve a problem without obstacles: i.e. learn how to count the number of ways to get from one cell to another. In one axis, we need to go through x cells, and on the other, y cells. From simple combinatorics, we get a formula using binomial coefficients:

$$\binom{x+y}{x}$$

Now to count the number of ways to get from one cell to another, avoiding all obstacles, you can use inclusion-exclusion to solve the inverse problem: count the number of ways to walk through the board stepping at a subset of obstacles (and subtract it from the total number of ways).

When iterating over a subset of the obstacles that we'll step, to count the number of ways to do this simply multiply the number of all paths from starting cell to the first of the selected obstacles, a first obstacle to the second, and so on, and then add or subtract this number from the answer, in accordance with the standard formula of inclusion-exclusion.

However, this will again be non-polynomial in complexity $O(2^k \cdot k)$.

Here goes a polynomial solution:

We will use dynamic programming. For convenience, push (1,1) to the beginning and (n,m) at the end of the obstacles array. Let's compute the numbers $d[i]$ — the number of ways to get from the starting point ($0-th$) to $i-th$, without stepping on any other obstacle (except for i, of course). We will compute this number for all the obstacle cells, and also for the ending one.

Let's forget for a second the obstacles and just count the number of paths from cell 0 to i. We need to consider some "bad" paths, the ones that pass through the obstacles, and subtract them from the total number of ways of going from 0 to i.

When considering an obstacle t between 0 and i ($0 < t < i$), on which we can step, we see that the number of paths from 0 to i that pass through t which have t as the **first obstacle between start and** i. We can compute that as: $d[t]$ multiplied by the number of arbitrary paths from t to i. We can count the number of "bad" ways summing this for all t between 0 and i.

We can compute $d[i]$ in $O(k)$ for $O(k)$ obstacles, so this solution has complexity $O(k^2)$.

THE NUMBER OF COPRIME QUADRUPLES

You're given n numbers: $a_1, a_2, \ldots, a_n$. You are required to count the number of ways to choose four numbers so that their combined greatest common divisor is equal to one.

We will solve the inverse problem — compute the number of "bad" quadruples, i.e. quadruples in which all numbers are divisible by a number $d > 1$.

We will use the inclusion-exclusion principle while summing over all possible groups of four numbers divisible by a divisor d.

$$ans = \sum_{d \geq 2} (-1)^{deg(d)-1} \cdot f(d)$$

where $deg(d)$ is the number of primes in the factorization of the number d and $f(d)$ the number of quadruples divisible by d.

To calculate the function $f(d)$, you just have to count the number of multiples of d (as mentioned on a previous task) and use binomial coefficients to count the number of ways to choose four of them.

Thus, using the formula of inclusions-exclusions we sum the number of groups of four divisible by a prime number, then subtract the number of quadruples which are divisible by the product of two primes, add quadruples divisible by three primes, etc.

THE NUMBER OF HARMONIC TRIPLETS

You are given a number $n \leq 10^6$. You are required to count the number of triples $2 \leq a < b < c \leq n$ that satisfy one of the following conditions:

- or $\gcd(a,b) = \gcd(a,c) = \gcd(b,c) = 1$,
- or $\gcd(a,b) > 1, \gcd(a,c) > 1, \gcd(b,c) > 1$.

First, go straight to the inverse problem — i.e. count the number of non-harmonic triples.

Second, note that any non-harmonic triplet is made of a pair of coprimes and a third number that is not coprime with at least one from the pair.

Thus, the number of non-harmonic triples that contain i is equal the number of integers from 2 to n that are coprimes with i multiplied by the number of integers that are not coprime with i.

Either $gcd(a,b) = 1 \wedge gcd(a,c) > 1 \wedge gcd(b,c) > 1$

or $gcd(a,b) = 1 \wedge gcd(a,c) = 1 \wedge gcd(b,c) > 1$

In both of these cases, it will be counted twice. The first case will be counted when $i = a$ and when $i = b$. The second case will be counted when $i = b$ and when $i = c$. Therefore, to compute the number of non-harmonic triples, we sum this calculation through all i from 2 to n and divide it by 2.

Now all we have left to solve is to learn to count the number of coprimes to i in the interval $[2; n]$. Although this problem has already been mentioned, the above solution is not suitable here — it would require the factorization of each of the integers from 2 to n, and then iterating through all subsets of these primes.

A faster solution is possible with such modification of the sieve of Eratosthenes:

1. First, we find all numbers in the interval $[2; n]$ such that its simple factorization does not include a prime factor twice. We will also need to know, for these numbers, how many factors it includes.
   - To do this we will maintain an array $deg[i]$ to store the number of primes in the factorization of i, and an array $good[i]$, to mark either if i contains each factor at most once ($good[i] = 1$) or not ($good[i] = 0$). When iterating from 2 to n, if we reach a number that has deg equal to 0, then it is a prime and its deg is 1.
   - During the sieve of Eratosthenes, we will iterate i from 2 to n. When processing a prime number we go through all of its multiples and increase their $deg[]$. If one of these multiples is multiple of the square of i, then we can put $good$ as false.
2. Second, we need to calculate the answer for all i from 2 to n, i.e., the array $cnt[]$ — the number of integers not coprime with i.
   - To do this, remember how the formula of inclusion-exclusion works — actually here we implement the same concept, but with inverted logic: we iterate over a component (a product of primes from the factorization) and add or subtract its term on the formula of inclusion-exclusion of each of its multiples.
   - So, let's say we are processing a number i such that $good[i] = true$, i.e., it is involved in the formula of inclusion-exclusion. Iterate through all numbers that are multiples of i, and either add or subtract $\lfloor N/i \rfloor$ from their $cnt[]$ (the signal depends on $deg[i]$: if $deg[i]$ is odd, then we must add, otherwise subtract).

Here's a C++ implementation:

```
int n;
bool good[MAXN];
int deg[MAXN], cnt[MAXN];

long long solve() {
    memset (good, 1, sizeof good);
    memset (deg, 0, sizeof deg);
    memset (cnt, 0, sizeof cnt);

    long long ans_bad = 0;
    for (int i=2; i<=n; ++i) {
```

```
        if (good[i]) {
            if (deg[i] == 0)  deg[i] = 1;
            for (int j=1; i*j<=n; ++j) {
                if (j > 1 && deg[i] == 1)
                    if (j % i == 0)
                        good[i*j] = false;
                    else
                        ++deg[i*j];
                cnt[i*j] += (n / i) * (deg[i]%2==1 ? +1 : -1);
            }
        }
        ans_bad += (cnt[i] - 1) * 1ll * (n-1 - cnt[i]);
    }

    return (n-1) * 1ll * (n-2) * (n-3) / 6 - ans_bad / 2;
}
```

The asymptotics of our solution is $O(n \log n)$, as for almost every number up to n we make n/i iterations on the nested loop.

THE NUMBER OF PERMUTATIONS WITHOUT FIXED POINTS (DERANGEMENTS)

Prove that the number of permutations of length n without fixed points (i.e. no number i is in position i - also called a derangement) is equal to the following number:

$$n! - \binom{n}{1} \cdot (n-1)! + \binom{n}{2} \cdot (n-2)! - \binom{n}{3} \cdot (n-3)! + \cdots \pm \binom{n}{n} \cdot (n-n)!$$

and approximately equal to:

$$\frac{n!}{e}$$

(if you round this expression to the nearest whole number — you get exactly the number of permutations without fixed points)

Denote by A_k the set of permutations of length n with a fixed point at position k ($1 \le k \le n$) (i.e. element k is at position k).

We now use the formula of inclusion-exclusion to count the number of permutations with at least one fixed point. For this we need to learn to count sizes of an intersection of sets A_i, as follows:

$$\begin{aligned} |A_p| &= (n-1)!\,, \\ |A_p \cap A_q| &= (n-2)!\,, \\ |A_p \cap A_q \cap A_r| &= (n-3)!\,, \\ &\cdots, \end{aligned}$$

because if we know that the number of fixed points is equal x, then we know the position of x elements of the permutation, and all other $(n-x)$ elements can be placed anywhere.

Substituting this into the formula of inclusion-exclusion, and given that the number of ways to choose a subset of size x from the set of n elements is equal to $\binom{n}{x}$, we obtain a formula for the number of permutations with at least one fixed point:

$$\binom{n}{1} \cdot (n-1)! - \binom{n}{2} \cdot (n-2)! + \binom{n}{3} \cdot (n-3)! - \cdots \pm \binom{n}{n} \cdot (n-n)!$$

Then the number of permutations without fixed points is equal to:

$$n! - \binom{n}{1} \cdot (n-1)! + \binom{n}{2} \cdot (n-2)! - \binom{n}{3} \cdot (n-3)! + \cdots \pm \binom{n}{n} \cdot (n-n)!$$

Simplifying this expression, we obtain **exact and approximate expressions for the number of permutations without fixed points**:

$$n!\left(1-\frac{1}{1!}+\frac{1}{2!}-\frac{1}{3!}+\cdots\pm\frac{1}{n!}\right)\approx\frac{n!}{e}$$

(because the sum in brackets are the first $n+1$ terms of the expansion in Taylor series e^{-1})

It is worth noting that a similar problem can be solved this way: when you need the fixed points were not among the m first elements of permutations (and not among all, as we just solved). The formula obtained is as the given above accurate formula, but it will go up to the sum of k, instead of n.

6.2.3 Stars and bars

Stars and bars is a mathematical technique for solving certain combinatorial problems. It occurs whenever you want to count the number of ways to group identical objects.

Theorem

The number of ways to put n identical objects into k labeled boxes is

$$\binom{n+k-1}{n}.$$

The proof involves turning the objects into stars and separating the boxes using bars (therefore the name). E.g. we can represent with ★|★★| |★★ the following situation: in the first box is one object, in the second box are two objects, the third one is empty and in the last box are two objects. This is one way of dividing 5 objects into 4 boxes.

It should be pretty obvious, that every partition can be represented using n stars and $k-1$ bars and every stars and bars permutation using n stars and $k-1$ bars represents one partition. Therefore the number of ways to divide n identical objects into k labeled boxes is the same number as there are permutations of n stars and $k-1$ bars. The Binomial Coefficient gives us the desired formula.

Number of non-negative integer sums

This problem is a direct application of the theorem.

You want to count the number of solution of the equation

$$x_1 + x_2 + \cdots + x_k = n$$

with $x_i \geq 0$.

Again we can represent a solution using stars and bars. E.g. the solution $1+3+0=4$ for $n=4$, $k=3$ can be represented using ★|★★★|.

It is easy to see, that this is exactly the stars and bars theorem. Therefore the solution is $\binom{n+k-1}{n}$.

Number of positive integer sums

A second theorem provides a nice interpretation for positive integers. Consider solutions to

$$x_1 + x_2 + \cdots + x_k = n$$

with $x_i \geq 1$.

We can consider n stars, but this time we can put at most *one bar* between stars, since two bars between stars would represent $x_i = 0$, i.e. an empty box. There are $n-1$ gaps between stars to place $k-1$ bars, so the solution is $\binom{n-1}{k-1}$.

Number of lower-bound integer sums

This can easily be extended to integer sums with different lower bounds. I.e. we want to count the number of solutions for the equation

$$x_1 + x_2 + \cdots + x_k = n$$

with $x_i \geq a_i$.

After substituting $x_i' := x_i - a_i$ we receive the modified equation

$$(x'_1 + a_i) + (x'_2 + a_i) + \cdots + (x'_k + a_k) = n$$

$$\Leftrightarrow \quad x'_1 + x'_2 + \cdots + x'_k = n - a_1 - a_2 - \cdots - a_k$$

with $x'_i \geq 0$. So we have reduced the problem to the simpler case with $x'_i \geq 0$ and again can apply the stars and bars theorem.

Number of upper-bound integer sums

With some help of the Inclusion-Exclusion Principle, you can also restrict the integers with upper bounds. See the Number of upper-bound integer sums section in the corresponding article.

7.1.2 Ternary Search

We are given a function $f(x)$ which is unimodal on an interval $[l, r]$. By unimodal function, we mean one of two behaviors of the function:

1. The function strictly increases first, reaches a maximum (at a single point or over an interval), and then strictly decreases.
2. The function strictly decreases first, reaches a minimum, and then strictly increases.

In this article, we will assume the first scenario. The second scenario is completely symmetrical to the first.

The task is to find the maximum of function $f(x)$ on the interval $[l, r]$.

Algorithm

Consider any 2 points m_1, and m_2 in this interval: $l < m_1 < m_2 < r$. We evaluate the function at m_1 and m_2, i.e. find the values of $f(m_1)$ and $f(m_2)$. Now, we get one of three options:

- $f(m_1) < f(m_2)$

  The desired maximum can not be located on the left side of m_1, i.e. on the interval $[l, m_1]$, since either both points m_1 and m_2 or just m_1 belong to the area where the function increases. In either case, this means that we have to search for the maximum in the segment $[m_1, r]$.
- $f(m_1) > f(m_2)$

  This situation is symmetrical to the previous one: the maximum can not be located on the right side of m_2, i.e. on the interval $[m_2, r]$, and the search space is reduced to the segment $[l, m_2]$.
- $f(m_1) = f(m_2)$

  We can see that either both of these points belong to the area where the value of the function is maximized, or m_1 is in the area of increasing values and m_2 is in the area of descending values (here we used the strictness of function increasing/decreasing). Thus, the search space is reduced to $[m_1, m_2]$. To simplify the code, this case can be combined with any of the previous cases.

Thus, based on the comparison of the values in the two inner points, we can replace the current interval $[l, r]$ with a new, shorter interval $[l', r']$. Repeatedly applying the described procedure to the interval, we can get an arbitrarily short interval. Eventually, its length will be less than a certain pre-defined constant (accuracy), and the process can be stopped. This is a numerical method, so we can assume that after that the function reaches its maximum at all points of the last interval $[l, r]$. Without loss of generality, we can take $f(l)$ as the return value.

We didn't impose any restrictions on the choice of points m_1 and m_2. This choice will define the convergence rate and the accuracy of the implementation. The most common way is to choose the points so that they divide the interval $[l, r]$ into three equal parts. Thus, we have

$$m_1 = l + \frac{(r - l)}{3}$$

$$m_2 = r - \frac{(r - l)}{3}$$

If m_1 and m_2 are chosen to be closer to each other, the convergence rate will increase slightly.

RUN TIME ANALYSIS

$$T(n) = T(2n/3) + O(1) = \Theta(\log n)$$

It can be visualized as follows: every time after evaluating the function at points m_1 and m_2, we are essentially ignoring about one third of the interval, either the left or right one. Thus the size of the search space is $2n/3$ of the original one.

Applying Master's Theorem, we get the desired complexity estimate.

THE CASE OF THE INTEGER ARGUMENTS

If $f(x)$ takes integer parameter, the interval $[l, r]$ becomes discrete. Since we did not impose any restrictions on the choice of points m_1 and m_2, the correctness of the algorithm is not affected. m_1 and m_2 can still be chosen to divide $[l, r]$ into 3 approximately equal parts.

The difference occurs in the stopping criterion of the algorithm. Ternary search will have to stop when $(r - l) < 3$, because in that case we can no longer select m_1 and m_2 to be different from each other as well as from l and r, and this can cause an infinite loop. Once $(r - l) < 3$, the remaining pool of candidate points $(l, l + 1, \ldots, r)$ needs to be checked to find the point which produces the maximum value $f(x)$.

GOLDEN SECTION SEARCH

In some cases computing $f(x)$ may be quite slow, but reducing the number of iterations is infeasible due to precision issues. Fortunately, it is possible to compute $f(x)$ only once at each iteration (except the first one).

To see how to do this, let's revisit the selection method for m_1 and m_2. Suppose that we select m_1 and m_2 on $[l, r]$ in such a way that $\frac{r-l}{r-m_1} = \frac{r-l}{m_2-l} = \varphi$ where φ is some constant. In order to reduce the amount of computations, we want to select such φ that on the next iteration one of the new evaluation points m_1', m_2' will coincide with either m_1 or m_2, so that we can reuse the already computed function value.

Now suppose that after the current iteration we set $l = m_1$. Then the point m_1' will satisfy $\frac{r-m_1}{r-m_1'} = \varphi$. We want this point to coincide with m_2, meaning that $\frac{r-m_1}{r-m_2} = \varphi$.

Multiplying both sides of $\frac{r-m_1}{r-m_2} = \varphi$ by $\frac{r-m_2}{r-l}$ we obtain $\frac{r-m_1}{r-l} = \varphi\frac{r-m_2}{r-l}$. Note that $\frac{r-m_1}{r-l} = \frac{1}{\varphi}$ and $\frac{r-m_2}{r-l} = \frac{r-l+l-m_2}{r-l} = 1 - \frac{1}{\varphi}$. Substituting that and multiplying by φ, we obtain the following equation:

$$\varphi^2 - \varphi - 1 = 0$$

This is a well-known golden section equation. Solving it yields $\frac{1\pm\sqrt{5}}{2}$. Since φ must be positive, we obtain $\varphi = \frac{1+\sqrt{5}}{2}$. By applying the same logic to the case when we set $r = m_2$ and want m_2' to coincide with m_1, we obtain the same value of φ as well. So, if we choose $m_1 = l + \frac{r-l}{1+\varphi}$ and $m_2 = r - \frac{r-l}{1+\varphi}$, on each iteration we can re-use one of the values $f(x)$ computed on the previous iteration.

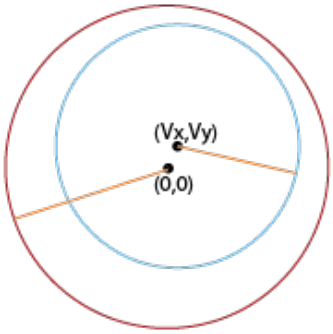
Implementation

```
double ternary_search(double l, double r) {
    double eps = 1e-9;              //set the error limit here
    while (r - l > eps) {
        double m1 = l + (r - l) / 3;
        double m2 = r - (r - l) / 3;
        double f1 = f(m1);      //evaluates the function at m1
        double f2 = f(m2);      //evaluates the function at m2
        if (f1 < f2)
            l = m1;
        else
            r = m2;
    }
    return f(l);                    //return the maximum of f(x) in [l, r]
}
```

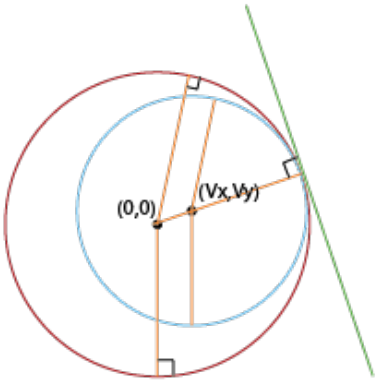
Here `eps` is in fact the absolute error (not taking into account errors due to the inaccurate calculation of the function).

Instead of the criterion `r - l > eps`, we can select a constant number of iterations as a stopping criterion. The number of iterations should be chosen to ensure the required accuracy. Typically, in most programming challenges the error limit is 10^{-6} and thus 200 - 300 iterations are sufficient. Also, the number of iterations doesn't depend on the values of l and r, so the number of iterations corresponds to the required relative error.

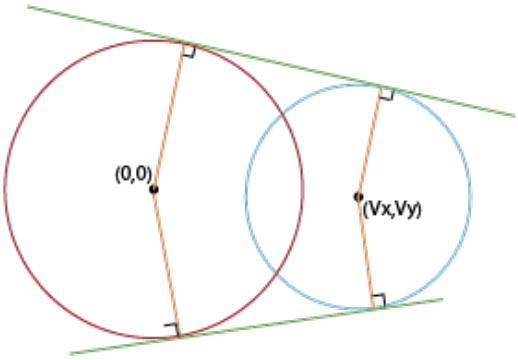
Case : No tangent



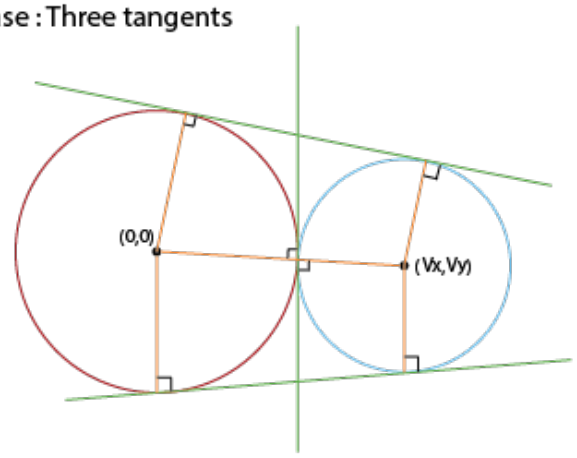
Case: One tangent



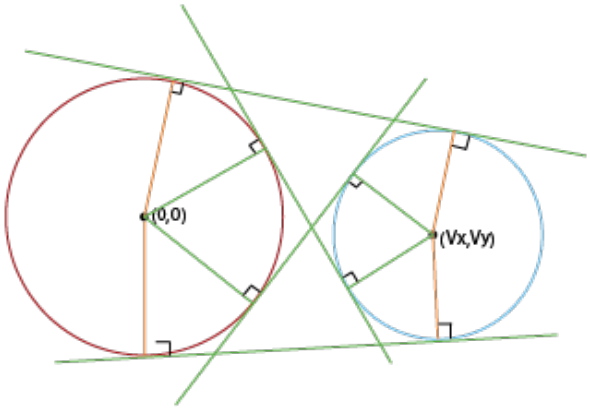
Case : Two tangents



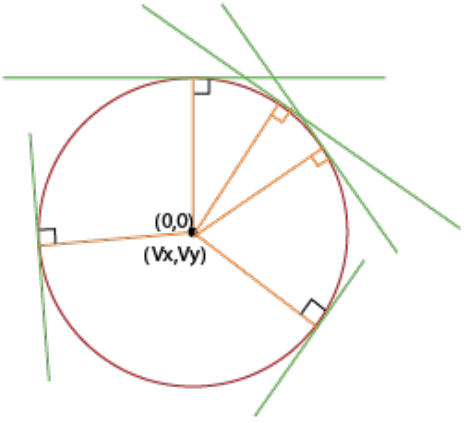
Case : Three tangents



Case : Four tangents



Case : Infinite



Here, we won't be considering **degenerate** cases, i.e *when the circles coincide (in this case they have infinitely many common tangents), or one circle lies inside the other (in this case they have no common tangents, or if the circles are tangent, there is one common tangent).*

In most cases, two circles have **four** common tangents.

If the circles **are tangent** , then they will have three common tangents, but this can be understood as a degenerate case: as if the two tangents coincided.

8.4 Sweep-line

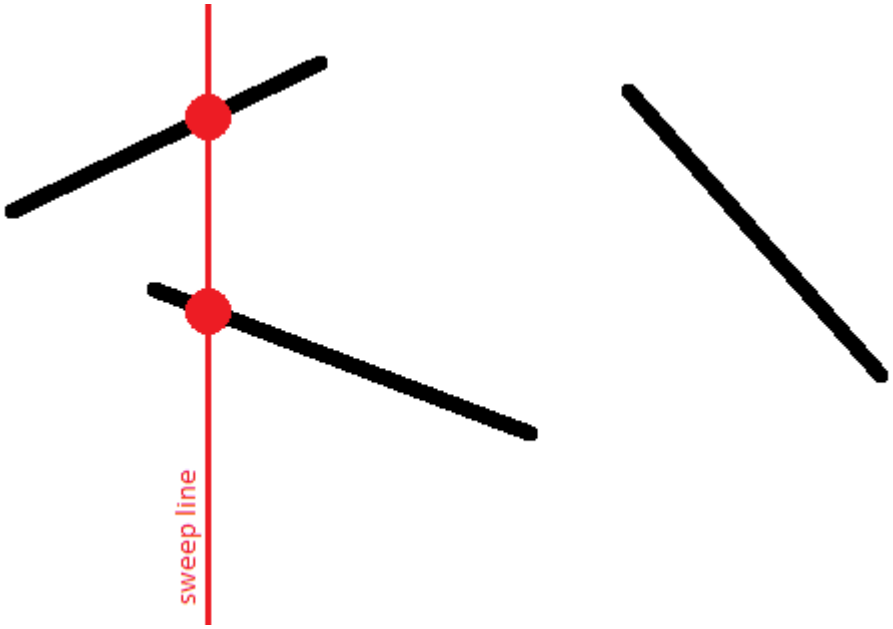
8.4.1 Search for a pair of intersecting segments

Given n line segments on the plane. It is required to check whether at least two of them intersect with each other. If the answer is yes, then print this pair of intersecting segments; it is enough to choose any of them among several answers.

The naive solution algorithm is to iterate over all pairs of segments in $O(n^2)$ and check for each pair whether they intersect or not. This article describes an algorithm with the runtime time $O(n \log n)$, which is based on the **sweep line algorithm**.

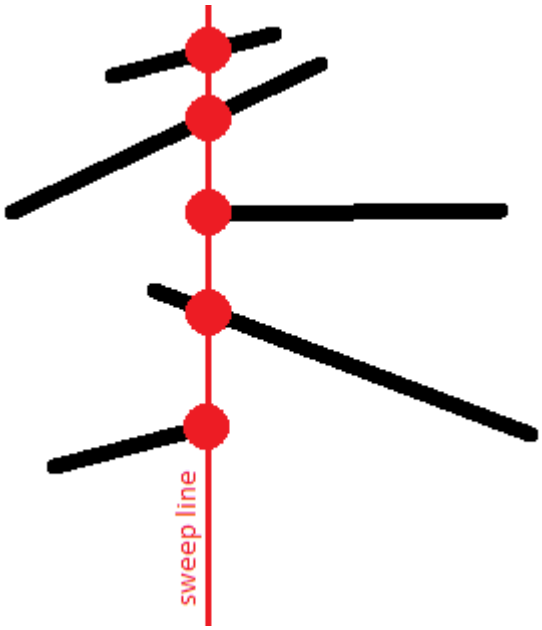
Algorithm

Let's draw a vertical line $x = -\infty$ mentally and start moving this line to the right. In the course of its movement, this line will meet with segments, and at each time a segment intersects with our line it intersects in exactly one point (we will assume that there are no vertical segments).

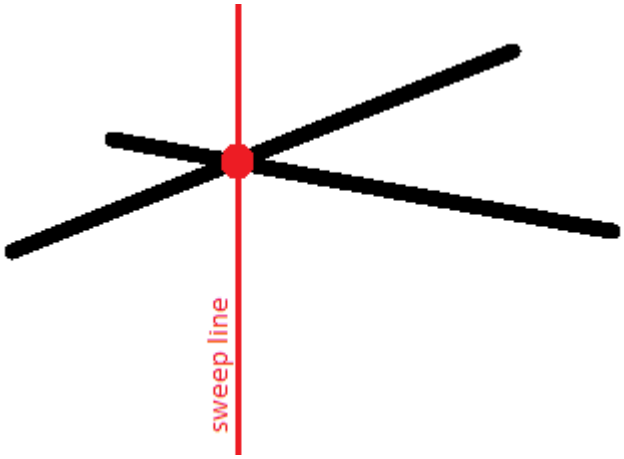


Thus, for each segment, at some point in time, its point will appear on the sweep line, then with the movement of the line, this point will move, and finally, at some point, the segment will disappear from the line.

We are interested in the **relative order of the segments** along the vertical. Namely, we will store a list of segments crossing the sweep line at a given time, where the segments will be sorted by their y-coordinate on the sweep line.



This order is interesting because intersecting segments will have the same y-coordinate at least at one time:



We formulate key statements:

- To find an intersecting pair, it is sufficient to consider **only adjacent segments** at each fixed position of the sweep line.
- It is enough to consider the sweep line not in all possible real positions $(-\infty \ldots +\infty)$, but **only in those positions when new segments appear or old ones disappear**. In other words, it is enough to limit yourself only to the positions equal to the abscissas of the end points of the segments.
- When a new line segment appears, it is enough to **insert** it to the desired location in the list obtained for the previous sweep line. We should only check for the intersection of the **added segment with its immediate neighbors in the list above and below**.
- If the segment disappears, it is enough to **remove** it from the current list. After that, it is necessary **check for the intersection of the upper and lower neighbors in the list**.
- Other changes in the sequence of segments in the list, except for those described, do not exist. No other intersection checks are required.

To understand the truth of these statements, the following remarks are sufficient:

- Two disjoint segments never change their **relative order**.
  In fact, if one segment was first higher than the other, and then became lower, then between these two moments there was an intersection of these two segments.
- Two non-intersecting segments also cannot have the same y-coordinates.
- From this it follows that at the moment of the segment appearance we can find the position for this segment in the queue, and we will not have to rearrange this segment in the queue any more: **its order relative to other segments in the queue will not change**.
- Two intersecting segments at the moment of their intersection point will be neighbors of each other in the queue.
- Therefore, for finding pairs of intersecting line segments is sufficient to check the intersection of all and only those pairs of segments that sometime during the movement of the sweep line at least once were neighbors to each other.
  It is easy to notice that it is enough only to check the added segment with its upper and lower neighbors, as well as when removing the segment — its upper and lower neighbors (which after removal will become neighbors of each other).
- It should be noted that at a fixed position of the sweep line, we must **first add all the segments** that start at this x-coordinate, and only **then remove all the segments** that end here.
  Thus, we do not miss the intersection of segments on the vertex: i.e. such cases when two segments have a common vertex.
- Note that **vertical segments** do not actually affect the correctness of the algorithm.
  These segments are distinguished by the fact that they appear and disappear at the same time. However, due to the previous comment, we know that all segments will be added to the queue first, and only then they will be deleted. Therefore, if the vertical segment intersects with some other segment opened at that moment (including the vertical one), it will be detected.
  In what place of the queue to place vertical segments? After all, a vertical segment does not have one specific y-coordinate, it extends for an entire segment along the y-coordinate. However, it is easy to understand that any coordinate from this segment can be taken as a y-coordinate.

Thus, the entire algorithm will perform no more than $2n$ tests on the intersection of a pair of segments, and will perform $O(n)$ operations with a queue of segments ($O(1)$ operations at the time of appearance and disappearance of each segment).

The final **asymptotic behavior of the algorithm** is thus $O(n \log n)$.

Implementation

We present the full implementation of the described algorithm:

```
const double EPS = 1E-9;

struct pt {
    double x, y;
};

struct seg {
    pt p, q;
    int id;

    double get_y(double x) const {
        if (abs(p.x - q.x) < EPS)
            return p.y;
        return p.y + (q.y - p.y) * (x - p.x) / (q.x - p.x);
    }
};

bool intersect1d(double l1, double r1, double l2, double r2) {
    if (l1 > r1)
        swap(l1, r1);
    if (l2 > r2)
        swap(l2, r2);
    return max(l1, l2) <= min(r1, r2) + EPS;
}

int vec(const pt& a, const pt& b, const pt& c) {
    double s = (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
    return abs(s) < EPS ? 0 : s > 0 ? +1 : -1;
}

bool intersect(const seg& a, const seg& b)
{
    return intersect1d(a.p.x, a.q.x, b.p.x, b.q.x) &&
```

```
            intersect1d(a.p.y, a.q.y, b.p.y, b.q.y) &&
            vec(a.p, a.q, b.p) * vec(a.p, a.q, b.q) <= 0 &&
            vec(b.p, b.q, a.p) * vec(b.p, b.q, a.q) <= 0;
}

bool operator<(const seg& a, const seg& b)
{
    double x = max(min(a.p.x, a.q.x), min(b.p.x, b.q.x));
    return a.get_y(x) < b.get_y(x) - EPS;
}

struct event {
    double x;
    int tp, id;

    event() {}
    event(double x, int tp, int id) : x(x), tp(tp), id(id) {}

    bool operator<(const event& e) const {
        if (abs(x - e.x) > EPS)
            return x < e.x;
        return tp > e.tp;
    }
};

set<seg> s;
vector<set<seg>::iterator> where;

set<seg>::iterator prev(set<seg>::iterator it) {
    return it == s.begin() ? s.end() : --it;
}

set<seg>::iterator next(set<seg>::iterator it) {
    return ++it;
}

pair<int, int> solve(const vector<seg>& a) {
    int n = (int)a.size();
    vector<event> e;
    for (int i = 0; i < n; ++i) {
        e.push_back(event(min(a[i].p.x, a[i].q.x), +1, i));
        e.push_back(event(max(a[i].p.x, a[i].q.x), -1, i));
    }
    sort(e.begin(), e.end());

    s.clear();
    where.resize(a.size());
    for (size_t i = 0; i < e.size(); ++i) {
        int id = e[i].id;
        if (e[i].tp == +1) {
            set<seg>::iterator nxt = s.lower_bound(a[id]), prv = prev(nxt);
            if (nxt != s.end() && intersect(*nxt, a[id]))
                return make_pair(nxt->id, id);
            if (prv != s.end() && intersect(*prv, a[id]))
                return make_pair(prv->id, id);
            where[id] = s.insert(nxt, a[id]);
        } else {
            set<seg>::iterator nxt = next(where[id]), prv = prev(where[id]);
            if (nxt != s.end() && prv != s.end() && intersect(*nxt, *prv))
                return make_pair(prv->id, nxt->id);
            s.erase(where[id]);
        }
    }

    return make_pair(-1, -1);
}
```

The main function here is `solve()`, which returns the intersecting segments if exists, or $(-1, -1)$, if there are no intersections.

Checking for the intersection of two segments is carried out by the `intersect ()` function, using an **algorithm based on the oriented area of the triangle**.

The queue of segments is the global variable `s`, a `set<event>`. Iterators that specify the position of each segment in the queue (for convenient removal of segments from the queue) are stored in the global array `where`.

Two auxiliary functions `prev()` and `next()` are also introduced, which return iterators to the previous and next elements (or `end()`, if one does not exist).

8.6 Miscellaneous

8.6.1 Finding the nearest pair of points

Problem statement

Given n points on the plane. Each point p_i is defined by its coordinates (x_i, y_i). It is required to find among them two such points, such that the distance between them is minimal:

$$\min_{\substack{i,j=0\ldots n-1,\\ i\neq j}} \rho(p_i, p_j).$$

We take the usual Euclidean distances:

$$\rho(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}.$$

The trivial algorithm - iterating over all pairs and calculating the distance for each — works in $O(n^2)$.

The algorithm running in time $O(n \log n)$ is described below. This algorithm was proposed by Shamos and Hoey in 1975. (Source: Ch. 5 Notes of *Algorithm Design* by Kleinberg & Tardos, also see here) Preparata and Shamos also showed that this algorithm is optimal in the decision tree model.

Algorithm

We construct an algorithm according to the general scheme of **divide-and-conquer** algorithms: the algorithm is designed as a recursive function, to which we pass a set of points; this recursive function splits this set in half, calls itself recursively on each half, and then performs some operations to combine the answers. The operation of combining consist of detecting the cases when one point of the optimal solution fell into one half, and the other point into the other (in this case, recursive calls from each of the halves cannot detect this pair separately). The main difficulty, as always in case of divide and conquer algorithms, lies in the effective implementation of the merging stage. If a set of n points is passed to the recursive function, then the merge stage should work no more than $O(n)$, then the asymptotics of the whole algorithm $T(n)$ will be found from the equation:

$$T(n) = 2T(n/2) + O(n).$$

The solution to this equation, as is known, is $T(n) = O(n \log n)$.

So, we proceed on to the construction of the algorithm. In order to come to an effective implementation of the merge stage in the future, we will divide the set of points into two subsets, according to their x-coordinates: In fact, we draw some vertical line dividing the set of points into two subsets of approximately the same size. It is convenient to make such a partition as follows: We sort the points in the standard way as pairs of numbers, ie.:

$$p_i < p_j \iff (x_i < x_j) \vee \Big((x_i = x_j) \wedge (y_i < y_j) \Big)$$

Then take the middle point after sorting $p_m (m = \lfloor n/2 \rfloor)$, and all the points before it and the p_m itself are assigned to the first half, and all the points after it - to the second half:

$$A_1 = \{p_i \mid i = 0 \ldots m\}$$

$$A_2 = \{p_i \mid i = m+1 \ldots n-1\}.$$

Now, calling recursively on each of the sets A_1 and A_2, we will find the answers h_1 and h_2 for each of the halves. And take the best of them: $h = \min(h_1, h_2)$.

Now we need to make a **merge stage**, i.e. we try to find such pairs of points, for which the distance between which is less than h and one point is lying in A_1 and the other in A_2. It is obvious that it is sufficient to consider only those points that are separated from the vertical line by a distance less than h, i.e. the set B of the points considered at this stage is equal to:

$$B = \{p_i \mid |x_i - x_m| < h\}.$$

For each point in the set B, we try to find the points that are closer to it than h. For example, it is sufficient to consider only those points whose y-coordinate differs by no more than h. Moreover, it makes no sense to consider those points whose y-coordinate is greater than the y-coordinate of the current point. Thus, for each point p_i we define the set of considered points $C(p_i)$ as follows:

$$C(p_i) = \{p_j \mid p_j \in B,\ \ y_i - h < y_j \le y_i\}.$$

If we sort the points of the set B by y-coordinate, it will be very easy to find $C(p_i)$: these are several points in a row ahead to the point p_i.

So, in the new notation, the **merging stage** looks like this: build a set B, sort the points in it by y-coordinate, then for each point $p_i \in B$ consider all points $p_j \in C(p_i)$, and for each pair (p_i, p_j) calculate the distance and compare with the current best distance.

At first glance, this is still a non-optimal algorithm: it seems that the sizes of sets $C(p_i)$ will be of order n, and the required asymptotics will not work. However, surprisingly, it can be proved that the size of each of the sets $C(p_i)$ is a quantity $O(1)$, i.e. it does not exceed some small constant regardless of the points themselves. Proof of this fact is given in the next section.

Finally, we pay attention to the sorting, which the above algorithm contains: first,sorting by pairs (x, y), and then second, sorting the elements of the set B by y. In fact, both of these sorts inside the recursive function can be eliminated (otherwise we would not reach the $O(n)$ estimate for the **merging stage**, and the general asymptotics of the algorithm would be $O(n \log^2 n)$). It is easy to get rid of the first sort — it is enough to perform this sort before starting the recursion: after all, the elements themselves do not change inside the recursion, so there is no need to sort again. With the second sorting a little more difficult to perform, performing it previously will not work. But, remembering the merge sort, which also works on the principle of divide-and-conquer, we can simply embed this sort in our recursion. Let recursion, taking some set of points (as we remember,ordered by pairs (x, y)), return the same set, but sorted by the y-coordinate. To do this, simply merge (in $O(n)$) the two results returned by recursive calls. This will result in a set sorted by y-coordinate.

Evaluation of the asymptotics

To show that the above algorithm is actually executed in $O(n \log n)$, we need to prove the following fact: $|C(p_i)| = O(1)$.

So, let us consider some point p_i; recall that the set $C(p_i)$ is a set of points whose y-coordinate lies in the segment $[y_i - h; y_i]$, and, moreover, along the x coordinate, the point p_i itself, and all the points of the set $C(p_i)$ lie in the band width $2h$. In other words, the points we are considering p_i and $C(p_i)$ lie in a rectangle of size $2h \times h$.

Our task is to estimate the maximum number of points that can lie in this rectangle $2h \times h$; thus, we estimate the maximum size of the set $C(p_i)$. At the same time, when evaluating, we must not forget that there may be repeated points.

Remember that h was obtained from the results of two recursive calls — on sets A_1 and A_2, and A_1 contains points to the left of the partition line and partially on it, A_2 contains the remaining points of the partition line and points to the right of it. For any pair of points from A_1, as well as from A_2, the distance can not be less than h — otherwise it would mean incorrect operation of the recursive function.

To estimate the maximum number of points in the rectangle $2h \times h$ we divide it into two squares $h \times h$, the first square include all points $C(p_i) \cap A_1$, and the second contains all the others, i.e. $C(p_i) \cap A_2$. It follows from the above considerations that in each of these squares the distance between any two points is at least h.

We show that there are at most four points in each square. For example, this can be done as follows: divide the square into 4 sub-squares with sides $h/2$. Then there can be no more than one point in each of these sub-squares (since even the diagonal is equal to $h/\sqrt{2}$, which is less than h). Therefore, there can be no more than 4 points in the whole square.

So, we have proved that in a rectangle $2h \times h$ can not be more than $4 \cdot 2 = 8$ points, and, therefore, the size of the set $C(p_i)$ cannot exceed 7, as required.

Implementation

We introduce a data structure to store a point (its coordinates and a number) and comparison operators required for two types of sorting:

```
struct pt {
    int x, y, id;
};

struct cmp_x {
    bool operator()(const pt & a, const pt & b) const {
        return a.x < b.x || (a.x == b.x && a.y < b.y);
    }
};

struct cmp_y {
    bool operator()(const pt & a, const pt & b) const {
        return a.y < b.y;
    }
};

int n;
vector<pt> a;
```

For a convenient implementation of recursion, we introduce an auxiliary function upd_ans(), which will calculate the distance between two points and check whether it is better than the current answer:

```
double mindist;
pair<int, int> best_pair;

void upd_ans(const pt & a, const pt & b) {
    double dist = sqrt((a.x - b.x)*(a.x - b.x) + (a.y - b.y)*(a.y - b.y));
    if (dist < mindist) {
        mindist = dist;
        best_pair = {a.id, b.id};
    }
}
```

Finally, the implementation of the recursion itself. It is assumed that before calling it, the array $a[]$ is already sorted by x-coordinate. In recursion we pass just two pointers l, r, which indicate that it should look for the answer for $a[l \dots r)$. If the distance between r and l is too small, the recursion must be stopped, and perform a trivial algorithm to find the nearest pair and then sort the subarray by y-coordinate.

To merge two sets of points received from recursive calls into one (ordered by y-coordinate), we use the standard STL $merge()$ function, and create an auxiliary buffer $t[]$(one for all recursive calls). (Using inplace_merge () is impractical because it generally does not work in linear time.)

Finally, the set B is stored in the same array t.

```
vector<pt> t;

void rec(int l, int r) {
    if (r - l <= 3) {
        for (int i = l; i < r; ++i) {
            for (int j = i + 1; j < r; ++j) {
                upd_ans(a[i], a[j]);
            }
        }
        sort(a.begin() + l, a.begin() + r, cmp_y());
        return;
    }

    int m = (l + r) >> 1;
    int midx = a[m].x;
    rec(l, m);
    rec(m, r);

    merge(a.begin() + l, a.begin() + m, a.begin() + m, a.begin() + r, t.begin(), cmp_y());
    copy(t.begin(), t.begin() + r - l, a.begin() + l);

    int tsz = 0;
    for (int i = l; i < r; ++i) {
        if (abs(a[i].x - midx) < mindist) {
            for (int j = tsz - 1; j >= 0 && a[i].y - t[j].y < mindist; --j)
                upd_ans(a[i], t[j]);
            t[tsz++] = a[i];
        }
```

```
    }
}
```

By the way, if all the coordinates are integer, then at the time of the recursion you can not move to fractional values, and store in $mindist$ the square of the minimum distance.

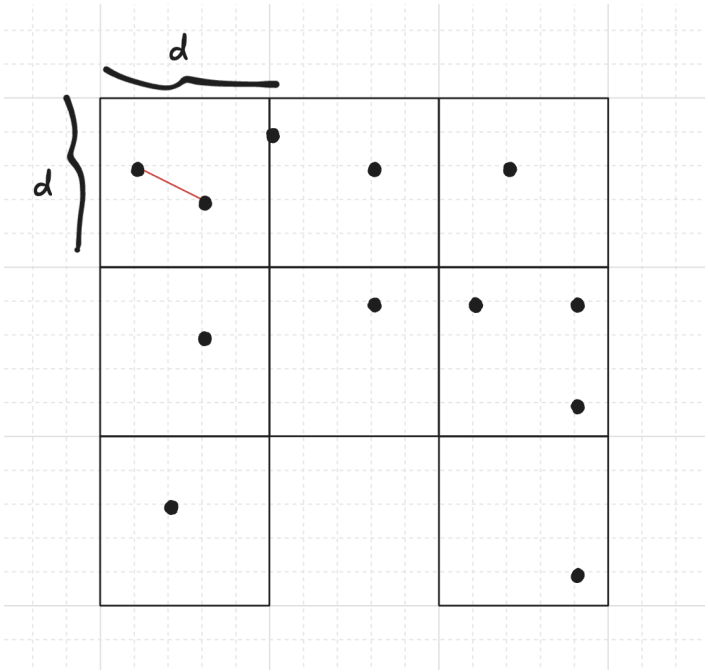
In the main program, recursion should be called as follows:

```
t.resize(n);
sort(a.begin(), a.end(), cmp_x());
mindist = 1E20;
rec(0, n);
```

Linear time randomized algorithms

A RANDOMIZED ALGORITHM WITH LINEAR EXPECTED TIME

An alternative method, originally proposed by Rabin in 1976, arises from a very simple idea to heuristically improve the runtime: We can divide the plane into a grid of $d \times d$ squares, then it is only required to test distances between same-block or adjacent-block points (unless all squares are disconnected from each other, but we will avoid this by design), since any other pair has a larger distance than the two points in the same square.



We will consider only the squares containing at least one point. Denote by $n_1, n_2, \ldots, n_k$ the number of points in each of the k remaining squares. Assuming at least two points are in the same or in adjacent squares, and that there are no duplicated points, the time complexity is $\Theta\left(\sum_{i=1}^{k} n_i^2\right)$. We can look for duplicated points in expected linear time using a hash table, and in the affirmative case, the answer is this pair.

Proof

For the i-th square containing n_i points, the number of pairs inside is $\Theta(n_i^2)$. If the i-th square is adjacent to the j-th square, then we also perform $n_i n_j \leq \max(n_i, n_j)^2 \leq n_i^2 + n_j^2$ distance comparisons. Notice that each square has at most 8 adjacent squares, so we can bound the sum of all comparisons by $\Theta(\sum_{i=1}^{k} n_i^2)$. ■

Now we need to decide on how to set d so that it minimizes $\Theta\left(\sum_{i=1}^{k} n_i^2\right)$.

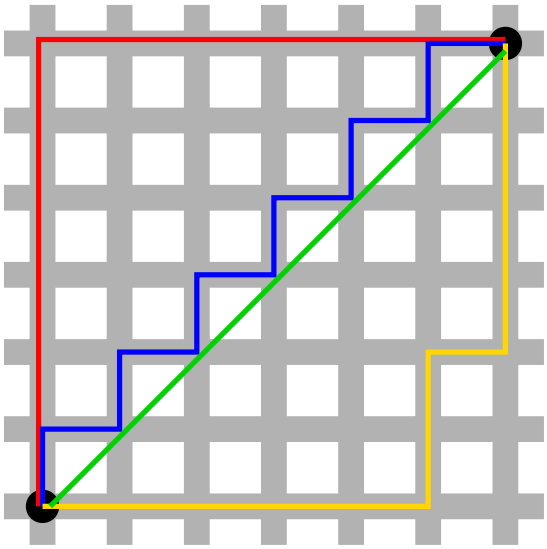
8.6.5 Manhattan Distance

Definition

For points p and q on a plane, we can define the distance between them as the sum of the differences between their x and y coordinates:

$$d(p, q) = |x_p - x_q| + |y_p - y_q|$$

Defined this way, the distance corresponds to the so-called Manhattan (taxicab) geometry, in which the points are considered intersections in a well designed city, like Manhattan, where you can only move on the streets horizontally or vertically, as shown in the image below:



This images show some of the smallest paths from one black point to the other, all of them with length 12.

There are some interesting tricks and algorithms that can be done with this distance, and we will show some of them here.

Farthest pair of points in Manhattan distance

Given n points P, we want to find the pair of points p, q that are farther apart, that is, maximize $|x_p - x_q| + |y_p - y_q|$.

Let's think first in one dimension, so $y = 0$. The main observation is that we can bruteforce if $|x_p - x_q|$ is equal to $x_p - x_q$ or $-x_p + x_q$, because if we "miss the sign" of the absolute value, we will get only a smaller value, so it can't affect the answer. More formally, it holds that:

$$|x_p - x_q| = \max(x_p - x_q, -x_p + x_q)$$

So, for example, we can try to have p such that x_p has the plus sign, and then q must have the negative sign. This way we want to find:

$$\max_{p,q \in P}(x_p + (-x_q)) = \max_{p \in P}(x_p) + \max_{q \in P}(-x_q).$$

Notice that we can extend this idea further for 2 (or more!) dimensions. For d dimensions, we must bruteforce 2^d possible values of the signs. For example, if we are in 2 dimensions and bruteforce that p has both the plus signs we want to find:

$$\max_{p,q \in P}[(x_p + (-x_q)) + (y_p + (-y_q))] = \max_{p \in P}(x_p + y_p) + \max_{q \in P}(-x_q - y_q).$$

As we made p and q independent, it is now easy to find the p and q that maximize the expression.

The code below generalizes this to d dimensions and runs in $O(n \cdot 2^d \cdot d)$.

```
long long ans = 0;
for (int msk = 0; msk < (1 << d); msk++) {
    long long mx = LLONG_MIN, mn = LLONG_MAX;
    for (int i = 0; i < n; i++) {
        long long cur = 0;
        for (int j = 0; j < d; j++) {
            if (msk & (1 << j)) cur += p[i][j];
            else cur -= p[i][j];
        }
        mx = max(mx, cur);
        mn = min(mn, cur);
    }
    ans = max(ans, mx - mn);
}
```

Rotating the points and Chebyshev distance

It's well known that, for all $m, n \in \mathbb{R}$,

$$|m| + |n| = \max(|m + n|, |m - n|).$$

To prove this, we just need to analyze the signs of m and n. And it's left as an exercise.

We may apply this equation to the Manhattan distance formula to find out that

$$d((x_1, y_1), (x_2, y_2)) = |x_1 - x_2| + |y_1 - y_2| = \max(|(x_1 + y_1) - (x_2 + y_2)|, |(y_1 - x_1) - (y_2 - x_2)|).$$

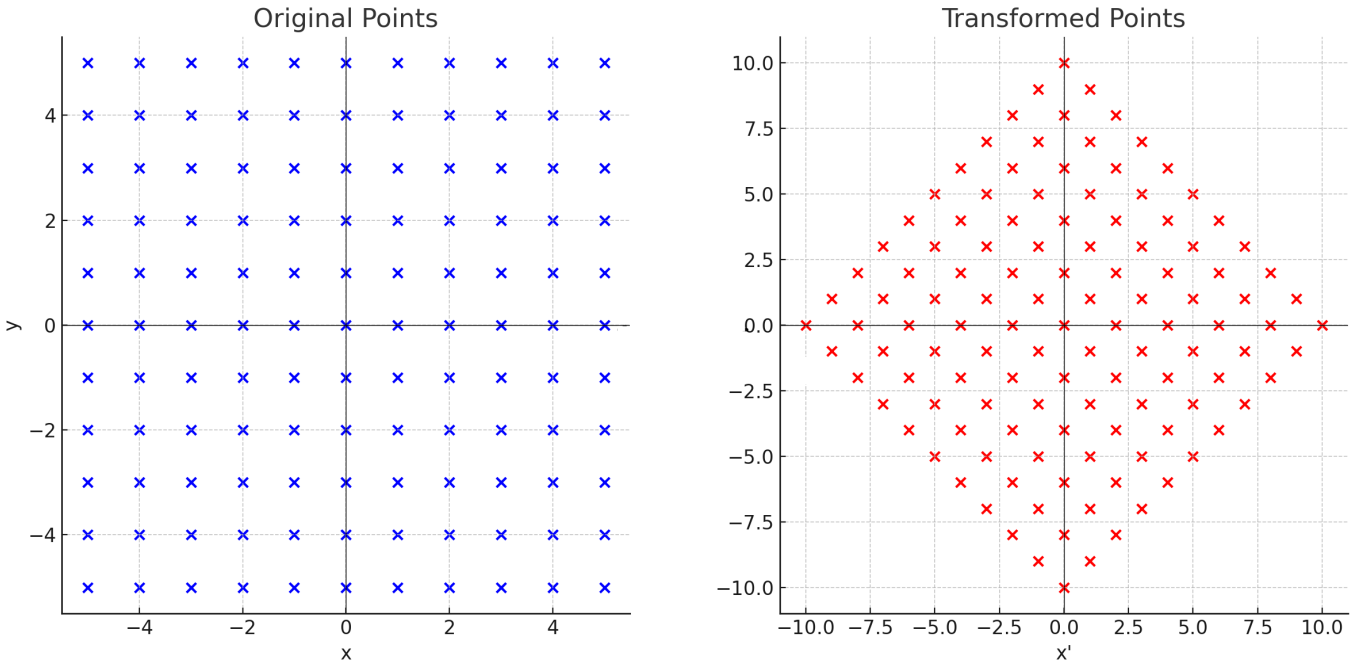
The last expression in the previous equation is the Chebyshev distance of the points $(x_1 + y_1, y_1 - x_1)$ and $(x_2 + y_2, y_2 - x_2)$. This means that, after applying the transformation

$$\alpha : (x, y) \to (x + y, y - x),$$

the Manhattan distance between the points p and q turns into the Chebyshev distance between $\alpha(p)$ and $\alpha(q)$.

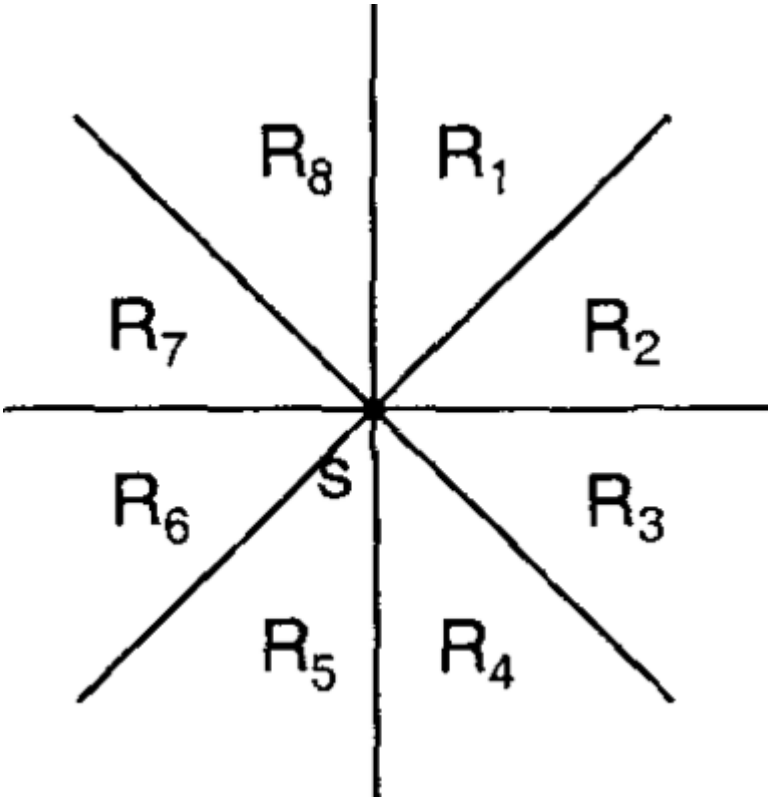
Also, we may realize that α is a spiral similarity (rotation of the plane followed by a dilation about a center O) with center $(0, 0)$, rotation angle of 45° in clockwise direction and dilation by $\sqrt{2}$.

Here's an image to help visualizing the transformation:



Manhattan Minimum Spanning Tree

The Manhattan MST problem consists of, given some points in the plane, find the edges that connect all the points and have a minimum total sum of weights. The weight of an edge that connects two points is their Manhattan distance. For simplicity, we assume that all points have different locations. Here we show a way of finding the MST in $O(n \log n)$ by finding for each point its nearest neighbor in each octant, as represented by the image below. This will give us $O(n)$ candidate edges, which, as we show below, will guarantee that they contain the MST. The final step is then using some standard MST, for example, Kruskal algorithm using disjoint set union.



The 8 octants relative to a point S

9.8 Flows and related problems

9.8.1 Maximum flow - Ford-Fulkerson and Edmonds-Karp

The Edmonds-Karp algorithm is an implementation of the Ford-Fulkerson method for computing a maximal flow in a flow network.

Flow network

First let's define what a **flow network**, a **flow**, and a **maximum flow** is.

A **network** is a directed graph G with vertices V and edges E combined with a function c, which assigns each edge $e \in E$ a non-negative integer value, the **capacity** of e. Such a network is called a **flow network**, if we additionally label two vertices, one as **source** and one as **sink**.

A **flow** in a flow network is function f, that again assigns each edge e a non-negative integer value, namely the flow. The function has to fulfill the following two conditions:

The flow of an edge cannot exceed the capacity.

$$f(e) \leq c(e)$$

And the sum of the incoming flow of a vertex u has to be equal to the sum of the outgoing flow of u except in the source and sink vertices.

$$\sum_{(v,u) \in E} f((v,u)) = \sum_{(u,v) \in E} f((u,v))$$

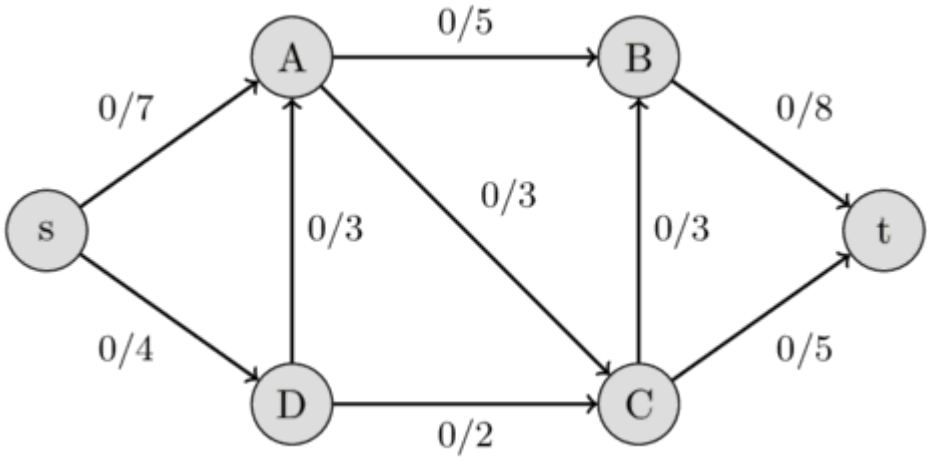
The source vertex s only has an outgoing flow, and the sink vertex t has only incoming flow.

It is easy to see that the following equation holds:

$$\sum_{(s,u) \in E} f((s,u)) = \sum_{(u,t) \in E} f((u,t))$$

A good analogy for a flow network is the following visualization: We represent edges as water pipes, the capacity of an edge is the maximal amount of water that can flow through the pipe per second, and the flow of an edge is the amount of water that currently flows through the pipe per second. This motivates the first flow condition. There cannot flow more water through a pipe than its capacity. The vertices act as junctions, where water comes out of some pipes, and then, these vertices distribute the water in some way to other pipes. This also motivates the second flow condition. All the incoming water has to be distributed to the other pipes in each junction. It cannot magically disappear or appear. The source s is origin of all the water, and the water can only drain in the sink t.

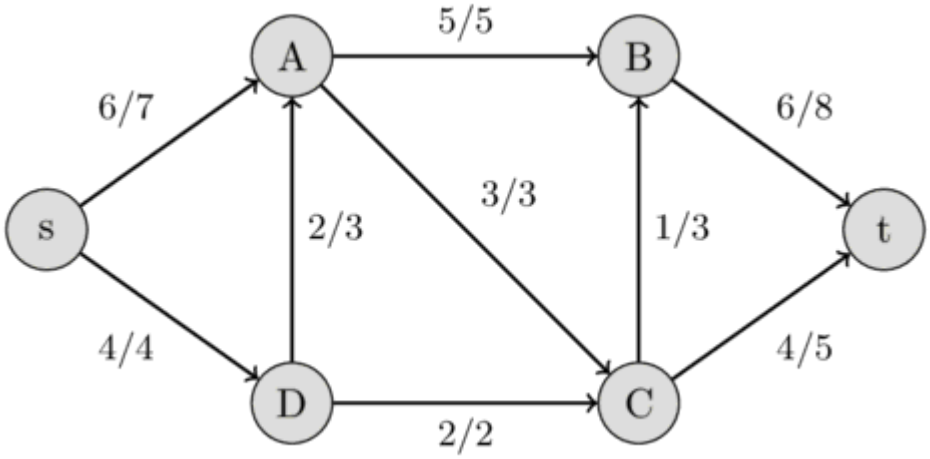
The following image shows a flow network. The first value of each edge represents the flow, which is initially 0, and the second value represents the capacity.



The value of the flow of a network is the sum of all the flows that get produced in the source s, or equivalently to the sum of all the flows that are consumed by the sink t. A **maximal flow** is a flow with the maximal possible value. Finding this maximal flow of a flow network is the problem that we want to solve.

In the visualization with water pipes, the problem can be formulated in the following way: how much water can we push through the pipes from the source to the sink?

The following image shows the maximal flow in the flow network.



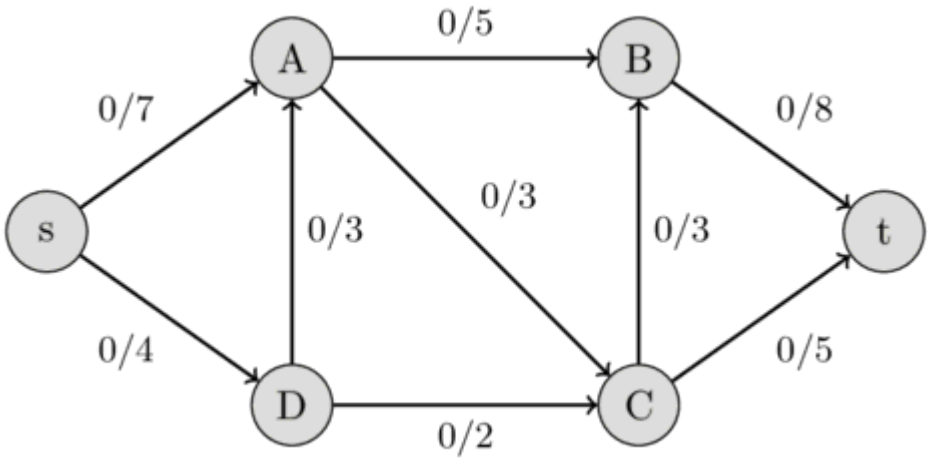
Ford-Fulkerson method

Let's define one more thing. A **residual capacity** of a directed edge is the capacity minus the flow. It should be noted that if there is a flow along some directed edge (u, v), then the reversed edge has capacity 0 and we can define the flow of it as $f((v, u)) = -f((u, v))$. This also defines the residual capacity for all the reversed edges. We can create a **residual network** from all these edges, which is just a network with the same vertices and edges, but we use the residual capacities as capacities.

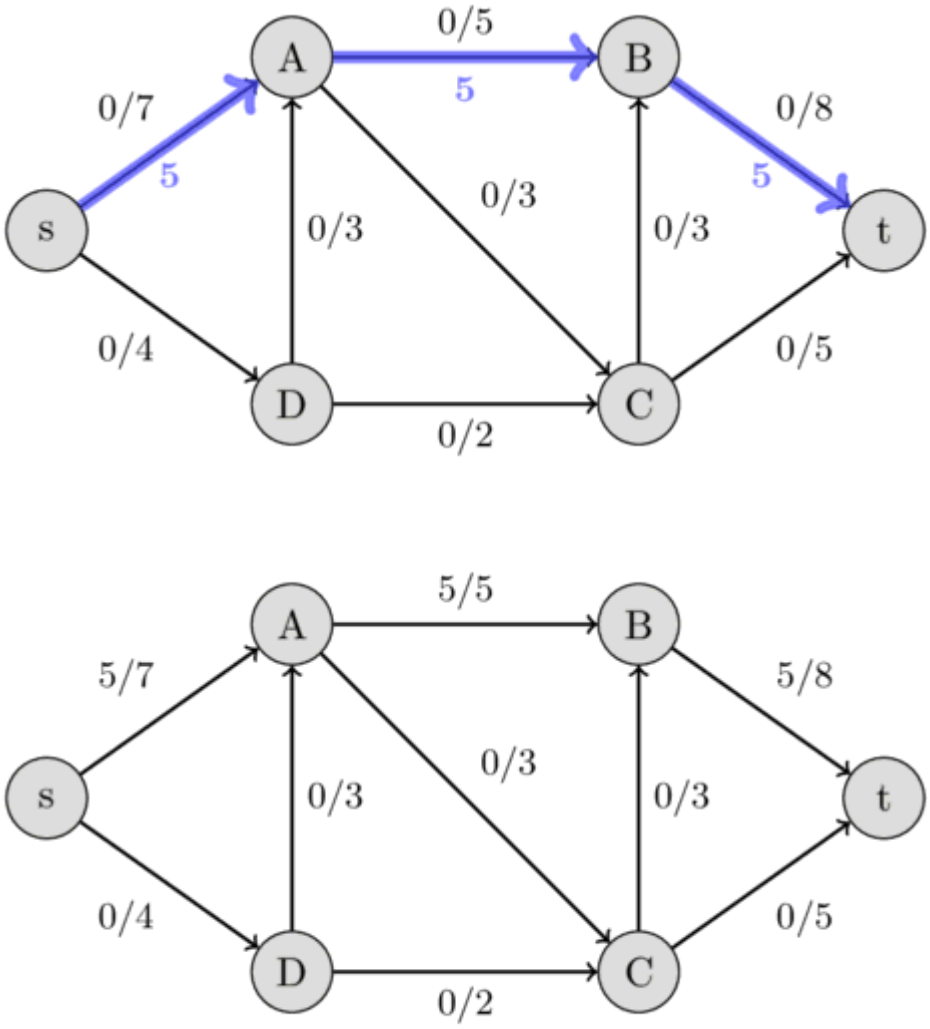
The Ford-Fulkerson method works as follows. First, we set the flow of each edge to zero. Then we look for an **augmenting path** from s to t. An augmenting path is a simple path in the residual graph where residual capacity is positive for all the edges along that path. If such a path is found, then we can increase the flow along these edges. We keep on searching for augmenting paths and increasing the flow. Once an augmenting path doesn't exist anymore, the flow is maximal.

Let us specify in more detail, what increasing the flow along an augmenting path means. Let C be the smallest residual capacity of the edges in the path. Then we increase the flow in the following way: we update $f((u,v))$ $+=$ C and $f((v,u))$ $-=$ C for every edge (u,v) in the path.

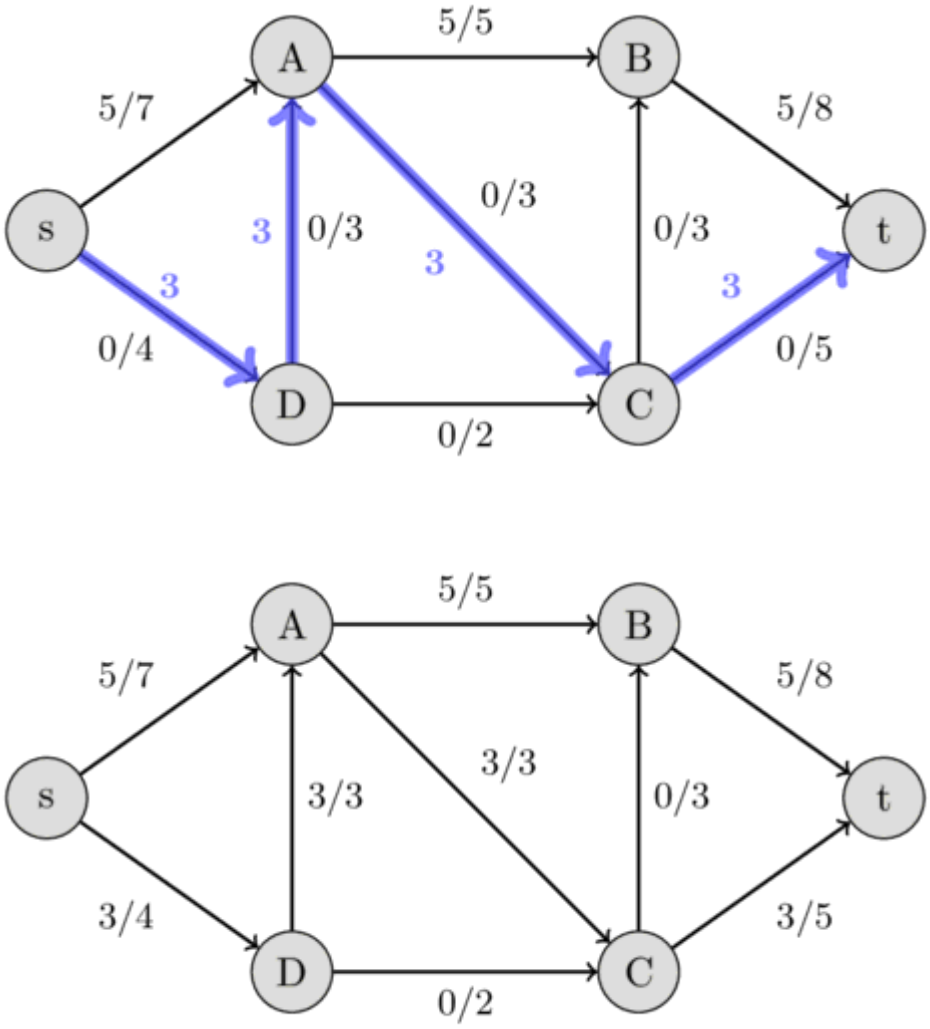
Here is an example to demonstrate the method. We use the same flow network as above. Initially we start with a flow of 0.



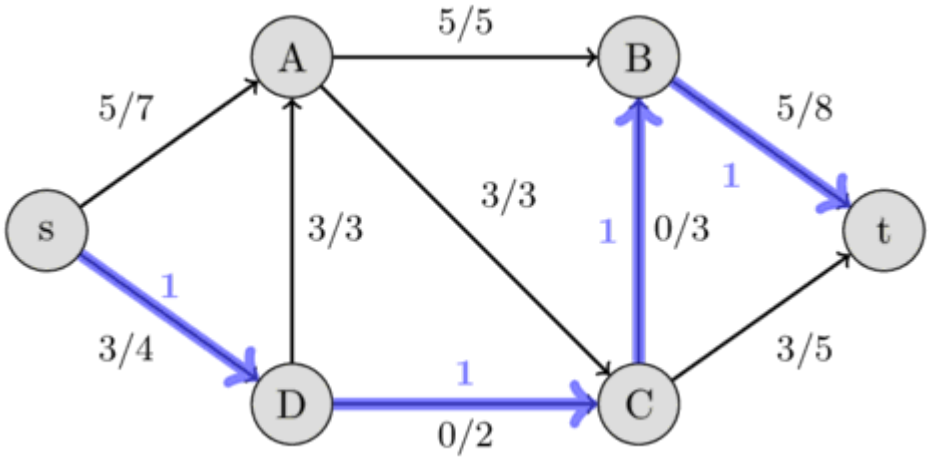
We can find the path $s-A-B-t$ with the residual capacities 7, 5, and 8. Their minimum is 5, therefore we can increase the flow along this path by 5. This gives a flow of 5 for the network.

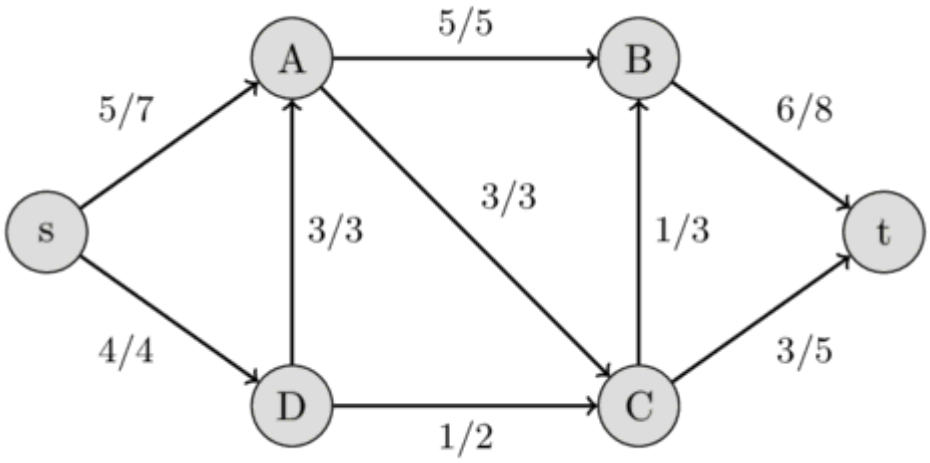


Again we look for an augmenting path, this time we find $s - D - A - C - t$ with the residual capacities 4, 3, 3, and 5. Therefore we can increase the flow by 3 and we get a flow of 8 for the network.

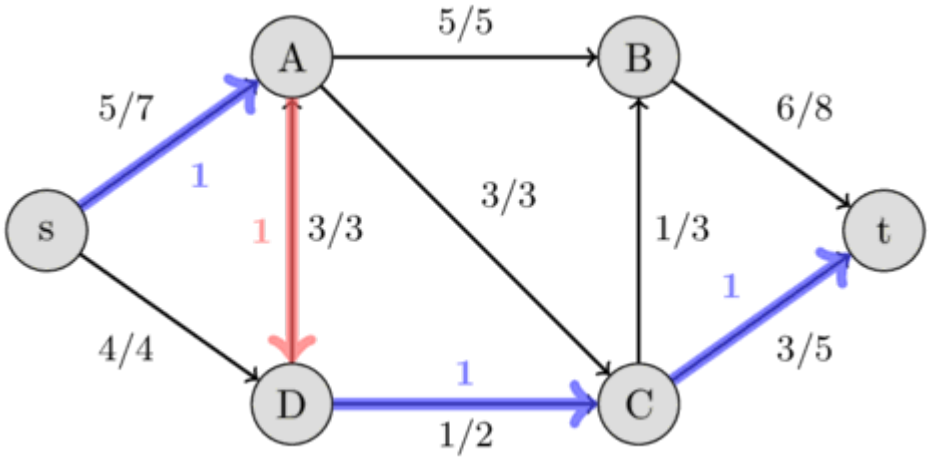


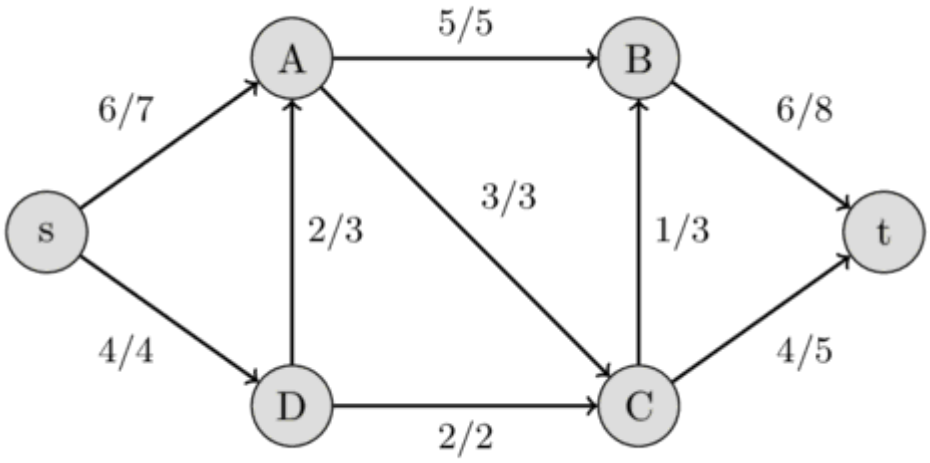
This time we find the path $s - D - C - B - t$ with the residual capacities 1, 2, 3, and 3, and hence, we increase the flow by 1.





This time we find the augmenting path $s - A - D - C - t$ with the residual capacities 2, 3, 1, and 2. We can increase the flow by 1. But this path is very interesting. It includes the reversed edge (A, D). In the original flow network, we are not allowed to send any flow from A to D. But because we already have a flow of 3 from D to A, this is possible. The intuition of it is the following: Instead of sending a flow of 3 from D to A, we only send 2 and compensate this by sending an additional flow of 1 from s to A, which allows us to send an additional flow of 1 along the path $D - C - t$.





Now, it is impossible to find an augmenting path between s and t, therefore this flow of 10 is the maximal possible. We have found the maximal flow.

It should be noted, that the Ford-Fulkerson method doesn't specify a method of finding the augmenting path. Possible approaches are using DFS or BFS which both work in $O(E)$. If all the capacities of the network are integers, then for each augmenting path the flow of the network increases by at least 1 (for more details see Integral flow theorem). Therefore, the complexity of Ford-Fulkerson is $O(EF)$, where F is the maximal flow of the network. In the case of rational capacities, the algorithm will also terminate, but the complexity is not bounded. In the case of irrational capacities, the algorithm might never terminate, and might not even converge to the maximal flow.

Edmonds-Karp algorithm

Edmonds-Karp algorithm is just an implementation of the Ford-Fulkerson method that uses BFS for finding augmenting paths. The algorithm was first published by Yefim Dinitz in 1970, and later independently published by Jack Edmonds and Richard Karp in 1972.

The complexity can be given independently of the maximal flow. The algorithm runs in $O(VE^2)$ time, even for irrational capacities. The intuition is, that every time we find an augmenting path one of the edges becomes saturated, and the distance from the edge to s will be longer if it appears later again in an augmenting path. The length of the simple paths is bounded by V.

IMPLEMENTATION

The matrix `capacity` stores the capacity for every pair of vertices. `adj` is the adjacency list of the **undirected graph**, since we also have to use the reversed of directed edges when we are looking for augmenting paths.

The function `maxflow` will return the value of the maximal flow. During the algorithm, the matrix `capacity` will actually store the residual capacity of the network. The value of the flow in each edge will actually not be stored, but it is easy to extend the implementation - by using an additional matrix - to also store the flow and return it.

```
int n;
vector<vector<int>> capacity;
vector<vector<int>> adj;

int bfs(int s, int t, vector<int>& parent) {
    fill(parent.begin(), parent.end(), -1);
    parent[s] = -2;
    queue<pair<int, int>> q;
    q.push({s, INF});

    while (!q.empty()) {
        int cur = q.front().first;
        int flow = q.front().second;
        q.pop();

        for (int next : adj[cur]) {
            if (parent[next] == -1 && capacity[cur][next]) {
                parent[next] = cur;
                int new_flow = min(flow, capacity[cur][next]);
                if (next == t)
                    return new_flow;
                q.push({next, new_flow});
            }
        }
    }

    return 0;
}

int maxflow(int s, int t) {
    int flow = 0;
    vector<int> parent(n);
    int new_flow;

    while (new_flow = bfs(s, t, parent)) {
        flow += new_flow;
        int cur = t;
        while (cur != s) {
            int prev = parent[cur];
            capacity[prev][cur] -= new_flow;
            capacity[cur][prev] += new_flow;
            cur = prev;
        }
    }

    return flow;
}
```

Integral flow theorem

The theorem says, that if every capacity in the network is an integer, then the size of the maximum flow is an integer, and there is a maximum flow such that the flow in each edge is an integer as well. In particular, Ford-Fulkerson method finds such a flow.

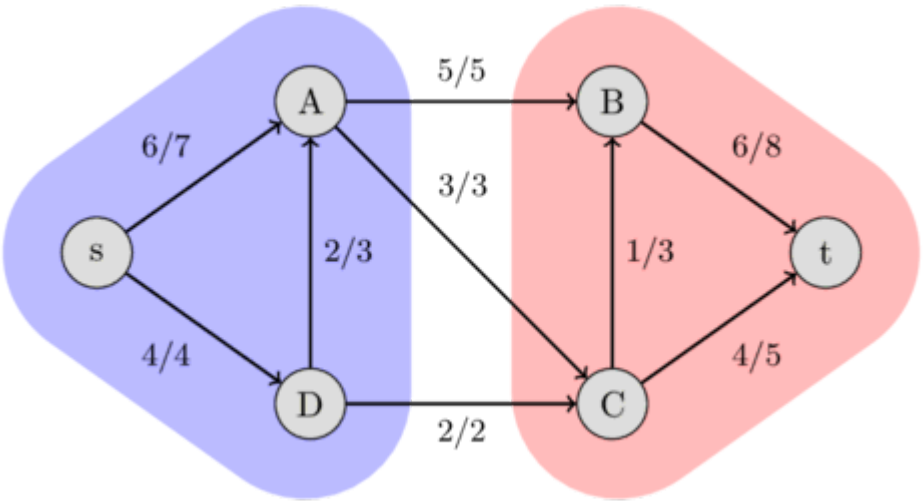
Max-flow min-cut theorem

A s-t-**cut** is a partition of the vertices of a flow network into two sets, such that a set includes the source s and the other one includes the sink t. The capacity of a s-t-cut is defined as the sum of capacities of the edges from the source side to the sink side.

Obviously, we cannot send more flow from s to t than the capacity of any s-t-cut. Therefore, the maximum flow is bounded by the minimum cut capacity.

The max-flow min-cut theorem goes even further. It says that the capacity of the maximum flow has to be equal to the capacity of the minimum cut.

In the following image, you can see the minimum cut of the flow network we used earlier. It shows that the capacity of the cut $\{s, A, D\}$ and $\{B, C, t\}$ is $5 + 3 + 2 = 10$, which is equal to the maximum flow that we found. Other cuts will have a bigger capacity, like the capacity between $\{s, A\}$ and $\{B, C, D, t\}$ is $4 + 3 + 5 = 12$.



A minimum cut can be found after performing a maximum flow computation using the Ford-Fulkerson method. One possible minimum cut is the following: the set of all the vertices that can be reached from s in the residual graph (using edges with positive residual capacity), and the set of all the other vertices. This partition can be easily found using DFS starting at s.

9.8.2 Maximum flow - Push-relabel algorithm

The push-relabel algorithm (or also known as preflow-push algorithm) is an algorithm for computing the maximum flow of a flow network. The exact definition of the problem that we want to solve can be found in the article Maximum flow - Ford-Fulkerson and Edmonds-Karp.

In this article we will consider solving the problem by pushing a preflow through the network, which will run in $O(V^4)$, or more precisely in $O(V^2E)$, time. The algorithm was designed by Andrew Goldberg and Robert Tarjan in 1985.

Definitions

During the algorithm we will have to handle a **preflow** - i.e. a function f that is similar to the flow function, but does not necessarily satisfies the flow conservation constraint. For it only the constraints

$$0 \leq f(e) \leq c(e)$$

and

$$\sum_{(v,u) \in E} f((v,u)) \geq \sum_{(u,v) \in E} f((u,v))$$

have to hold.

So it is possible for some vertex to receive more flow than it distributes. We say that this vertex has some excess flow, and define the amount of it with the **excess** function $x(u) = \sum_{(v,u) \in E} f((v,u)) - \sum_{(u,v) \in E} f((u,v))$.

In the same way as with the flow function, we can define the residual capacities and the residual graph with the preflow function.

The algorithm will start off with an initial preflow (some vertices having excess), and during the execution the preflow will be handled and modified. Giving away some details already, the algorithm will pick a vertex with excess, and push the excess to neighboring vertices. It will repeat this until all vertices, except the source and the sink, are free from excess. It is easy to see, that a preflow without excess is a valid flow. This makes the algorithm terminate with an actual flow.

There are still two problem, we have to deal with. First, how do we guarantee that this actually terminates? And secondly, how do we guarantee that this will actually give us a maximum flow, and not just any random flow?

To solve these problems we need the help of another function, namely the **labeling** functions h, often also called **height** function, which assigns each vertex an integer. We call a labeling is valid, if $h(s) = |V|$, $h(t) = 0$, and $h(u) \leq h(v) + 1$ if there is an edge (u, v) in the residual graph - i.e. the edge (u, v) has a positive capacity in the residual graph. In other words, if it is possible to increase the flow from u to v, then the height of v can be at most one smaller than the height of u, but it can be equal or even higher.

It is important to note, that if there exists a valid labeling function, then there doesn't exist an augmenting path from s to t in the residual graph. Because such a path will have a length of at most $|V| - 1$ edges, and each edge can decrease the height only by at most by one, which is impossible if the first height is $h(s) = |V|$ and the last height is $h(t) = 0$.

Using this labeling function we can state the strategy of the push-relabel algorithm: We start with a valid preflow and a valid labeling function. In each step we push some excess between vertices, and update the labels of vertices. We have to make sure, that after each step the preflow and the labeling are still valid. If then the algorithm determines, the preflow is a valid flow. And because we also have a valid labeling, there doesn't exists a path between s and t in the residual graph, which means that the flow is actually a maximum flow.

If we compare the Ford-Fulkerson algorithm with the push-relabel algorithm it seems like the algorithms are the duals of each other. The Ford-Fulkerson algorithm keeps a valid flow at all time and improves it until there doesn't exists an augmenting path any more, while in the push-relabel algorithm there doesn't exists an augmenting path at any time, and we will improve the preflow until it is a valid flow.

Algorithm

First we have to initialize the graph with a valid preflow and labeling function.

Using the empty preflow - like it is done in the Ford-Fulkerson algorithm - is not possible, because then there will be an augmenting path and this implies that there doesn't exists a valid labeling. Therefore we will initialize each edges outgoing from s with its maximal capacity: $f((s,u)) = c((s,u))$. And all other edges with zero. In this case there exists a valid labeling, namely $h(s) = |V|$ for the source vertex and $h(u) = 0$ for all other.

Now let's describe the two operations in more detail.

With the `push` operation we try to push as much excess flow from one vertex u to a neighboring vertex v. We have one rule: we are only allowed to push flow from u to v if $h(u) = h(v) + 1$. In layman's terms, the excess flow has to flow downwards, but not too steeply. Of course we only can push $\min(x(u), c((u,v)) - f((u,v)))$ flow.

If a vertex has excess, but it is not possible to push the excess to any adjacent vertex, then we need to increase the height of this vertex. We call this operation `relabel`. We will increase it by as much as it is possible, while still maintaining validity of the labeling.

To recap, the algorithm in a nutshell is: We initialize a valid preflow and a valid labeling. While we can perform push or relabel operations, we perform them. Afterwards the preflow is actually a flow and we return it.

Complexity

It is easy to show, that the maximal label of a vertex is $2|V| - 1$. At this point all remaining excess can and will be pushed back to the source. This gives at most $O(V^2)$ relabel operations.

It can also be showed, that there will be at most $O(VE)$ saturating pushes (a push where the total capacity of the edge is used) and at most $O(V^2E)$ non-saturating pushes (a push where the capacity of an edge is not fully used) performed. If we pick a data structure that allows us to find the next vertex with excess in $O(1)$ time, then the total complexity of the algorithm is $O(V^2E)$.

Implementation

```
const int inf = 1000000000;

int n;
vector<vector<int>> capacity, flow;
vector<int> height, excess, seen;
queue<int> excess_vertices;

void push(int u, int v) {
    int d = min(excess[u], capacity[u][v] - flow[u][v]);
    flow[u][v] += d;
    flow[v][u] -= d;
    excess[u] -= d;
    excess[v] += d;
    if (d && excess[v] == d)
        excess_vertices.push(v);
}

void relabel(int u) {
    int d = inf;
    for (int i = 0; i < n; i++) {
        if (capacity[u][i] - flow[u][i] > 0)
            d = min(d, height[i]);
    }
    if (d < inf)
        height[u] = d + 1;
}

void discharge(int u) {
    while (excess[u] > 0) {
        if (seen[u] < n) {
            int v = seen[u];
            if (capacity[u][v] - flow[u][v] > 0 && height[u] > height[v])
                push(u, v);
            else
                seen[u]++;
        } else {
            relabel(u);
            seen[u] = 0;
        }
    }
}
```

```
int max_flow(int s, int t) {
    height.assign(n, 0);
    height[s] = n;
    flow.assign(n, vector<int>(n, 0));
    excess.assign(n, 0);
    excess[s] = inf;
    for (int i = 0; i < n; i++) {
        if (i != s)
            push(s, i);
    }
    seen.assign(n, 0);

    while (!excess_vertices.empty()) {
        int u = excess_vertices.front();
        excess_vertices.pop();
        if (u != s && u != t)
            discharge(u);
    }

    int max_flow = 0;
    for (int i = 0; i < n; i++)
        max_flow += flow[i][t];
    return max_flow;
}
```

Here we use the queue `excess_vertices` to store all vertices that currently have excess. In that way we can pick the next vertex for a push or a relabel operation in constant time.

And to make sure that we don't spend too much time finding the adjacent vertex to whom we can push, we use a data structure called **current-arc**. Basically we will iterate over the edges in a circular order and always store the last edge that we used. This way, for a certain labeling value, we will switch the current edge only $O(n)$ time. And since the relabeling already takes $O(n)$ time, we don't make the complexity worse.

9.8.3 Maximum flow - Push-relabel method improved

We will modify the push-relabel method to achieve a better runtime.

Description

The modification is extremely simple: In the previous article we chosen a vertex with excess without any particular rule. But it turns out, that if we always choose the vertices with the **greatest height**, and apply push and relabel operations on them, then the complexity will become better. Moreover, to select the vertices with the greatest height we actually don't need any data structures, we simply store the vertices with the greatest height in a list, and recalculate the list once all of them are processed (then vertices with already lower height will be added to the list), or whenever a new vertex with excess and a greater height appears (after relabeling a vertex).

Despite the simplicity, this modification reduces the complexity by a lot. To be precise, the complexity of the resulting algorithm is $O(VE + V^2\sqrt{E})$, which in the worst case is $O(V^3)$.

This modification was proposed by Cheriyan and Maheshwari in 1989.

Implementation

```
const int inf = 1000000000;

int n;
vector<vector<int>> capacity, flow;
vector<int> height, excess;

void push(int u, int v)
{
    int d = min(excess[u], capacity[u][v] - flow[u][v]);
    flow[u][v] += d;
    flow[v][u] -= d;
    excess[u] -= d;
    excess[v] += d;
}

void relabel(int u)
{
    int d = inf;
    for (int i = 0; i < n; i++) {
        if (capacity[u][i] - flow[u][i] > 0)
            d = min(d, height[i]);
    }
    if (d < inf)
        height[u] = d + 1;
}

vector<int> find_max_height_vertices(int s, int t) {
    vector<int> max_height;
    for (int i = 0; i < n; i++) {
        if (i != s && i != t && excess[i] > 0) {
            if (!max_height.empty() && height[i] > height[max_height[0]])
                max_height.clear();
            if (max_height.empty() || height[i] == height[max_height[0]])
                max_height.push_back(i);
        }
    }
    return max_height;
}

int max_flow(int s, int t)
{
    height.assign(n, 0);
    height[s] = n;
    flow.assign(n, vector<int>(n, 0));
    excess.assign(n, 0);
    excess[s] = inf;
    for (int i = 0; i < n; i++) {
        if (i != s)
            push(s, i);
    }

    vector<int> current;
```

```
    while (!(current = find_max_height_vertices(s, t)).empty()) {
        for (int i : current) {
            bool pushed = false;
            for (int j = 0; j < n && excess[i]; j++) {
                if (capacity[i][j] - flow[i][j] > 0 && height[i] == height[j] + 1) {
                    push(i, j);
                    pushed = true;
                }
            }
            if (!pushed) {
                relabel(i);
                break;
            }
        }
    }

    return excess[t];
}
```

9.8.4 Maximum flow - Dinic's algorithm

Dinic's algorithm solves the maximum flow problem in $O(V^2E)$. The maximum flow problem is defined in this article Maximum flow - Ford-Fulkerson and Edmonds-Karp. This algorithm was discovered by Yefim Dinitz in 1970.

Definitions

A **residual network** G^R of network G is a network which contains two edges for each edge $(v, u) \in G$:

- (v, u) with capacity $c^R_{vu} = c_{vu} - f_{vu}$
- (u, v) with capacity $c^R_{uv} = f_{vu}$

A **blocking flow** of some network is such a flow that every path from s to t contains at least one edge which is saturated by this flow. Note that a blocking flow is not necessarily maximal.

A **layered network** of a network G is a network built in the following way. Firstly, for each vertex v we calculate $level[v]$ - the shortest path (unweighted) from s to this vertex using only edges with positive capacity. Then we keep only those edges (v, u) for which $level[v] + 1 = level[u]$. Obviously, this network is acyclic.

Algorithm

The algorithm consists of several phases. On each phase we construct the layered network of the residual network of G. Then we find an arbitrary blocking flow in the layered network and add it to the current flow.

Proof of correctness

Let's show that if the algorithm terminates, it finds the maximum flow.

If the algorithm terminated, it couldn't find a blocking flow in the layered network. It means that the layered network doesn't have any path from s to t. It means that the residual network doesn't have any path from s to t. It means that the flow is maximum.

Number of phases

The algorithm terminates in less than V phases. To prove this, we must firstly prove two lemmas.

Lemma 1. The distances from s to each vertex don't decrease after each iteration, i. e. $level_{i+1}[v] \geq level_i[v]$.

Proof. Fix a phase i and a vertex v. Consider any shortest path P from s to v in G^R_{i+1}. The length of P equals $level_{i+1}[v]$. Note that G^R_{i+1} can only contain edges from G^R_i and back edges for edges from G^R_i. If P has no back edges for G^R_i, then $level_{i+1}[v] \geq level_i[v]$ because P is also a path in G^R_i. Now, suppose that P has at least one back edge. Let the first such edge be (u, w).Then $level_{i+1}[u] \geq level_i[u]$ (because of the first case). The edge (u, w) doesn't belong to G^R_i, so the edge (w, u) was affected by the blocking flow on the previous iteration. It means that $level_i[u] = level_i[w] + 1$. Also, $level_{i+1}[w] = level_{i+1}[u] + 1$. From these two equations and $level_{i+1}[u] \geq level_i[u]$ we obtain $level_{i+1}[w] \geq level_i[w] + 2$. Now we can use the same idea for the rest of the path.

Lemma 2. $level_{i+1}[t] > level_i[t]$

Proof. From the previous lemma, $level_{i+1}[t] \geq level_i[t]$. Suppose that $level_{i+1}[t] = level_i[t]$. Note that G^R_{i+1} can only contain edges from G^R_i and back edges for edges from G^R_i. It means that there is a shortest path in G^R_i which wasn't blocked by the blocking flow. It's a contradiction.

From these two lemmas we conclude that there are less than V phases because $level[t]$ increases, but it can't be greater than $V - 1$.

Finding blocking flow

In order to find the blocking flow on each iteration, we may simply try pushing flow with DFS from s to t in the layered network while it can be pushed. In order to do it more quickly, we must remove the edges which can't be used to push anymore. To do this we can keep a pointer in each vertex which points to the next edge which can be used.

A single DFS run takes $O(k + V)$ time, where k is the number of pointer advances on this run. Summed up over all runs, number of pointer advances can not exceed E. On the other hand, total number of runs won't exceed E, as every run saturates at least one edge. In this way, total running time of finding a blocking flow is $O(VE)$.

Complexity

There are less than V phases, so the total complexity is $O(V^2E)$.

Unit networks

A **unit network** is a network in which for any vertex except s and t **either incoming or outgoing edge is unique and has unit capacity**. That's exactly the case with the network we build to solve the maximum matching problem with flows.

On unit networks Dinic's algorithm works in $O(E\sqrt{V})$. Let's prove this.

Firstly, each phase now works in $O(E)$ because each edge will be considered at most once.

Secondly, suppose there have already been $\sqrt{V}$ phases. Then all the augmenting paths with the length $\leq \sqrt{V}$ have been found. Let f be the current flow, f' be the maximum flow. Consider their difference $f' - f$. It is a flow in G^R of value $|f'| - |f|$ and on each edge it is either 0 or 1. It can be decomposed into $|f'| - |f|$ paths from s to t and possibly cycles. As the network is unit, they can't have common vertices, so the total number of vertices is $\geq (|f'| - |f|)\sqrt{V}$, but it is also $\leq V$, so in another $\sqrt{V}$ iterations we will definitely find the maximum flow.

UNIT CAPACITIES NETWORKS

In a more generic settings when all edges have unit capacities, *but the number of incoming and outgoing edges is unbounded*, the paths can't have common edges rather than common vertices. In a similar way it allows to prove the bound of $\sqrt{E}$ on the number of iterations, hence the running time of Dinic algorithm on such networks is at most $O(E\sqrt{E})$.

Finally, it is also possible to prove that the number of phases on unit capacity networks doesn't exceed $O(V^{2/3})$, providing an alternative estimate of $O(EV^{2/3})$ on the networks with particularly large number of edges.

Implementation

```
struct FlowEdge {
    int v, u;
    long long cap, flow = 0;
    FlowEdge(int v, int u, long long cap) : v(v), u(u), cap(cap) {}
};

struct Dinic {
    const long long flow_inf = 1e18;
    vector<FlowEdge> edges;
    vector<vector<int>> adj;
    int n, m = 0;
    int s, t;
    vector<int> level, ptr;
    queue<int> q;

    Dinic(int n, int s, int t) : n(n), s(s), t(t) {
        adj.resize(n);
        level.resize(n);
        ptr.resize(n);
    }

    void add_edge(int v, int u, long long cap) {
        edges.emplace_back(v, u, cap);
        edges.emplace_back(u, v, 0);
        adj[v].push_back(m);
        adj[u].push_back(m + 1);
        m += 2;
    }

    bool bfs() {
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            for (int id : adj[v]) {
                if (edges[id].cap == edges[id].flow)
```

```
                    continue;
                if (level[edges[id].u] != -1)
                    continue;
                level[edges[id].u] = level[v] + 1;
                q.push(edges[id].u);
            }
        }
        return level[t] != -1;
    }

    long long dfs(int v, long long pushed) {
        if (pushed == 0)
            return 0;
        if (v == t)
            return pushed;
        for (int& cid = ptr[v]; cid < (int)adj[v].size(); cid++) {
            int id = adj[v][cid];
            int u = edges[id].u;
            if (level[v] + 1 != level[u])
                continue;
            long long tr = dfs(u, min(pushed, edges[id].cap - edges[id].flow));
            if (tr == 0)
                continue;
            edges[id].flow += tr;
            edges[id ^ 1].flow -= tr;
            return tr;
        }
        return 0;
    }

    long long flow() {
        long long f = 0;
        while (true) {
            fill(level.begin(), level.end(), -1);
            level[s] = 0;
            q.push(s);
            if (!bfs())
                break;
            fill(ptr.begin(), ptr.end(), 0);
            while (long long pushed = dfs(s, flow_inf)) {
                f += pushed;
            }
        }
        return f;
    }
};
```

9.8.5 Maximum flow - MPM algorithm

MPM (Malhotra, Pramodh-Kumar and Maheshwari) algorithm solves the maximum flow problem in $O(V^3)$. This algorithm is similar to Dinic's algorithm.

Algorithm

Like Dinic's algorithm, MPM runs in phases, during each phase we find the blocking flow in the layered network of the residual network of G. The main difference from Dinic's is how we find the blocking flow. Consider the layered network L. For each node we define its' *inner potential* and *outer potential* as:

$$p_{in}(v) = \sum_{(u,v) \in L} (c(u,v) - f(u,v))$$

$$p_{out}(v) = \sum_{(v,u) \in L} (c(v,u) - f(v,u))$$

Also we set $p_{in}(s) = p_{out}(t) = \infty$. Given p_{in} and p_{out} we define the *potential* as $p(v) = min(p_{in}(v), p_{out}(v))$. We call a node r a *reference node* if $p(r) = min\{p(v)\}$. Consider a reference node r. We claim that the flow can be increased by $p(r)$ in such a way that $p(r)$ becomes 0. It is true because L is acyclic, so we can push the flow out of r by outgoing edges and it will reach t because each node has enough outer potential to push the flow out when it reaches it. Similarly, we can pull the flow from s. The construction of the blocked flow is based on this fact. On each iteration we find a reference node and push the flow from s to t through r. This process can be simulated by BFS. All completely saturated arcs can be deleted from L as they won't be used later in this phase anyway. Likewise, all the nodes different from s and t without outgoing or incoming arcs can be deleted.

Each phase works in $O(V^2)$ because there are at most V iterations (because at least the chosen reference node is deleted), and on each iteration we delete all the edges we passed through except at most V. Summing, we get $O(V^2 + E) = O(V^2)$. Since there are less than V phases (see the proof here), MPM works in $O(V^3)$ total.

Implementation

```
struct MPM{
    struct FlowEdge{
        int v, u;
        long long cap, flow;
        FlowEdge(){}
        FlowEdge(int _v, int _u, long long _cap, long long _flow)
            : v(_v), u(_u), cap(_cap), flow(_flow){}
        FlowEdge(int _v, int _u, long long _cap)
            : v(_v), u(_u), cap(_cap), flow(0ll){}
    };
    const long long flow_inf = 1e18;
    vector<FlowEdge> edges;
    vector<char> alive;
    vector<long long> pin, pout;
    vector<list<int> > in, out;
    vector<vector<int> > adj;
    vector<long long> ex;
    int n, m = 0;
    int s, t;
    vector<int> level;
    vector<int> q;
    int qh, qt;
    void resize(int _n){
        n = _n;
        ex.resize(n);
        q.resize(n);
        pin.resize(n);
        pout.resize(n);
        adj.resize(n);
        level.resize(n);
        in.resize(n);
        out.resize(n);
    }
    MPM(){}
```

```
    MPM(int _n, int _s, int _t){resize(_n); s = _s; t = _t;}
    void add_edge(int v, int u, long long cap){
        edges.push_back(FlowEdge(v, u, cap));
        edges.push_back(FlowEdge(u, v, 0));
        adj[v].push_back(m);
        adj[u].push_back(m + 1);
        m += 2;
    }
    bool bfs(){
        while(qh < qt){
            int v = q[qh++];
            for(int id : adj[v]){
                if(edges[id].cap - edges[id].flow < 1)continue;
                if(level[edges[id].u] != -1)continue;
                level[edges[id].u] = level[v] + 1;
                q[qt++] = edges[id].u;
            }
        }
        return level[t] != -1;
    }
    long long pot(int v){
        return min(pin[v], pout[v]);
    }
    void remove_node(int v){
        for(int i : in[v]){
            int u = edges[i].v;
            auto it = find(out[u].begin(), out[u].end(), i);
            out[u].erase(it);
            pout[u] -= edges[i].cap - edges[i].flow;
        }
        for(int i : out[v]){
            int u = edges[i].u;
            auto it = find(in[u].begin(), in[u].end(), i);
            in[u].erase(it);
            pin[u] -= edges[i].cap - edges[i].flow;
        }
    }
    void push(int from, int to, long long f, bool forw){
        qh = qt = 0;
        ex.assign(n, 0);
        ex[from] = f;
        q[qt++] = from;
        while(qh < qt){
            int v = q[qh++];
            if(v == to)
                break;
            long long must = ex[v];
            auto it = forw ? out[v].begin() : in[v].begin();
            while(true){
                int u = forw ? edges[*it].u : edges[*it].v;
                long long pushed = min(must, edges[*it].cap - edges[*it].flow);
                if(pushed == 0)break;
                if(forw){
                    pout[v] -= pushed;
                    pin[u] -= pushed;
                }
                else{
                    pin[v] -= pushed;
                    pout[u] -= pushed;
                }
                if(ex[u] == 0)
                    q[qt++] = u;
                ex[u] += pushed;
                edges[*it].flow += pushed;
                edges[(*it)^1].flow -= pushed;
                must -= pushed;
                if(edges[*it].cap - edges[*it].flow == 0){
                    auto jt = it;
                    ++jt;
                    if(forw){
                        in[u].erase(find(in[u].begin(), in[u].end(), *it));
                        out[v].erase(it);
                    }
                    else{
                        out[u].erase(find(out[u].begin(), out[u].end(), *it));
                        in[v].erase(it);
                    }
                    it = jt;
                }
                else break;
```

```
                if(!must)break;
            }
        }
    }
    long long flow(){
        long long ans = 0;
        while(true){
            pin.assign(n, 0);
            pout.assign(n, 0);
            level.assign(n, -1);
            alive.assign(n, true);
            level[s] = 0;
            qh = 0; qt = 1;
            q[0] = s;
            if(!bfs())
                break;
            for(int i = 0; i < n; i++){
                out[i].clear();
                in[i].clear();
            }
            for(int i = 0; i < m; i++){
                if(edges[i].cap - edges[i].flow == 0)
                    continue;
                int v = edges[i].v, u = edges[i].u;
                if(level[v] + 1 == level[u] && (level[u] < level[t] || u == t)){
                    in[u].push_back(i);
                    out[v].push_back(i);
                    pin[u] += edges[i].cap - edges[i].flow;
                    pout[v] += edges[i].cap - edges[i].flow;
                }
            }
            pin[s] = pout[t] = flow_inf;
            while(true){
                int v = -1;
                for(int i = 0; i < n; i++){
                    if(!alive[i])continue;
                    if(v == -1 || pot(i) < pot(v))
                        v = i;
                }
                if(v == -1)
                    break;
                if(pot(v) == 0){
                    alive[v] = false;
                    remove_node(v);
                    continue;
                }
                long long f = pot(v);
                ans += f;
                push(v, s, f, false);
                push(v, t, f, true);
                alive[v] = false;
                remove_node(v);
            }
        }
        return ans;
    }
};
```

9.8.6 Flows with demands

In a normal flow network the flow of an edge is only limited by the capacity $c(e)$ from above and by 0 from below. In this article we will discuss flow networks, where we additionally require the flow of each edge to have a certain amount, i.e. we bound the flow from below by a **demand** function $d(e)$:

$$d(e) \leq f(e) \leq c(e)$$

So next each edge has a minimal flow value, that we have to pass along the edge.

This is a generalization of the normal flow problem, since setting $d(e) = 0$ for all edges e gives a normal flow network. Notice, that in the normal flow network it is extremely trivial to find a valid flow, just setting $f(e) = 0$ is already a valid one. However if the flow of each edge has to satisfy a demand, than suddenly finding a valid flow is already pretty complicated.

We will consider two problems:

1. finding an arbitrary flow that satisfies all constraints
2. finding a minimal flow that satisfies all constraints

Finding an arbitrary flow

We make the following changes in the network. We add a new source s' and a new sink t', a new edge from the source s' to every other vertex, a new edge for every vertex to the sink t', and one edge from t to s. Additionally we define the new capacity function c' as:

- $c'((s', v)) = \sum_{u \in V} d((u, v))$ for each edge (s', v).
- $c'((v, t')) = \sum_{w \in V} d((v, w))$ for each edge (v, t').
- $c'((u, v)) = c((u, v)) - d((u, v))$ for each edge (u, v) in the old network.
- $c'((t, s)) = \infty$

If the new network has a saturating flow (a flow where each edge outgoing from s' is completely filled, which is equivalent to every edge incoming to t' is completely filled), then the network with demands has a valid flow, and the actual flow can be easily reconstructed from the new network. Otherwise there doesn't exist a flow that satisfies all conditions. Since a saturating flow has to be a maximum flow, it can be found by any maximum flow algorithm, like the Edmonds-Karp algorithm or the Push-relabel algorithm.

The correctness of these transformations is more difficult to understand. We can think of it in the following way: Each edge $e = (u, v)$ with $d(e) > 0$ is originally replaced by two edges: one with the capacity $d(i)$, and the other with $c(i) - d(i)$. We want to find a flow that saturates the first edge (i.e. the flow along this edge must be equal to its capacity). The second edge is less important - the flow along it can be anything, assuming that it doesn't exceed its capacity. Consider each edge that has to be saturated, and we perform the following operation: we draw the edge from the new source s' to its end v, draw the edge from its start u to the new sink t', remove the edge itself, and from the old sink t to the old source s we draw an edge of infinite capacity. By these actions we simulate the fact that this edge is saturated - from v there will be an additionally $d(e)$ flow outgoing (we simulate it with a new source that feeds the right amount of flow to v), and u will also push $d(e)$ additional flow (but instead along the old edge, this flow will go directly to the new sink t'). A flow with the value $d(e)$, that originally flowed along the path $s - \cdots - u - v - \ldots t$ can now take the new path $s' - v - \cdots - t - s - \cdots - u - t'$. The only thing that got simplified in the definition of the new network, is that if procedure created multiple edges between the same pair of vertices, then they are combined to one single edge with the summed capacity.

Minimal flow

Note that along the edge (t, s) (from the old sink to the old source) with the capacity ∞ flows the entire flow of the corresponding old network. I.e. the capacity of this edge effects the flow value of the old network. By giving this edge a sufficient large capacity (i.e. ∞), the flow of the old network is unlimited. By limiting this edge by smaller capacities, the flow value will decrease. However if we limit this edge by a too small value, than the network will not have a saturated solution, e.g. the corresponding solution for the original network will not satisfy the demand of the edges. Obviously here can use a binary search to find the lowest value with which all constraints are still satisfied. This gives the minimal flow of the original network.

9.8.7 Minimum-cost flow - Successive shortest path algorithm

Given a network G consisting of n vertices and m edges. For each edge (generally speaking, oriented edges, but see below), the capacity (a non-negative integer) and the cost per unit of flow along this edge (some integer) are given. Also the source s and the sink t are marked.

For a given value K, we have to find a flow of this quantity, and among all flows of this quantity we have to choose the flow with the lowest cost. This task is called **minimum-cost flow problem**.

Sometimes the task is given a little differently: you want to find the maximum flow, and among all maximal flows we want to find the one with the least cost. This is called the **minimum-cost maximum-flow problem**.

Both these problems can be solved effectively with the algorithm of successive shortest paths.

Algorithm

This algorithm is very similar to the Edmonds-Karp for computing the maximum flow.

SIMPLEST CASE

First we only consider the simplest case, where the graph is oriented, and there is at most one edge between any pair of vertices (e.g. if (i, j) is an edge in the graph, then (j, i) cannot be part in it as well).

Let U_{ij} be the capacity of an edge (i, j) if this edge exists. And let C_{ij} be the cost per unit of flow along this edge (i, j). And finally let $F_{i,j}$ be the flow along the edge (i, j). Initially all flow values are zero.

We **modify** the network as follows: for each edge (i, j) we add the **reverse edge** (j, i) to the network with the capacity $U_{ji} = 0$ and the cost $C_{ji} = -C_{ij}$. Since, according to our restrictions, the edge (j, i) was not in the network before, we still have a network that is not a multigraph (graph with multiple edges). In addition we will always keep the condition $F_{ji} = -F_{ij}$ true during the steps of the algorithm.

We define the **residual network** for some fixed flow F as follow (just like in the Ford-Fulkerson algorithm): the residual network contains only unsaturated edges (i.e. edges in which $F_{ij} < U_{ij}$), and the residual capacity of each such edge is $R_{ij} = U_{ij} - F_{ij}$.

Now we can talk about the **algorithms** to compute the minimum-cost flow. At each iteration of the algorithm we find the shortest path in the residual graph from s to t. In contrast to Edmonds-Karp, we look for the shortest path in terms of the cost of the path instead of the number of edges. If there doesn't exists a path anymore, then the algorithm terminates, and the stream F is the desired one. If a path was found, we increase the flow along it as much as possible (i.e. we find the minimal residual capacity R of the path, and increase the flow by it, and reduce the back edges by the same amount). If at some point the flow reaches the value K, then we stop the algorithm (note that in the last iteration of the algorithm it is necessary to increase the flow by only such an amount so that the final flow value doesn't surpass K).

It is not difficult to see, that if we set K to infinity, then the algorithm will find the minimum-cost maximum-flow. So both variations of the problem can be solved by the same algorithm.

UNDIRECTED GRAPHS / MULTIGRAPHS

The case of an undirected graph or a multigraph doesn't differ conceptually from the algorithm above. The algorithm will also work on these graphs. However it becomes a little more difficult to implement it.

An **undirected edge** (i, j) is actually the same as two oriented edges (i, j) and (j, i) with the same capacity and values. Since the above-described minimum-cost flow algorithm generates a back edge for each directed edge, so it splits the undirected edge into 4 directed edges, and we actually get a **multigraph**.

How do we deal with **multiple edges**? First the flow for each of the multiple edges must be kept separately. Secondly, when searching for the shortest path, it is necessary to take into account that it is important which of the multiple edges is used in the path. Thus instead of the usual ancestor array we additionally must store the edge number from which we came from along with the ancestor. Thirdly, as the flow increases along a certain edge, it is necessary to reduce the flow along the back edge. Since we have multiple edges, we have to store the edge number for the reversed edge for each edge.

There are no other obstructions with undirected graphs or multigraphs.

COMPLEXITY

The algorithm here is generally exponential in the size of the input. To be more specific, in the worst case it may push only as much as 1 unit of flow on each iteration, taking $O(F)$ iterations to find a minimum-cost flow of size F, making a total runtime to be $O(F \cdot T)$, where T is the time required to find the shortest path from source to sink.

If Bellman-Ford algorithm is used for this, it makes the running time $O(Fmn)$. It is also possible to modify Dijkstra's algorithm, so that it needs $O(nm)$ pre-processing as an initial step and then works in $O(m \log n)$ per iteration, making the overall running time to be $O(mn + Fm \log n)$. Here is a generator of a graph, on which such algorithm would require $O(2^{n/2} n^2 \log n)$ time.

The modified Dijkstra's algorithm uses so-called potentials from Johnson's algorithm. It is possible to combine the ideas of this algorithm and Dinic's algorithm to reduce the number of iterations from F to $\min(F, nC)$, where C is the maximum cost found among edges. You may read further about potentials and their combination with Dinic algorithm here.

Implementation

Here is an implementation using the SPFA algorithm for the simplest case.

```
struct Edge
{
    int from, to, capacity, cost;
};

vector<vector<int>> adj, cost, capacity;

const int INF = 1e9;

void shortest_paths(int n, int v0, vector<int>& d, vector<int>& p) {
    d.assign(n, INF);
    d[v0] = 0;
    vector<bool> inq(n, false);
    queue<int> q;
    q.push(v0);
    p.assign(n, -1);

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        inq[u] = false;
        for (int v : adj[u]) {
            if (capacity[u][v] > 0 && d[v] > d[u] + cost[u][v]) {
                d[v] = d[u] + cost[u][v];
                p[v] = u;
                if (!inq[v]) {
                    inq[v] = true;
                    q.push(v);
                }
            }
        }
    }
}

int min_cost_flow(int N, vector<Edge> edges, int K, int s, int t) {
    adj.assign(N, vector<int>());
    cost.assign(N, vector<int>(N, 0));
    capacity.assign(N, vector<int>(N, 0));
    for (Edge e : edges) {
        adj[e.from].push_back(e.to);
        adj[e.to].push_back(e.from);
        cost[e.from][e.to] = e.cost;
        cost[e.to][e.from] = -e.cost;
        capacity[e.from][e.to] = e.capacity;
    }

    int flow = 0;
    int cost = 0;
    vector<int> d, p;
    while (flow < K) {
        shortest_paths(N, s, d, p);
        if (d[t] == INF)
            break;

        // find max flow on that path
        int f = K - flow;
```

```
        int cur = t;
        while (cur != s) {
            f = min(f, capacity[p[cur]][cur]);
            cur = p[cur];
        }

        // apply flow
        flow += f;
        cost += f * d[t];
        cur = t;
        while (cur != s) {
            capacity[p[cur]][cur] -= f;
            capacity[cur][p[cur]] += f;
            cur = p[cur];
        }
    }

    if (flow < K)
        return -1;
    else
        return cost;
}
```

9.8.8 Solving assignment problem using min-cost-flow

The **assignment problem** has two equivalent statements:

- Given a square matrix $A[1..N, 1..N]$, you need to select N elements in it so that exactly one element is selected in each row and column, and the sum of the values of these elements is the smallest.
- There are N orders and N machines. The cost of manufacturing on each machine is known for each order. Only one order can be performed on each machine. It is required to assign all orders to the machines so that the total cost is minimized.

Here we will consider the solution of the problem based on the algorithm for finding the minimum cost flow (min-cost-flow), solving the assignment problem in $\mathcal{O}(N^3)$.

Description

Let's build a bipartite network: there is a source S, a drain T, in the first part there are N vertices (corresponding to rows of the matrix, or orders), in the second there are also N vertices (corresponding to the columns of the matrix, or machines). Between each vertex i of the first set and each vertex j of the second set, we draw an edge with bandwidth 1 and cost A_{ij}. From the source S we draw edges to all vertices i of the first set with bandwidth 1 and cost 0. We draw an edge with bandwidth 1 and cost 0 from each vertex of the second set j to the drain T.

We find in the resulting network the maximum flow of the minimum cost. Obviously, the value of the flow will be N. Further, for each vertex i of the first segment there is exactly one vertex j of the second segment, such that the flow F_{ij} = 1. Finally, this is a one-to-one correspondence between the vertices of the first segment and the vertices of the second part, which is the solution to the problem (since the found flow has a minimal cost, then the sum of the costs of the selected edges will be the lowest possible, which is the optimality criterion).

The complexity of this solution of the assignment problem depends on the algorithm by which the search for the maximum flow of the minimum cost is performed. The complexity will be $\mathcal{O}(N^3)$ using Dijkstra or $\mathcal{O}(N^4)$ using Bellman-Ford. This is due to the fact that the flow is of size $O(N)$ and each iteration of Dijkstra algorithm can be performed in $O(N^2)$, while it is $O(N^3)$ for Bellman-Ford.

Implementation

The implementation given here is long, it can probably be significantly reduced. It uses the SPFA algorithm for finding shortest paths.

```
const int INF = 1000 * 1000 * 1000;

vector<int> assignment(vector<vector<int>> a) {
    int n = a.size();
    int m = n * 2 + 2;
    vector<vector<int>> f(m, vector<int>(m));
    int s = m - 2, t = m - 1;
    int cost = 0;
    while (true) {
        vector<int> dist(m, INF);
        vector<int> p(m);
        vector<bool> inq(m, false);
        queue<int> q;
        dist[s] = 0;
        p[s] = -1;
        q.push(s);
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            inq[v] = false;
            if (v == s) {
                for (int i = 0; i < n; ++i) {
                    if (f[s][i] == 0) {
                        dist[i] = 0;
                        p[i] = s;
                        inq[i] = true;
                        q.push(i);
                    }
                }
            } else {
                if (v < n) {
                    for (int j = n; j < n + n; ++j) {
                        if (f[v][j] < 1 && dist[j] > dist[v] + a[v][j - n]) {
                            dist[j] = dist[v] + a[v][j - n];
                            p[j] = v;
```

```
                            if (!inq[j]) {
                                q.push(j);
                                inq[j] = true;
                            }
                        }
                    }
                } else {
                    for (int j = 0; j < n; ++j) {
                        if (f[v][j] < 0 && dist[j] > dist[v] - a[j][v - n]) {
                            dist[j] = dist[v] - a[j][v - n];
                            p[j] = v;
                            if (!inq[j]) {
                                q.push(j);
                                inq[j] = true;
                            }
                        }
                    }
                }
            }
        }

        int curcost = INF;
        for (int i = n; i < n + n; ++i) {
            if (f[i][t] == 0 && dist[i] < curcost) {
                curcost = dist[i];
                p[t] = i;
            }
        }
        if (curcost == INF)
            break;
        cost += curcost;
        for (int cur = t; cur != -1; cur = p[cur]) {
            int prev = p[cur];
            if (prev != -1)
                f[cur][prev] = -(f[prev][cur] = 1);
        }
    }

    vector<int> answer(n);
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            if (f[i][j + n] == 1)
                answer[i] = j;
        }
    }
    return answer;
}
```

9.8.9 Minimum cut - Stoer-Wagner algorithm

Problem statement

Given an undirected weighted graph G with n vertices and m edges. A cut C is a non-empty proper subset of the vertices (effectively, a cut is a partition of the vertices into two non-empty sets: the ones belonging to C and all the others). The weight of a cut is the sum of the weights of the edges crossing the cut, i.e. the edges that have exactly one endpoint in C:

$$w(C) = \sum_{\substack{(v,u) \in E \\ u \in C,\, v \notin C}} c(v,u),$$

where E denotes the set of all edges of the graph G, and $c(v,u)$ is the weight of the edge (v,u).

The task is to find a **cut of minimum weight**.

Sometimes this problem is called the "global minimum cut" — in contrast to the problem where a source vertex and a sink vertex are given, and we have to find a minimum cut C containing the sink but not the source. The global minimum cut equals the minimum among the minimum-cost cuts over all possible source-sink pairs.

Although this problem can be solved with a maximum flow algorithm (by running it $O(n^2)$ times for all possible pairs of source and sink), below we describe a much simpler and faster algorithm, proposed by Mechthild Stoer and Frank Wagner in 1994.

In general, loops and multiple edges are allowed, although loops obviously do not influence the result in any way, and multiple edges can always be replaced by a single edge with their combined weight. Therefore, for simplicity, we will assume that the input graph contains no loops or multiple edges.

Description of the algorithm

The **basic idea** of the algorithm is very simple. We iteratively repeat the following process: find the minimum cut between some pair of vertices s and t, and then merge these two vertices into one (connecting their adjacency lists). Eventually, after $n-1$ iterations, the graph will be compressed into a single vertex and the process stops. After that, the answer is the minimum among all $n-1$ cuts found. Indeed, at every i-th stage, the found minimum cut C_i between the vertices s_i and t_i either turns out to be the desired global minimum cut, or, on the contrary, it is disadvantageous to put s_i and t_i into different sets, so we do not worsen anything by merging these two vertices into one.

Thus we reduced the problem to the following one: for a given graph, find the **minimum cut between some, arbitrary, pair of vertices** s and t. To solve this problem, the following, also iterative, process was proposed. We introduce a set of vertices A, which initially contains a single arbitrary vertex. At every step, we find the vertex **most strongly connected** to the set A, i.e. the vertex $v \notin A$ for which the following quantity is maximal:

$$w(v, A) = \sum_{\substack{(v,u) \in E \\ u \in A}} c(v,u)$$

(i.e. the sum of the weights of the edges with one endpoint at v and the other in A is maximal).

Again, this process terminates after $n-1$ iterations, when all vertices have moved into the set A (incidentally, this process strongly resembles Prim's algorithm). Then, as the **Stoer-Wagner theorem** states, if we denote by s and t the last two vertices added to A, the minimum cut between the vertices s and t consists of a single vertex $-$ t. The proof of this theorem will be given in the next section (as is often the case, by itself it does not contribute to the understanding of the algorithm in any way).

Thus the overall **scheme of the Stoer-Wagner algorithm** is as follows. The algorithm consists of $n-1$ phases. In every phase, the set A is initially set to contain some vertex, and the starting weights $w(v, A)$ of the vertices are computed. Then $n-1$ iterations follow, in each of which the vertex u with the largest value $w(v, A)$ is chosen and added to the set A, after which the values of w for the remaining vertices are recalculated (for which, obviously, we have to go through all the edges in the adjacency list of the selected vertex u). After performing all the iterations, we record in s and t the last two added vertices, and the value $w(t, A \setminus t)$ can be taken as the cost of the found minimum cut between s and t. Then we compare the found minimum cut with the current answer, and if it is smaller, we update the answer, and proceed to the next phase.

If we do not use any sophisticated data structures, the most critical part is finding the vertex with the largest value of w. If we do this in $O(n)$, then, given that there are $n-1$ phases with $n-1$ iterations each, the resulting **complexity of the algorithm** is $O(n^3)$.

If we use **Fibonacci heaps** for finding the vertex with the largest value of w (which allow increasing the value of a key in $O(1)$ amortized and extracting the maximum in $O(\log n)$ amortized), then all operations related to the set A in one phase are performed in $O(m + n \log n)$. The resulting complexity of the algorithm in this case is $O(nm + n^2 \log n)$.

Proof of the Stoer-Wagner theorem

Let us recall the statement of this theorem. If we add all the vertices to the set A one by one, every time adding the vertex most strongly connected to this set, then denote the second-to-last added vertex by s and the last one by t. Then the minimum s-t cut consists of a single vertex $-$ t.

To prove it, consider an arbitrary s-t cut C and show that its weight cannot be less than the weight of the cut consisting of the single vertex t:

$$w(\{t\}) \leq w(C).$$

To do this, we prove the following fact. Let A_v be the state of the set A immediately before adding the vertex v. Let C_v be the cut of the set $A_v \cup \{v\}$ induced by the cut C (simply put, C_v equals the intersection of these two sets of vertices). Further, a vertex v is called active (with respect to the cut C) if the vertex v and the previously added vertex belong to different parts of the cut C. Then, we claim, for any active vertex v the following inequality holds:

$$w(v, A_v) \leq w(C_v).$$

In particular, t is an active vertex (since the vertex added before it was s), and for $v = t$ this inequality turns into the statement of the theorem:

$$w(t, A_t) = w(\{t\}) \leq w(C_t) = w(C).$$

So, we will prove this inequality using mathematical induction.

For the first active vertex v the inequality holds (moreover, it turns into an equality) $-$ since all vertices of A_v belong to one part of the cut, and v belongs to the other.

Now suppose this inequality holds for all active vertices up to some vertex v; let us prove it for the next active vertex u. To do this, transform the left-hand side:

$$w(u, A_u) \equiv w(u, A_v) + w(u, A_u \setminus A_v).$$

First, note that:

$$w(u, A_v) \leq w(v, A_v),$$

which follows from the fact that when the set A was equal to A_v, the vertex added to it was precisely v and not u, which means it had the largest value of w.

Further, since $w(v, A_v) \leq w(C_v)$ by the induction hypothesis, we obtain:

$$w(u, A_v) \leq w(C_v),$$

from which we have:

$$w(u, A_u) \leq w(C_v) + w(u, A_u \setminus A_v).$$

Now note that the vertex u and all vertices of $A_u \setminus A_v$ are in different parts of the cut C, so the quantity $w(u, A_u \setminus A_v)$ denotes the sum of the weights of the edges that are counted in $w(C_u)$ but were not yet counted in $w(C_v)$, from which we get:

$$w(u, A_u) \leq w(C_v) + w(u, A_u \setminus A_v) \leq w(C_u),$$

as required.

We have proven the relation $w(v, A_v) \leq w(C_v)$, and, as mentioned above, the whole theorem follows from it.

Implementation

For the simplest and clearest implementation (with $O(n^3)$ complexity), the graph is represented as an adjacency matrix. The answer is stored in the variables `best_cost` and `best_cut` (the cost of the minimum cut and the vertices contained in it).

For every vertex, the array `exist` stores whether it still exists, or whether it has been merged with some other vertex. The list `v[i]` for every compressed vertex i stores the numbers of the original vertices that were compressed into this vertex i.

The algorithm consists of $n - 1$ phases (the loop over the variable `ph`). In every phase, all vertices are initially outside the set A, so the array `in_a` is filled with zeros, and the connectivities w of all vertices are zero. In each of the $n - \text{ph}$ iterations, the vertex `sel` with the largest value of w is found. If this is the last iteration, the answer is updated if necessary, and the second-to-last `prev` and the last `sel` selected vertices are merged into one. If the iteration is not the last one, then `sel` is added to the set A, after which the weights of all the remaining vertices are recalculated.

Note that the algorithm "spoils" the graph `g` during its work, so if you still need it later, you have to save a copy of it before calling the function.

```
const int MAXN = 500;
int n;
long long g[MAXN][MAXN];
long long best_cost = (1LL << 62);
vector<int> best_cut;

void mincut() {
    vector<int> v[MAXN];
    for (int i = 0; i < n; ++i)
        v[i].assign(1, i);
    long long w[MAXN];
    bool exist[MAXN], in_a[MAXN];
    memset(exist, true, sizeof exist);
    for (int ph = 0; ph < n - 1; ++ph) {
        memset(in_a, false, sizeof in_a);
        memset(w, 0, sizeof w);
        for (int it = 0, prev; it < n - ph; ++it) {
            int sel = -1;
            for (int i = 0; i < n; ++i)
                if (exist[i] && !in_a[i] && (sel == -1 || w[i] > w[sel]))
                    sel = i;
            if (it == n - ph - 1) {
                if (w[sel] < best_cost) {
                    best_cost = w[sel];
                    best_cut = v[sel];
                }
                v[prev].insert(v[prev].end(), v[sel].begin(), v[sel].end());
                for (int i = 0; i < n; ++i)
                    g[prev][i] = g[i][prev] += g[sel][i];
                exist[sel] = false;
            } else {
                in_a[sel] = true;
                for (int i = 0; i < n; ++i)
                    w[i] += g[sel][i];
                prev = sel;
            }
        }
    }
}
```

Literature

- Mechthild Stoer, Frank Wagner. A Simple Min-Cut Algorithm. Journal of the ACM, 44(4):585-591, 1997
- Kurt Mehlhorn, Christian Uhrig. The minimum cut algorithm of Stoer and Wagner [1995]

9.9 Matchings and related problems

9.9.1 Check whether a graph is bipartite

A bipartite graph is a graph whose vertices can be divided into two disjoint sets so that every edge connects two vertices from different sets (i.e. there are no edges which connect vertices from the same set). These sets are usually called sides.

You are given an undirected graph. Check whether it is bipartite, and if it is, output its sides.

Algorithm

There exists a theorem which claims that a graph is bipartite if and only if all its cycles have even length. However, in practice it's more convenient to use a different formulation of the definition: a graph is bipartite if and only if it is two-colorable.

Let's use a series of breadth-first searches, starting from each vertex which hasn't been visited yet. In each search, assign the vertex from which we start to side 1. Each time we visit a yet unvisited neighbor of a vertex assigned to one side, we assign it to the other side. When we try to go to a neighbor of a vertex assigned to one side which has already been visited, we check that it has been assigned to the other side; if it has been assigned to the same side, we conclude that the graph is not bipartite. Once we've visited all vertices and successfully assigned them to sides, we know that the graph is bipartite and we have constructed its partitioning.

Implementation

```
int n;
vector<vector<int>> adj;

vector<int> side(n, -1);
bool is_bipartite = true;
queue<int> q;
for (int st = 0; st < n; ++st) {
    if (side[st] == -1) {
        q.push(st);
        side[st] = 0;
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            for (int u : adj[v]) {
                if (side[u] == -1) {
                    side[u] = side[v] ^ 1;
                    q.push(u);
                } else {
                    is_bipartite &= side[u] != side[v];
                }
            }
        }
    }
}

cout << (is_bipartite ? "YES" : "NO") << endl;
```

9.9.2 Kuhn's Algorithm for Maximum Bipartite Matching

Problem

You are given a bipartite graph G containing n vertices and m edges. Find the maximum matching, i.e., select as many edges as possible so that no selected edge shares a vertex with any other selected edge.

Algorithm Description

REQUIRED DEFINITIONS

- A **matching** M is a set of pairwise non-adjacent edges of a graph (in other words, no more than one edge from the set should be incident to any vertex of the graph M). The **cardinality** of a matching is the number of edges in it. All those vertices that have an adjacent edge from the matching (i.e., which have degree exactly one in the subgraph formed by M) are called **saturated** by this matching.
- A **maximal matching** is a matching M of a graph G that is not a subset of any other matching.
- A **maximum matching** (also known as maximum-cardinality matching) is a matching that contains the largest possible number of edges. Every maximum matching is a maximal matching.
- A **path** of length k here means a *simple* path (i.e. not containing repeated vertices or edges) containing k edges, unless specified otherwise.
- An **alternating path** (in a bipartite graph, with respect to some matching) is a path in which the edges alternately belong / do not belong to the matching.
- An **augmenting path** (in a bipartite graph, with respect to some matching) is an alternating path whose initial and final vertices are unsaturated, i.e., they do not belong in the matching.
- The **symmetric difference** (also known as the **disjunctive union**) of sets A and B, represented by $A \oplus B$, is the set of all elements that belong to exactly one of A or B, but not to both. That is, $A \oplus B = (A - B) \cup (B - A) = (A \cup B) - (A \cap B)$.

BERGE'S LEMMA

This lemma was proven by the French mathematician **Claude Berge** in 1957, although it already was observed by the Danish mathematician **Julius Petersen** in 1891 and the Hungarian mathematician **Denés Kőnig** in 1931.

Formulation

A matching M is maximum $\Leftrightarrow$ there is no augmenting path relative to the matching M.

Proof

Both sides of the bi-implication will be proven by contradiction.

1. A matching M is maximum $\Rightarrow$ there is no augmenting path relative to the matching M.

   Let there be an augmenting path P relative to the given maximum matching M. This augmenting path P will necessarily be of odd length, having one more edge not in M than the number of edges it has that are also in M. We create a new matching M' by including all edges in the original matching M except those also in the P, and the edges in P that are not in M. This is a valid matching because the initial and final vertices of P are unsaturated by M, and the rest of the vertices are saturated only by the matching $P \cap M$. This new matching M' will have one more edge than M, and so M could not have been maximum.

   Formally, given an augmenting path P w.r.t. some maximum matching M, the matching $M' = P \oplus M$ is such that $|M'| = |M| + 1$, a contradiction.

2. A matching M is maximum $\Leftarrow$ there is no augmenting path relative to the matching M.

   Let there be a matching M' of greater cardinality than M. We consider the symmetric difference $Q = M \oplus M'$. The subgraph Q is no longer necessarily a matching. Any vertex in Q has a maximum degree of 2, which means that all connected components in it are one of the three -

   - an isolated vertex
   - a (simple) path whose edges are alternately from M and M'
   - a cycle of even length whose edges are alternately from M and M'

   Since M' has a cardinality greater than M, Q has more edges from M' than M. By the Pigeonhole principle, at least one connected component will be a path having more edges from M' than M. Because any such path is alternating, it will have initial and final vertices unsaturated by M, making it an augmenting path for M, which contradicts the premise. ■

KUHN'S ALGORITHM

Kuhn's algorithm is a direct application of Berge's lemma. It is essentially described as follows:

First, we take an empty matching. Then, while the algorithm is able to find an augmenting path, we update the matching by alternating it along this path and repeat the process of finding the augmenting path. As soon as it is not possible to find such a path, we stop the process - the current matching is the maximum.

It remains to detail the way to find augmenting paths. Kuhn's algorithm simply searches for any of these paths using depth-first or breadth-first traversal. The algorithm looks through all the vertices of the graph in turn, starting each traversal from it, trying to find an augmenting path starting at this vertex.

The algorithm is more convenient to describe if we assume that the input graph is already split into two parts (although, in fact, the algorithm can be implemented in such a way that the input graph is not explicitly split into two parts).

The algorithm looks at all the vertices v of the first part of the graph: $v = 1 \dots n_1$. If the current vertex v is already saturated with the current matching (i.e., some edge adjacent to it has already been selected), then skip this vertex. Otherwise, the algorithm tries to saturate this vertex, for which it starts a search for an augmenting path starting from this vertex.

The search for an augmenting path is carried out using a special depth-first or breadth-first traversal (usually depth-first traversal is used for ease of implementation). Initially, the depth-first traversal is at the current unsaturated vertex v of the first part. Let's look through all edges from this vertex. Let the current edge be an edge (v, to). If the vertex to is not yet saturated with matching, then we have succeeded in finding an augmenting path: it consists of a single edge (v, to); in this case, we simply include this edge in the matching and stop searching for the augmenting path from the vertex v. Otherwise, if to is already saturated with some edge (to, p), then will go along this edge: thus we will try to find an augmenting path passing through the edges $(v, to), (to, p), \dots$. To do this, simply go to the vertex p in our traversal - now we try to find an augmenting path from this vertex.

So, this traversal, launched from the vertex v, will either find an augmenting path, and thereby saturate the vertex v, or it will not find such an augmenting path (and, therefore, this vertex v cannot be saturated).

After all the vertices $v = 1 \dots n_1$ have been scanned, the current matching will be maximum.

RUNNING TIME

Kuhn's algorithm can be thought of as a series of n depth/breadth-first traversal runs on the entire graph. Therefore, the whole algorithm is executed in time $O(nm)$, which in the worst case is $O(n^3)$.

However, this estimate can be improved slightly. It turns out that for Kuhn's algorithm, it is important which part of the graph is chosen as the first and which as the second. Indeed, in the implementation described above, the depth/breadth-first traversal starts only from the vertices of the first part, so the entire algorithm is executed in time $O(n_1 m)$, where n_1 is the number of vertices of the first part. In the worst case, this is $O(n_1^2 n_2)$ (where n_2 is the number of vertices of the second part). This shows that it is more profitable when the first part contains fewer vertices than the second. On very unbalanced graphs (when n_1 and n_2 are very different), this translates into a significant difference in runtimes.

Implementation

STANDARD IMPLEMENTATION

Let us present here an implementation of the above algorithm based on depth-first traversal and accepting a bipartite graph in the form of a graph explicitly split into two parts. This implementation is very concise, and perhaps it should be remembered in this form.

Here n is the number of vertices in the first part, k - in the second part, $g[v]$ is the list of edges from the top of the first part (i.e. the list of numbers of the vertices to which these edges lead from v). The vertices in both parts are numbered independently, i.e. vertices in the first part are numbered $1 \dots n$, and those in the second are numbered $1 \dots k$.

Then there are two auxiliary arrays: mt and used. The first - mt - contains information about the current matching. For convenience of programming, this information is contained only for the vertices of the second part: $\mathrm{mt}[i]$ - this is the number of the vertex of the first part connected by an edge with the vertex i of the second part (or -1, if no matching edge comes out of it). The second array is used: the usual array of "visits" to the vertices in the depth-first traversal (it is needed just so that the depth-first traversal does not enter the same vertex twice).

A function $\mathrm{try\_kuhn}$ is a depth-first traversal. It returns true if it was able to find an augmenting path from the vertex v, and it is considered that this function has already performed the alternation of matching along the found chain.

Inside the function, all the edges outgoing from the vertex v of the first part are scanned, and then the following is checked: if this edge leads to an unsaturated vertex to, or if this vertex to is saturated, but it is possible to find an increasing chain by recursively starting from $\mathrm{mt}[to]$, then we say that we have found an augmenting path, and before returning from the function with the result true, we alternate the current edge: we redirect the edge adjacent to to to the vertex v.

The main program first indicates that the current matching is empty (the list mt is filled with numbers -1). Then the vertex v of the first part is searched by $\mathrm{try\_kuhn}$, and a depth-first traversal is started from it, having previously zeroed the array used.

It is worth noting that the size of the matching is easy to get as the number of calls $\mathrm{try\_kuhn}$ in the main program that returned the result true. The desired maximum matching itself is contained in the array mt.

```
int n, k;
vector<vector<int>> g;
vector<int> mt;
vector<bool> used;

bool try_kuhn(int v) {
    if (used[v])
        return false;
    used[v] = true;
    for (int to : g[v]) {
        if (mt[to] == -1 || try_kuhn(mt[to])) {
            mt[to] = v;
            return true;
        }
    }
    return false;
}

int main() {
    //... reading the graph ...

    mt.assign(k, -1);
    for (int v = 0; v < n; ++v) {
        used.assign(n, false);
        try_kuhn(v);
    }

    for (int i = 0; i < k; ++i)
        if (mt[i] != -1)
            printf("%d %d\n", mt[i] + 1, i + 1);
}
```

We repeat once again that Kuhn's algorithm is easy to implement in such a way that it works on graphs that are known to be bipartite, but their explicit splitting into two parts has not been given. In this case, it will be necessary to abandon the convenient division into two parts, and store all the information for all vertices of the graph. For this, an array of lists g is now specified not only for the vertices of the first part, but for all the vertices of the graph (of course, now the vertices of both parts are numbered in a common numbering - from 1 to n). Arrays mt and are used are now also defined for the vertices of both parts, and, accordingly, they need to be kept in this state.

IMPROVED IMPLEMENTATION

Let us modify the algorithm as follows. Before the main loop of the algorithm, we will find an **arbitrary matching** by some simple algorithm (a simple **heuristic algorithm**), and only then we will execute a loop with calls to the $\mathrm{try\_kuhn}()$ function, which will improve this matching. As a result, the algorithm will work noticeably faster on random graphs - because in most graphs, you can easily find a matching of a sufficiently large size using heuristics, and then improve the found matching to the maximum using the usual Kuhn's algorithm. Thus, we will save on launching a depth-first traversal from those vertices that we have already included using the heuristic into the current matching.

For example, you can simply iterate over all the vertices of the first part, and for each of them, find an arbitrary edge that can be added to the matching, and add it. Even such a simple heuristic can speed up Kuhn's algorithm several times.

Please note that the main loop will have to be slightly modified. Since when calling the function $\mathrm{try\_kuhn}$ in the main loop, it is assumed that the current vertex is not yet included in the matching, you need to add an appropriate check.

In the implementation, only the code in the $\mathrm{main}()$ function will change:

```
int main() {
    // ... reading the graph ...

    mt.assign(k, -1);
    vector<bool> used1(n, false);
    for (int v = 0; v < n; ++v) {
        for (int to : g[v]) {
            if (mt[to] == -1) {
                mt[to] = v;
                used1[v] = true;
                break;
            }
        }
    }
    for (int v = 0; v < n; ++v) {
        if (used1[v])
            continue;
        used.assign(n, false);
        try_kuhn(v);
    }

    for (int i = 0; i < k; ++i)
        if (mt[i] != -1)
            printf("%d %d\n", mt[i] + 1, i + 1);
}
```

Another good heuristic is as follows. At each step, it will search for the vertex of the smallest degree (but not isolated), select any edge from it and add it to the matching, then remove both these vertices with all incident edges from the graph. Such greed works very well on random graphs; in many cases it even builds the maximum matching (although there is a test case against it, on which it will find a matching that is much smaller than the maximum).

Notes

- Kuhn's algorithm is a subroutine in the **Hungarian algorithm**, also known as the **Kuhn-Munkres algorithm**.
- Kuhn's algorithm runs in $O(nm)$ time. It is generally simple to implement, however, more efficient algorithms exist for the maximum bipartite matching problem - such as the **Hopcroft-Karp-Karzanov algorithm**, which runs in $O(\sqrt{n}m)$ time.
- The minimum vertex cover problem is NP-hard for general graphs. However, Kőnig's theorem gives that, for bipartite graphs, the cardinality of the maximum matching equals the cardinality of the minimum vertex cover. Hence, we can use maximum bipartite matching algorithms to solve the minimum vertex cover problem in polynomial time for bipartite graphs.

9.9.3 Hungarian algorithm for solving the assignment problem

Statement of the assignment problem

There are several standard formulations of the assignment problem (all of which are essentially equivalent). Here are some of them:

- There are n jobs and n workers. Each worker specifies the amount of money they expect for a particular job. Each worker can be assigned to only one job. The objective is to assign jobs to workers in a way that minimizes the total cost.
- Given an $n \times n$ matrix A, the task is to select one number from each row such that exactly one number is chosen from each column, and the sum of the selected numbers is minimized.
- Given an $n \times n$ matrix A, the task is to find a permutation p of length n such that the value $\sum A[i]\ [p[i]]$ is minimized.
- Consider a complete bipartite graph with n vertices per part, where each edge is assigned a weight. The objective is to find a perfect matching with the minimum total weight.

It is important to note that all the above scenarios are "**square**" problems, meaning both dimensions are always equal to n. In practice, similar "**rectangular**" formulations are often encountered, where n is not equal to m, and the task is to select $\min(n, m)$ elements. However, it can be observed that a "rectangular" problem can always be transformed into a "square" problem by adding rows or columns with zero or infinite values, respectively.

We also note that by analogy with the search for a **minimum** solution, one can also pose the problem of finding a **maximum** solution. However, these two problems are equivalent to each other: it is enough to multiply all the weights by -1.

Hungarian algorithm

HISTORICAL REFERENCE

The algorithm was developed and published by Harold **Kuhn** in 1955. Kuhn himself gave it the name "Hungarian" because it was based on the earlier work by Hungarian mathematicians Dénes Kőnig and Jenő Egerváry.
In 1957, James **Munkres** showed that this algorithm runs in (strictly) polynomial time, independently from the cost.
Therefore, in literature, this algorithm is known not only as the "Hungarian", but also as the "Kuhn-Mankres algorithm" or "Mankres algorithm".
However, it was recently discovered in 2006 that the same algorithm was invented **a century before Kuhn** by the German mathematician Carl Gustav **Jacobi**. His work, *About the research of the order of a system of arbitrary ordinary differential equations*, which was published posthumously in 1890, contained, among other findings, a polynomial algorithm for solving the assignment problem. Unfortunately, since the publication was in Latin, it went unnoticed among mathematicians.

It is also worth noting that Kuhn's original algorithm had an asymptotic complexity of $\mathcal{O}(n^4)$, and only later Jack **Edmonds** and Richard **Karp** (and independently **Tomizawa**) showed how to improve it to an asymptotic complexity of $\mathcal{O}(n^3)$.

THE $\mathcal{O}(n^4)$ ALGORITHM

To avoid ambiguity, we note right away that we are mainly concerned with the assignment problem in a matrix formulation (i.e., given a matrix A, you need to select n cells from it that are in different rows and columns). We index arrays starting with 1, i.e., for example, a matrix A has indices $A[1 \dots n][1 \dots n]$.

We will also assume that all numbers in matrix A are **non-negative** (if this is not the case, you can always make the matrix non-negative by adding some constant to all numbers).

Let's call a **potential** two arbitrary arrays of numbers $u[1 \dots n]$ and $v[1 \dots n]$, such that the following condition is satisfied:

$$u[i] + v[j] \le A[i][j], \quad i = 1 \dots n,\ j = 1 \dots n$$

(As you can see, $u[i]$ corresponds to the i-th row, and $v[j]$ corresponds to the j-th column of the matrix).

Let's call **the value f of the potential** the sum of its elements:

$$f = \sum_{i=1}^{n} u[i] + \sum_{j=1}^{n} v[j].$$

On one hand, it is easy to see that the cost of the desired solution sol **is not less than** the value of any potential.

Lemma. $sol \geq f$.

Proof

The desired solution of the problem consists of n cells of the matrix A, so $u[i] + v[j] \leq A[i][j]$ for each of them. Since all the elements in sol are in different rows and columns, summing these inequalities over all the selected $A[i][j]$, you get f on the left side of the inequality, and sol on the right side.

On the other hand, it turns out that there is always a solution and a potential that turns this inequality into **equality**. The Hungarian algorithm described below will be a constructive proof of this fact. For now, let's just pay attention to the fact that if any solution has a cost equal to any potential, then this solution is **optimal**.

Let's fix some potential. Let's call an edge (i, j) **rigid** if $u[i] + v[j] = A[i][j]$.

Recall an alternative formulation of the assignment problem, using a bipartite graph. Denote with H a bipartite graph composed only of rigid edges. The Hungarian algorithm will maintain, for the current potential, **the maximum-number-of-edges matching** M of the graph H. As soon as M contains n edges, then the solution to the problem will be just M (after all, it will be a solution whose cost coincides with the value of a potential).

Let's proceed directly to **the description of the algorithm**.

Step 1. At the beginning, the potential is assumed to be zero ($u[i] = v[i] = 0$ for all i), and the matching M is assumed to be empty.

Step 2. Further, at each step of the algorithm, we try, without changing the potential, to increase the cardinality of the current matching M by one (recall that the matching is searched in the graph of rigid edges H). To do this, the usual Kuhn Algorithm for finding the maximum matching in bipartite graphs is used. Let us recall the algorithm here. All edges of the matching M are oriented in the direction from the right part to the left one, and all other edges of the graph H are oriented in the opposite direction.

Recall (from the terminology of searching for matchings) that a vertex is called saturated if an edge of the current matching is adjacent to it. A vertex that is not adjacent to any edge of the current matching is called unsaturated. A path of odd length, in which the first edge does not belong to the matching, and for all subsequent edges there is an alternating belonging to the matching (belongs/does not belong) - is called an augmenting path. From all unsaturated vertices in the left part, a depth-first or breadth-first traversal is started. If, as a result of the search, it was possible to reach an unsaturated vertex of the right part, we have found an augmenting path from the left part to the right one. If we include odd edges of the path and remove the even ones in the matching (i.e. include the first edge in the matching, exclude the second, include the third, etc.), then we will increase the matching cardinality by one.

If there was no augmenting path, then the current matching M is maximal in the graph H.

Step 3. If at the current step, it is not possible to increase the cardinality of the current matching, then a recalculation of the potential is performed in such a way that, at the next steps, there will be more opportunities to increase the matching.

Denote by Z_1 the set of vertices of the left part that were visited during the last traversal of Kuhn's algorithm, and through Z_2 the set of visited vertices of the right part.

Let's calculate the value Δ:

$$\Delta = \min_{i \in Z_1,\, j \notin Z_2} A[i][j] - u[i] - v[j].$$

Lemma. $\Delta > 0$.

Proof

Suppose $\Delta = 0$. Then there exists a rigid edge (i, j) with $i \in Z_1$ and $j \notin Z_2$. It follows that the edge (i, j) must be oriented from the right part to the left one, i.e. (i, j) must be included in the matching M. However, this is impossible, because we could not get to the saturated vertex i except by going along the edge from j to i. So $\Delta > 0$.

Now let's **recalculate the potential** in this way:

- for all vertices $i \in Z_1$, do $u[i] \leftarrow u[i] + \Delta$,
- for all vertices $j \in Z_2$, do $v[j] \leftarrow v[j] - \Delta$.

Lemma. The resulting potential is still a correct potential.

Proof

We will show that, after recalculation, $u[i] + v[j] \leq A[i][j]$ for all i, j. For all the elements of A with $i \in Z_1$ and $j \in Z_2$, the sum $u[i] + v[j]$ does not change, so the inequality remains true. For all the elements with $i \notin Z_1$ and $j \in Z_2$, the sum $u[i] + v[j]$ decreases by Δ, so the inequality is still true. For the other elements whose $i \in Z_1$ and $j \notin Z_2$, the sum increases, but the inequality is still preserved, since the value Δ is, by definition, the maximum increase that does not change the inequality.

Lemma. The old matching M of rigid edges is valid, i.e. all edges of the matching will remain rigid.

Proof

For some rigid edge (i, j) to stop being rigid as a result of a change in potential, it is necessary that equality $u[i] + v[j] = A[i][j]$ turns into inequality $u[i] + v[j] < A[i][j]$. However, this can happen only when $i \notin Z_1$ and $j \in Z_2$. But $i \notin Z_1$ implies that the edge (i, j) could not be a matching edge.

Lemma. After each recalculation of the potential, the number of vertices reachable by the traversal, i.e. $|Z_1| + |Z_2|$, strictly increases.

Proof

First, note that any vertex that was reachable before recalculation, is still reachable. Indeed, if some vertex is reachable, then there is some path from reachable vertices to it, starting from the unsaturated vertex of the left part; since for edges of the form $(i, j),\ i \in Z_1,\ j \in Z_2$ the sum $u[i] + v[j]$ does not change, this entire path will be preserved after changing the potential. Secondly, we show that after a recalculation, at least one new vertex will be reachable. This follows from the definition of Δ: the edge (i, j) which Δ refers to will become rigid, so vertex j will be reachable from vertex i.

Due to the last lemma, **no more than n potential recalculations can occur** before an augmenting path is found and the matching cardinality of M is increased. Thus, sooner or later, a potential that corresponds to a perfect matching M^* will be found, and M^* will be the answer to the problem. If we talk about the complexity of the algorithm, then it is $\mathcal{O}(n^4)$: in total there should be at most n increases in matching, before each of which there are no more than n potential recalculations, each of which is performed in time $\mathcal{O}(n^2)$.

We will not give the implementation for the $\mathcal{O}(n^4)$ algorithm here, since it will turn out to be no shorter than the implementation for the $\mathcal{O}(n^3)$ one, described below.

THE $\mathcal{O}(n^3)$ ALGORITHM

Now let's learn how to implement the same algorithm in $\mathcal{O}(n^3)$ (for rectangular problems $n \times m$, $\mathcal{O}(n^2 m)$).

The key idea is to **consider matrix rows one by one**, and not all at once. Thus, the algorithm described above will take the following form:

1. Consider the next row of the matrix A.
2. While there is no increasing path starting in this row, recalculate the potential.
3. As soon as an augmenting path is found, propagate the matching along it (thus including the last edge in the matching), and restart from step 1 (to consider the next line).

To achieve the required complexity, it is necessary to implement steps 2-3, which are performed for each row of the matrix, in time $\mathcal{O}(n^2)$ (for rectangular problems in $\mathcal{O}(nm)$).

To do this, recall two facts proved above:

- With a change in the potential, the vertices that were reachable by Kuhn's traversal will remain reachable.
- In total, only $\mathcal{O}(n)$ recalculations of the potential could occur before an augmenting path was found.

From this follow these **key ideas** that allow us to achieve the required complexity:

- To check for the presence of an augmenting path, there is no need to start the Kuhn traversal again after each potential recalculation. Instead, you can make the Kuhn traversal in an **iterative form**: after each recalculation of the potential, look at the added rigid edges and, if their left ends were reachable, mark their right ends reachable as well and continue the traversal from them.
- Developing this idea further, we can present the algorithm as follows: at each step of the loop, the potential is recalculated. Subsequently, a column that has become reachable is identified (which will always exist as new reachable vertices emerge after every potential recalculation). If the column is unsaturated, an augmenting chain is discovered. Conversely, if the column is saturated, the matching row also becomes reachable.
- To quickly recalculate the potential (faster than the $\mathcal{O}(n^2)$ naive version), you need to maintain auxiliary minima for each of the columns:

  $$minv[j] = \min_{i \in Z_1} A[i][j] - u[i] - v[j].$$

  It's easy to see that the desired value Δ is expressed in terms of them as follows:

  $$\Delta = \min_{j \notin Z_2} minv[j].$$

  Thus, finding Δ can now be done in $\mathcal{O}(n)$.

  It is necessary to update the array $minv$ when new visited rows appear. This can be done in $\mathcal{O}(n)$ for the added row (which adds up over all rows to $\mathcal{O}(n^2)$). It is also necessary to update the array $minv$ when recalculating the potential, which is also done in time $\mathcal{O}(n)$ ($minv$ changes only for columns that have not yet been reached: namely, it decreases by Δ).

Thus, the algorithm takes the following form: in the outer loop, we consider matrix rows one by one. Each row is processed in time $\mathcal{O}(n^2)$, since only $\mathcal{O}(n)$ potential recalculations could occur (each in time $\mathcal{O}(n)$), and the array $minv$ is maintained in time $\mathcal{O}(n^2)$; Kuhn's algorithm will work in time $\mathcal{O}(n^2)$ (since it is presented in the form of $\mathcal{O}(n)$ iterations, each of which visits a new column).

The resulting complexity is $\mathcal{O}(n^3)$ or, if the problem is rectangular, $\mathcal{O}(n^2 m)$.

Implementation of the Hungarian algorithm

The implementation below was developed by **Andrey Lopatin** several years ago. It is distinguished by amazing conciseness: the entire algorithm consists of **30 lines of code**.

The implementation finds a solution for the rectangular matrix $A[1 \dots n][1 \dots m]$, where $n \le m$. The matrix is 1-based for convenience and code brevity: this implementation introduces a dummy zero row and zero column, which allows us to write many cycles in a general form, without additional checks.

Arrays $u[0 \dots n]$ and $v[0 \dots m]$ store potential. Initially, they are set to zero, which is consistent with a matrix of zero rows (Note that it is unimportant for this implementation whether or not the matrix A contains negative numbers).

The array $p[0 \dots m]$ contains a matching: for each column $j = 1 \dots m$, it stores the number $p[j]$ of the selected row (or 0 if nothing has been selected yet). For the convenience of implementation, $p[0]$ is assumed to be equal to the number of the current row.

The array $minv[1 \dots m]$ contains, for each column j, the auxiliary minima necessary for a quick recalculation of the potential, as described above.

The array $way[1 \dots m]$ contains information about where these minimums are reached so that we can later reconstruct the augmenting path. Note that, to reconstruct the path, it is sufficient to store only column values, since the row numbers can be taken from the matching (i.e., from the array p). Thus, $way[j]$, for each column j, contains the number of the previous column in the path (or 0 if there is none).

The algorithm itself is an outer **loop through the rows of the matrix**, inside which the i-th row of the matrix is considered. The first *do-while* loop runs until a free column $j0$ is found. Each iteration of the loop marks visited a new column with the number $j0$ (calculated at the last iteration; and initially equal to zero - i.e. we start from a dummy column), as well as a new row $i0$ - adjacent to it in the matching (i.e. $p[j0]$; and initially when $j0 = 0$ the i-th row is taken). Due to the appearance of a new visited row $i0$, you need to recalculate the array $minv$ and Δ accordingly. If Δ is updated, then the column $j1$ becomes the minimum that has been reached (note that with such an implementation Δ could turn out to be equal to zero, which means that the potential cannot be changed at the current step: there is already a new reachable column). After that, the potential and the $minv$ array are recalculated. At the end of the "do-while" loop, we found an augmenting path ending in a column $j0$ that can be "unrolled" using the ancestor array way.

The constant `INF` is "infinity", i.e. some number, obviously greater than all possible numbers in the input matrix A.

```
vector<int> u (n+1), v (m+1), p (m+1), way (m+1);
for (int i=1; i<=n; ++i) {
    p[0] = i;
    int j0 = 0;
    vector<int> minv (m+1, INF);
    vector<bool> used (m+1, false);
    do {
        used[j0] = true;
        int i0 = p[j0],  delta = INF,  j1;
        for (int j=1; j<=m; ++j)
            if (!used[j]) {
                int cur = A[i0][j]-u[i0]-v[j];
                if (cur < minv[j])
                    minv[j] = cur,  way[j] = j0;
                if (minv[j] < delta)
                    delta = minv[j],  j1 = j;
            }
        for (int j=0; j<=m; ++j)
            if (used[j])
                u[p[j]] += delta,  v[j] -= delta;
            else
                minv[j] -= delta;
        j0 = j1;
    } while (p[j0] != 0);
    do {
        int j1 = way[j0];
        p[j0] = p[j1];
        j0 = j1;
    } while (j0);
}
```

To restore the answer in a more familiar form, i.e. finding for each row $i = 1 \dots n$ the number $ans[i]$ of the column selected in it, can be done as follows:

```
vector<int> ans (n+1);
for (int j=1; j<=m; ++j)
    ans[p[j]] = j;
```

The cost of the matching can simply be taken as the potential of the zero column (taken with the opposite sign). Indeed, as you can see from the code, $-v[0]$ contains the sum of all the values of Δ, i.e. total change in potential. Although several values of $u[i]$ and $v[j]$ could change at once, the total change in the potential is exactly equal to Δ, since until there is an augmenting path, the number of reachable rows is exactly one more than the number of the reachable columns (only the current row i does not have a "pair" in the form of a visited column):

```
int cost = -v[0];
```

Connection to the Successive Shortest Path Algorithm

The Hungarian algorithm can be seen as the Successive Shortest Path Algorithm, adapted for the assignment problem. Without going into the details, let's provide an intuition regarding the connection between them.

The Successive Path algorithm uses a modified version of Johnson's algorithm as reweighting technique. This one is divided into four steps:

- Use the Bellman-Ford algorithm, starting from the sink s and, for each node, find the minimum weight $h(v)$ of a path from s to v.

For every step of the main algorithm:

- Reweight the edges of the original graph in this way: $w(u, v) \leftarrow w(u, v) + h(u) - h(v)$.
- Use Dijkstra's algorithm to find the shortest-paths subgraph of the original network.
- Update potentials for the next iteration.

Given this description, we can observe that there is a strong analogy between $h(v)$ and potentials: it can be checked that they are equal up to a constant offset. In addition, it can be shown that, after reweighting, the set of all zero-weight edges represents the shortest-path subgraph where the main algorithm tries to increase the flow. This also happens in the Hungarian algorithm: we create a subgraph made of rigid edges (the ones for which the quantity $A[i][j] - u[i] - v[j]$ is zero), and we try to increase the size of the matching.

In step 4, all the $h(v)$ are updated: every time we modify the flow network, we should guarantee that the distances from the source are correct (otherwise, in the next iteration, Dijkstra's algorithm might fail). This sounds like the update performed on the potentials, but in this case, they are not equally incremented.

To deepen the understanding of potentials, refer to this article.

Task examples

Here are a few examples related to the assignment problem, from very trivial to less obvious tasks:

- Given a bipartite graph, it is required to find in it **the maximum matching with the minimum weight** (i.e., first of all, the size of the matching is maximized, and secondly, its cost is minimized).
  To solve it, we simply build an assignment problem, putting the number "infinity" in place of the missing edges. After that, we solve the problem with the Hungarian algorithm, and remove edges of infinite weight from the answer (they could enter the answer if the problem does not have a solution in the form of a perfect matching).
- Given a bipartite graph, it is required to find in it **the maximum matching with the maximum weight**.
  The solution is again obvious, all weights must be multiplied by minus one.
- The task of **detecting moving objects in images**: two images were taken, as a result of which two sets of coordinates were obtained. It is required to correlate the objects in the first and second images, i.e. determine for each point of the second image, which point of the first image it corresponded to. In this case, it is required to minimize the sum of distances between the compared points (i.e., we are looking for a solution in which the objects have taken the shortest path in total).
  To solve, we simply build and solve an assignment problem, where the weights of the edges are the Euclidean distances between points.
- The task of **detecting moving objects by locators**: there are two locators that can't determine the position of an object in space, but only its direction. Both locators (located at different points) received information in the form of n such directions. It is required to determine the position of objects, i.e. determine the expected positions of objects and their corresponding pairs of directions in such a way that the sum of distances from objects to direction rays is minimized.
  Solution: again, we simply build and solve the assignment problem, where the vertices of the left part are the n directions from the first locator, the vertices of the right part are the n directions from the second locator, and the weights of the edges are the distances between the corresponding rays.
- Covering a **directed acyclic graph with paths**: given a directed acyclic graph, it is required to find the smallest number of paths (if equal, with the smallest total weight) so that each vertex of the graph lies in exactly one path.
  The solution is to build the corresponding bipartite graph from the given graph and find the maximum matching of the minimum weight in it. See separate article for more details.
- **Tree coloring book**. Given a tree in which each vertex, except for leaves, has exactly $k-1$ children. It is required to choose for each vertex one of the k colors available so that no two adjacent vertices have the same color. In addition, for each vertex and each color, the cost of painting this vertex with this color is known, and it is required to minimize the total cost.
  To solve this problem, we use dynamic programming. Namely, let's learn how to calculate the value $d[v][c]$, where v is the vertex number, c is the color number, and the value $d[v][c]$ itself is the minimum cost needed to color all the vertices in the subtree rooted at v, and the vertex v itself with color c. To calculate such a value $d[v][c]$, it is necessary to distribute the remaining $k-1$ colors among the children of the vertex v, and for this, it is necessary to build and solve the assignment problem (in which the vertices of the left part are colors, the vertices of the right part are children, and the weights of the edges are the corresponding values of d).
  Thus, each value $d[v][c]$ is calculated using the solution of the assignment problem, which ultimately gives the asymptotic $\mathcal{O}(nk^4)$.
- If, in the assignment problem, the weights are not on the edges, but on the vertices, and only **on the vertices of the same part**, then it's not necessary to use the Hungarian algorithm: just sort the vertices by weight and run the usual Kuhn algorithm (for more details, see a separate article).
- Consider the following **special case**. Let each vertex of the left part be assigned some number $\alpha[i]$, and each vertex of the right part $\beta[j]$. Let the weight of any edge (i, j) be equal to $\alpha[i] \cdot \beta[j]$ (the numbers $\alpha[i]$ and $\beta[j]$ are known). Solve the assignment problem.
  To solve it without the Hungarian algorithm, we first consider the case when both parts have two vertices. In this case, as you can easily see, it is better to connect the vertices in the reverse order: connect the vertex with the smaller $\alpha[i]$ to the vertex with the larger $\beta[j]$. This rule can be easily generalized to an arbitrary number of vertices: you need to sort the vertices of the first part in increasing order of $\alpha[i]$ values, the second part in decreasing order of $\beta[j]$ values, and connect the vertices in pairs in that order. Thus, we obtain a solution with complexity of $\mathcal{O}(n \log n)$.
- **The Problem of Potentials**. Given a matrix $A[1 \dots n][1 \dots m]$, it is required to find two arrays $u[1 \dots n]$ and $v[1 \dots m]$ such that, for any i and j, $u[i] + v[j] \le a[i][j]$ and the sum of elements of arrays u and v is maximum.
  Knowing the Hungarian algorithm, the solution to this problem will not be difficult: the Hungarian algorithm just finds such a potential u, v that satisfies the condition of the problem. On the other hand, without knowledge of the Hungarian algorithm, it seems almost impossible to solve such a problem.

> **Remark**
>
> This task is also called the **dual problem** of the assignment problem: minimizing the total cost of the assignment is equivalent to maximizing the sum of the potentials.

Literature

- Ravindra Ahuja, Thomas Magnanti, James Orlin. Network Flows [1993]
- Harold Kuhn. The Hungarian Method for the Assignment Problem [1955]
- James Munkres. Algorithms for Assignment and Transportation Problems [1957]

10.3 Schedules

10.3.1 Scheduling jobs on one machine

This task is about finding an optimal schedule for n jobs on a single machine, if the job i can be processed in t_i time, but for the t seconds waiting before processing the job a penalty of $f_i(t)$ has to be paid.

Thus the task asks to find such an permutation of the jobs, so that the total penalty is minimal. If we denote by π the permutation of the jobs (π_1 is the first processed item, π_2 the second, etc.), then the total penalty is equal to:

$$F(\pi) = f_{\pi_1}(0) + f_{\pi_2}(t_{\pi_1}) + f_{\pi_3}(t_{\pi_1} + t_{\pi_2}) + \cdots + f_{\pi_n}\left(\sum_{i=1}^{n-1} t_{\pi_i}\right)$$

Solutions for special cases

LINEAR PENALTY FUNCTIONS

First we will solve the problem in the case that all penalty functions $f_i(t)$ are linear, i.e. they have the form $f_i(t) = c_i \cdot t$, where c_i is a non-negative number. Note that these functions don't have a constant term. Otherwise we can sum up all constant term, and resolve the problem without them.

Let us fixate some permutation π, and take an index $i = 1 \dots n-1$. Let the permutation π' be equal to the permutation π with the elements i and $i+1$ switched. Let's see how much the penalty changed.

$$F(\pi') - F(\pi) =$$

It is easy to see that the changes only occur in the i-th and $(i+1)$-th summands:

$$\begin{aligned}
&= c_{\pi'_i} \cdot \sum_{k=1}^{i-1} t_{\pi'_k} + c_{\pi'_{i+1}} \cdot \sum_{k=1}^{i} t_{\pi'_k} - c_{\pi_i} \cdot \sum_{k=1}^{i-1} t_{\pi_k} - c_{\pi_{i+1}} \cdot \sum_{k=1}^{i} t_{\pi_k} \\
&= c_{\pi_{i+1}} \cdot \sum_{k=1}^{i-1} t_{\pi'_k} + c_{\pi_i} \cdot \sum_{k=1}^{i} t_{\pi'_k} - c_{\pi_i} \cdot \sum_{k=1}^{i-1} t_{\pi_k} - c_{\pi_{i+1}} \cdot \sum_{k=1}^{i} t_{\pi_k} \\
&= c_{\pi_i} \cdot t_{\pi_{i+1}} - c_{\pi_{i+1}} \cdot t_{\pi_i}
\end{aligned}$$

It is easy to see, that if the schedule π is optimal, than any change in it leads to an increased penalty (or to the identical penalty), therefore for the optimal schedule we can write down the following condition:

$$c_{\pi_i} \cdot t_{\pi_{i+1}} - c_{\pi_{i+1}} \cdot t_{\pi_i} \geq 0 \quad \forall i = 1 \dots n-1$$

And after rearranging we get:

$$\frac{c_{\pi_i}}{t_{\pi_i}} \geq \frac{c_{\pi_{i+1}}}{t_{\pi_{i+1}}} \quad \forall i = 1 \dots n-1$$

Thus we obtain the **optimal schedule** by simply **sorting** the jobs by the fraction $\frac{c_i}{t_i}$ in non-ascending order.

It should be noted, that we constructed this algorithm by the so-called **permutation method**: we tried to swap two adjacent elements, calculated how much the penalty changed, and then derived the algorithm for finding the optimal method.

EXPONENTIAL PENALTY FUNCTION

Let the penalty function look like this:

$$f_i(t) = c_i \cdot e^{\alpha \cdot t},$$

where all numbers c_i are non-negative and the constant α is positive.

By applying the permutation method, it is easy to determine that the jobs must be sorted in non-ascending order of the value:

$$v_i = \frac{1 - e^{\alpha \cdot t_i}}{c_i}$$

IDENTICAL MONOTONE PENALTY FUNCTION

In this case we consider the case that all $f_i(t)$ are equal, and this function is monotone increasing.

It is obvious that in this case the optimal permutation is to arrange the jobs by non-descending processing time t_i.

The Livshits-Kladov theorem

The Livshits-Kladov theorem establishes that the permutation method is only applicable for the above mentioned three cases, i.e.:

- Linear case: $f_i(t) = c_i(t) + d_i$, where c_i are non-negative constants,
- Exponential case: $f_i(t) = c_i \cdot e_{\alpha \cdot t} + d_i$, where c_i and α are positive constants,
- Identical case: $f_i(t) = \phi(t)$, where ϕ is a monotone increasing function.

In all other cases the method cannot be applied.

The theorem is proven under the assumption that the penalty functions are sufficiently smooth (the third derivatives exists).

In all three case we apply the permutation method, through which the desired optimal schedule can be found by sorting, hence in $O(n \log n)$ time.

10.3.2 Scheduling jobs on two machines

This task is about finding an optimal schedule for n jobs on two machines. Every item must first be processed on the first machine, and afterwards on the second one. The i-th job takes a_i time on the first machine, and b_i time on the second machine. Each machine can only process one job at a time.

We want to find the optimal order of the jobs, so that the final processing time is the minimum possible.

This solution that is discussed here is called Johnson's rule (named after S. M. Johnson).

It is worth noting, that the task becomes NP-complete, if we have more than two machines.

Construction of the algorithm

Note first, that we can assume that the order of jobs for the first and the second machine have to coincide. In fact, since the jobs for the second machine become available after processing them at the first, and if there are several jobs available for the second machine, than the processing time will be equal to the sum of their b_i, regardless of their order. Therefore it is only advantageous to send the jobs to the second machine in the same order as we sent them to the first machine.

Consider the order of the jobs, which coincides with their input order $1, 2, \dots, n$.

We denote by x_i the **idle time** of the second machine immediately before processing i. Our goal is to **minimize the total idle time**:

$$F(x) = \sum x_i \ \to \min$$

For the first job we have $x_1 = a_1$. For the second job, since it gets sent to the machine at the time $a_1 + a_2$, and the second machine gets free at $x_1 + b_1$, we have $x_2 = \max\left((a_1 + a_2) - (x_1 + b_1), 0\right)$. In general we get the equation:

$$x_k = \max\left(\sum_{i=1}^{k} a_i - \sum_{i=1}^{k-1} b_i - \sum_{i=1}^{k-1} x_i, 0\right)$$

We can now calculate the **total idle time** $F(x)$. It is claimed that it has the form

$$F(x) = \max_{k=1\dots n} K_i,$$

where

$$K_i = \sum_{i=1}^{k} a_i - \sum_{i=1}^{k-1} b_i.$$

This can be easily verified using induction.

We now use the **permutation method**: we will exchange two neighboring jobs j and $j+1$ and see how this will change the total idle time.

By the form of the expression of K_i, it is clear that only K_j and K_{j+1} change, we denote their new values with K'_j and K'_{j+1}.

If this change from of the jobs j and $j+1$ increased the total idle time, it has to be the case that:

$$\max(K_j, K_{j+1}) \le \max(K'_j, K'_{j+1})$$

(Switching two jobs might also have no impact at all. The above condition is only a sufficient one, but not a necessary one.)

After removing $\sum_{i=1}^{j+1} a_i - \sum_{i=1}^{j-1} b_i$ from both sides of the inequality, we get:

$$\max(-a_{j+1}, -b_j) \leq \max(-b_{j+1}, -a_j)$$

And after getting rid of the negative signs:

$$\min(a_j, b_{j+1}) \leq \min(b_j, a_{j+1})$$

Thus we obtained a **comparator**: by sorting the jobs on it, we obtain an optimal order of the jobs, in which no two jobs can be switched with an improvement of the final time.

However you can further **simplify** the sorting, if you look at the comparator from a different angle. The comparator can be interpreted in the following way: If we have the four times $(a_j, a_{j+1}, b_j, b_{j+1})$, and the minimum of them is a time corresponding to the first machine, then the corresponding job should be done first. If the minimum time is a time from the second machine, then it should go later. Thus we can sort the jobs by $\min(a_i, b_i)$, and if the processing time of the current job on the first machine is less then the processing time on the second machine, then this job must be done before all the remaining jobs, and otherwise after all remaining tasks.

One way or another, it turns out that by Johnson's rule we can solve the problem by sorting the jobs, and thus receive a time complexity of $O(n \log n)$.

Implementation

Here we implement the second variation of the described algorithm.

```
struct Job {
    int a, b, idx;

    bool operator<(Job o) const {
        return min(a, b) < min(o.a, o.b);
    }
};

vector<Job> johnsons_rule(vector<Job> jobs) {
    sort(jobs.begin(), jobs.end());
    vector<Job> a, b;
    for (Job j : jobs) {
        if (j.a < j.b)
            a.push_back(j);
        else
            b.push_back(j);
    }
    a.insert(a.end(), b.rbegin(), b.rend());
    return a;
}

pair<int, int> finish_times(vector<Job> const& jobs) {
    int t1 = 0, t2 = 0;
    for (Job j : jobs) {
        t1 += j.a;
        t2 = max(t2, t1) + j.b;
    }
    return make_pair(t1, t2);
}
```

All the information about each job is store in struct. The first function sorts all jobs and computes the optimal schedule. The second function computes the finish times of both machines given a schedule.

10.3.3 Optimal schedule of jobs given their deadlines and durations

Suppose, we have a set of jobs, and we are aware of every job's deadline and its duration. The execution of a job cannot be interrupted prior to its ending. It is required to create such a schedule to accomplish the biggest number of jobs.

Solving

The algorithm of the solving is **greedy**. Let's sort all the jobs by their deadlines and look at them in descending order. Also, let's create a queue q, in which we'll gradually put the jobs and extract one with the least run-time (for instance, we can use set or priority_queue). Initially, q is empty.

Suppose, we're looking at the i-th job. First of all, let's put it into q. Let's consider the period of time between the deadline of i-th job and the deadline of $i-1$-th job. That is the segment of some length T. We will extract jobs from q (in their left duration ascending order) and execute them until the whole segment T is filled. Important: if at any moment of time the extracted job can only be partly executed until segment T is filled, then we execute this job partly just as far as possible, i.e., during the T-time, and we put the remaining part of a job back into q.

On the algorithm's completion we'll choose the optimal solution (or, at least, one of several solutions). The running time of algorithm is $O(n \log n)$.

Implementation

The following function takes a vector of jobs (consisting of a deadline, a duration, and the job's index) and computes a vector containing all indices of the used jobs in the optimal schedule. Notice that you still need to sort these jobs by their deadline, if you want to write down the plan explicitly.

```
struct Job {
    int deadline, duration, idx;

    bool operator<(Job o) const {
        return deadline < o.deadline;
    }
};

vector<int> compute_schedule(vector<Job> jobs) {
    sort(jobs.begin(), jobs.end());

    set<pair<int,int>> s;
    vector<int> schedule;
    for (int i = jobs.size()-1; i >= 0; i--) {
        int t = jobs[i].deadline - (i ? jobs[i-1].deadline : 0);
        s.insert(make_pair(jobs[i].duration, jobs[i].idx));
        while (t && !s.empty()) {
            auto it = s.begin();
            if (it->first <= t) {
                t -= it->first;
                schedule.push_back(it->second);
            } else {
                s.insert(make_pair(it->first - t, it->second));
                t = 0;
            }
            s.erase(it);
        }
    }
    return schedule;
}
```